At-Compromise Security
The Case for Alert Blindness

Martin R. Albrecht[1], Simone Colombo[1], Benjamin Dowling[1], and Rikke Bjerg Jensen[2⋆]

[1] King's College London
{martin.albrecht,simone.colombo,benjamin.dowling}@kcl.ac.uk
[2] Royal Holloway, University of London
rikke.jensen@royalholloway.ac.uk

Version: 10th April 2026

Abstract We start from the observation in prior work that cryptography broadly intuits security goals – as modelled in games or ideal functionalities – while claiming realism. This stands in contrast to cryptography's attentive approach towards examining assumptions and constructions through cryptanalysis and reductions. To close this gap, we introduce a technique for determining security goals. Given that games and ideal functionalities model specific social relations between various honest and adversarial parties, our methodology is ethnography: a careful social science methodology for studying social relations in their contexts. As a first application of this technique, i.e. ethnography in cryptography, we study security *at-compromise* (neither *pre-* nor *post-*) and introduce the security goal of *alert blindness*. Specifically, in our 2024/2025 six-and-a-half-month ethnographic fieldwork with protesters in Kenya, we observed that alert blindness captures a security goal of abducted persons who were taken by Kenyan security forces for their presumed activism. We show this notion is achievable under standard assumptions by providing a construction secure in our model. We discussed both the notion and the construction with some interlocutors in Kenya.

⋆ This work was supported by UKRI under grants EP/X017524/1 and EP/X016226/2.

Contents

1 Introduction

Post-compromise security (PCS) [CCG16, BBB+19, BBL+23] enables automatic 'healing' after a state compromise. Inherently, PCS assumes an eventually passive adversary to enable this healing through the injection of fresh randomness. PCS has been established as a central design goal in the secure messaging literature. In [ABJM21] it was shown that PCS was related but incomparable to the security needs of protesters in Hong Kong in 2019/2020.[1] While they did aim to mitigate compromises, e.g. arrests, they did so *as they happened* and thus while dealing with an active adversary. On the other hand, the compromise was detected by at least one party: the person being arrested.[2]

Critically, our own 2024/2025 ethnographic fieldwork in Kenya established that responding to compromises as they happened was an overriding concern for protest organisers and frontliners targeted by Kenyan President William Ruto's security forces for arrest and abduction. *Based on these ethnographic findings*, we initiate the cryptographic study of *at-compromise security* (AtC).

1.1 Contributions

Content Warning. Our work, especially Section 4, details abductions and torture.

Our contributions are twofold. First, our work responds to the critique of cryptography put forward in the work of Blanchette [Bla12], which observed that cryptographic games and ideal functionalities model (adversarial) social relations. Given such a model, cryptography insists on formal proofs in the model which relate the security guarantees of constructions to explicit hardness assumptions and it insists on studying these hardness assumptions. However, the initial models of social relations are generally arrived at speculatively, rather than derived from a study of those relations, i.e. from data. Yet, these models claim social validity or realism, often by giving motivating scenarios. This approach is viable for, say, essentially complexity-theoretic security goals such as OWFs or Witness Encryption [GGSW13]. In contrast, our discussion of PCS above highlights that it is not necessarily the right notion for some significant contexts.[3] Moreover, current MPC or secure messaging protocols have become so multifaceted that the realism of their models cannot simply be intuited.

Our work, establishing a new security notion, avoids this dependence on intuition by relying on a rigorous social science methodology – ethnography – for studying social relations. Ethnography is a qualitative method and thus appropriate to establish *what* security means in a specific context. An ethnographic methodology favours depth over breadth and is hence ideally suited to study questions where details matter. It is a methodology that pays close attention to the *context* where security is required, allowing us to capture appropriately detailed data to reason about fine-grained security guarantees. Our first contribution is then the introduction of this methodology to cryptography for introducing realistic security notions. We set out our research design in Section 3 and believe it will have applications in other areas of cryptography.

Our second contribution is an execution of this methodology, where we establish a security notion by recourse to ethnographic findings. As we report in Section 4, our field data – especially from the Gen-Z protests in Kenya before, during and after 25 June 2024 – shows that *abductions* posed a particular threat. The data shows that the Kenyan Government used abductions and torture as a tactic to respond to these protests. Protesters responded by adopting defensive strategies aimed at *raising the alarm*, i.e. informing one or more trusted contacts that they had been taken. In particular, widespread public calls for the release of missing persons (on X) secured the release of some of them. Our data shows that several targets attribute their survival to this mechanism.[4]

[1] This work builds on [EHM17] which established a disconnect between secure messaging also developed for people in higher-risk settings and their security needs.

[2] The first work formalising PCS [CCG16] justified it using various settings: "later-removed malware, short-term physical access, and confiscation at a border crossing". These differ in detectability: the first two may go undetected, but the last is detectable by at least the device owner. This distinction appeared in prior work, e.g. while WhatsApp offers no strong PCS, it allows revoking (assumed) compromised devices [ADJ25].

[3] Moreover, a series of works have established that no mainstream deployed secure messaging system provides strong PCS guarantees [CFKN20, CJN23, ADJ24, ADJ25] and e.g [DH21, DGP22, BCC+23, DH20] explored its limitations. Recently, [CMN25] formally showed that the prevention of certain types of realistic state loss is incompatible with full PCS.

[4] This has some similarities with [AL10] which models covert adversaries.

In practice, protesters used various techniques to prepare to send alerts under duress: scheduling messages to be sent at a future time (like a 'dead man's switch'), preparing a message in advance to require only a single tap to send during an abduction or carrying a second (clean) phone at all times. They attempted to hide these mechanisms from their abductors who threatened them with violence and death. Thus, the security goal of alert *blindness* (AB) emerged in the data: the ability of parties to communicate a distress signal without alerting the adversary until a point of the parties' choosing, such as when protective mechanisms can be deployed.

In Section 5 we introduce the At-Compromise Alert Blindness (AtC-AB) security goal. Broadly, this goal is that a person must be able to send an alert using their device without being detected by their abductors. While Section 4 establishes this security need for our context, it is a priori unclear if this need can be met without relying on unrealistic assumptions.

In particular, at a first glance, AtC appears impossible: security when all secrets are compromised is a contradiction in terms. However, we side-step this 'folklore impossibility result' by distinguishing between people and their devices.[5] Building on this distinction, we obtain a design space in which not all secrets are compromised, though the remaining ones may have relatively low entropy because they must be remembered by people. Thus, our techniques borrow from low-entropy cryptography such as [BJSL11, BRT12, BS23, DHP+18, Lun19, FHH+24, BDG+24] but – as we will see – we may *also* assume high entropy secrets stored on a device.

Still, how can we compel the adversary – i.e. the abductors – to allow the abducted person to send an alert message? We address this challenge by again relying on ethnographic findings, which indicate that the adversary's primary goal during this period was to gather intelligence about the protests. The data shows that the abductors compelled their targets to reveal their personal secrets to unlock their devices. This allows us to exploit the adversary's interest in breaking confidentiality to realise AtC-AB. If the decryption keys sought by the adversary are stored remotely, breaking confidentiality requires communication; which we then leverage for a covert channel between the person and a server.

To formally reason about this approach, in Section 5.1 we introduce the *alert blindness with one message* (ABOM) (sub-)protocol between a client – a person and their device – and a server. This protocol requires a personal secret or PIN. During setup, the person chooses a *real* PIN. Under normal circumstances, they use this real PIN as the input to our protocol.[6] When they need to send an alert, they instead use *any other* PIN. The use of a different PIN then allows a server, using its state, to detect the compromise.[7]

In Section 5.3 we introduce the Key Indistinguishability with At-Compromise Alert Blindness (KIND-AtC-AB) security notion of ABOM, where KIND is meant to capture confidentiality. In particular, this allows a client to reconstruct their decryption key by running the ABOM protocol. The notion then ensures that the adversary cannot detect when a target sends an alert, and cannot learn the KIND bit without allowing the target to alert the server.[8]

However, for blindness we require that *any* PIN leads to the correct key being reconstructed on the device. In other words, to achieve blindness, we sacrifice confidentiality. This counter-intuitive decision is well-justified from our data, and we consider this justification in Section 5.4 a central contribution of this work.

We also introduce the OBLIVIOUS notion: a server does not learn anything about the key or the personal secret after an honest setup (Section 5.5).

In Section 6 we prove that our notions are satisfiable from standard assumptions by presenting a construction for the ABOM protocol that, after the setup phase, relies solely on efficient Minicrypt primitives. The

[5] This distinction also appeared in e.g. [DH21, DH20] and other prior works discussed below.

[6] While the discussion above is about device unlocking, the reader should rather think of an app-specific PIN here such as that available for Signal, see Appendix A.

[7] The tactic of using personal secrets for some form of at-compromise security is not new. Several open-source projects and products offer such a feature, typically under the name "duress", e.g. Partisan-Telegram, pam-duress, Duress. Often they aim to preserve confidentiality by deleting data or revealing partial data only, an idea that goes back at least as far as VeraCrypt's 'hidden volumes' [Ver25]. These solutions would not be secure in our context and model: they typically do not achieve blindness or assume the adversary is unaware of the duress mechanism to offer security.

[8] In Appendix B we also establish a stand-alone security guarantee of ABOM. However, this variant relies on stronger assumptions about the adversary's control of the network.

client evaluates a PRF on the personal secret and sends the evaluation – com – over a forward-secure channel to the server. The client also secret shares its encryption key k with the server. In future interactions, when the server receives a value com' different from com stored locally, the server detects the alert. Regardless of whether $\mathsf{com}' \stackrel{?}{=} \mathsf{com}$, the server then sends back its share of the key k to the client. We implemented our construction in Go (available here) and achieve a concrete communication complexity of at most 96 bytes per message after the setup phase, which fits within a single SMS – the most common communication medium in the fieldwork context. We discussed our construction with some interlocutors in Kenya, who agreed with our model (Section 3.2).

1.2 Context

> *"I am so happy. This is not just the poor, but this is everyone coming together. People had femicide, then they had the floods, so when it came to the Finance Bill they'd had enough."* (Protester, Nairobi, 25 June 2024 – FN4[9])

On 25 June 2024, a large crowd of protesters broke through the fence around the Kenyan Parliament in Nairobi. Here, they met anti-riot police, special security forces, snipers and soldiers [BBC25], who killed some of them and injured more. International diplomatic forces publicly urged the Kenyan security forces to practice "restraint" [Bri24]. Addressing the nation that evening, Ruto called the breaching of Parliament "treasonous" and protesters "organised criminals" [MBNP24].

These events marked a pivotal moment in the Gen-Z protests that swept across Kenya in June, July and August 2024. Protesters filled the streets of most larger cities, starting in Nairobi on 18 June before spreading to most counties from 20 June. The catalyst was the 2024 Finance Bill, which would raise taxes on essential goods and services that were already unaffordable for many [Nan24]. *#RejectFinanceBill2024* spread on X, which became the main collective protest platform, where protest posters and schedules were shared and collective decision-making took place. Ruto withdrew the Finance Bill 2024 on 26 June [Off24]. This shifted the focus of the protests from a rejection of this Bill to calls for Ruto's resignation (*#RutoMustGo*), rooted in long-standing grievances with what many felt were broken promises over a lack of economic growth, state-sanctioned police brutality and a disregard for the constitution [Mbu25]. Smaller-scale protests continued on most Tuesdays and Thursdays (dubbed 'protest days') until 8 August. All protests were marked by violent confrontations between the protesters and Kenyan security forces who used tear gas, water cannons, blank and live ammunition, rubber bullets and stun grenades. A large number of protesters were arrested, some were forcefully disappeared and many were killed or injured [Ken24a].

Mass protests in Kenya have a long history, but the Gen-Z protests saw the poor, the working and middle classes come together against the establishment for the first time, while many (young) women also took to the street for the first time. This composition of protesters – on the one hand young, educated and with resources and on the other young, jobless, with the perception of having no future – was considered a threat to the establishment. Many protesters expressed a feeling of *revolutionary relief*.

Abductions. The crackdown on activist networks across Kenya has continued since the June 2024 protests; including a surge in abductions carried out by Ruto's security forces.[10] From June to December 2024, there were 82 abductions [Ken24b] of people who participated in or publicly supported the protests, while 15 abductions were reported following the one-year anniversary protests on 25 June 2025 [Ken25]. A Human Rights Watch report [Hum24] points to how *abduction squads* comprised central security units, specifically the Directorate of Criminal Investigations (DCI), supported by the Rapid Deployment Unit (RPU), military intelligence, the Anti-Terrorism Police Unit (ATPU) and the National Intelligence Service (NIS). On 30 December 2024, protests against the surge in cases of abductions took place in several Kenyan cities and towns. The whereabouts of several abducted people remain unknown at the time of writing.

2 Preliminaries

We introduce the notation and the primitives that we use in this work. We further introduce ethnography.

[9] 'FN4' refers to *fieldnotes, week 4 of fieldwork*.

[10] An abducted person is kept 'off the books', which is different to an (arbitrary) arrest where the arrestee is recorded in police records.

2.1 Notation

We use $\mathbb{N}$ to denote the set of natural numbers. For $n \in \mathbb{N}$, $[n] = \{1, \ldots, n\}$. PPT abbreviates probabilistic polynomially bounded, which we use in the context of algorithms bounded in terms of the security parameter λ. We also refer to these algorithms as efficient algorithms.[11] For a probability distribution $\mathcal{D}$, $d \xleftarrow{R} \mathcal{D}$ denotes the random sampling of d from $\mathcal{D}$. For a set $\mathcal{S}$, $s \xleftarrow{R} \mathcal{S}$ denotes the uniform random sampling of s from $\mathcal{S}$. We let $\mathsf{SD}(\mathcal{D}_0, \mathcal{D}_1) \in [0, 1]$ denote the total variation distance or "statistical distance" between two distributions $\mathcal{D}_0$ and $\mathcal{D}_1$. We write $H_\infty(\mathcal{D})$ for the min-entropy of the distribution $\mathcal{D}$ and note that it expresses the success of guessing an element from $\mathcal{D}$ in a single shot. We write $|\mathcal{S}|$ for the size of the set $\mathcal{S}$ and the size of the support of the distribution $\mathcal{S}$ respectively. We denote the length of a bitstring b by $|b|$, and the empty string by ε. We write $x, y \leftarrow 0$ as a shorthand for $x \leftarrow 0; y \leftarrow 0$. We use object oriented notation to indicate membership access, e.g. $\mathsf{X.y}$ for accessing the member y of X which may be an object or a tuple.

2.2 Personal Secrets

People in our data and model interact with their devices through personal secrets, e.g. PINs or passwords. Building on prior models for password-based encryption [BRT12, BS23] and encrypted cloud storage [BDG+24], we parametrise some security games by a distribution $\mathcal{S}$ of personal secrets and sample secrets accordingly. Security then depends on the strengths of the personal secret that people select. In Definition 1, we capture the advantage of an adversary in guessing a personal secret, given a candidate secret. Looking ahead, this will be the trivial advantage of an adversary against our main security notion in Definition 17.

Definition 1 (Personal secret guessing). *We define the advantage of an adversary $\mathcal{A}$ in guessing the personal secret as*

$$\mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}(\mathcal{A}) = \Pr\left[(b' = b) \vee \big(b = 0 \wedge (x' = x_1)\big) \;\middle|\; \begin{array}{r} x_1 \xleftarrow{R} \mathcal{S}; x_0 \xleftarrow{R} \mathcal{S} \setminus \{x_1\} \\ b \xleftarrow{R} \{0,1\}; (b', x') \leftarrow \mathcal{A}(x_b) \end{array}\right] .$$

To quantify this advantage, we rely on the following proposition.

Proposition 1 (adapted from [LS20]). *Let $\mathcal{D}_0, \mathcal{D}_1$ be two discrete distributions with the same support. There is an algorithm (the Maximum Likelihood Estimator) that given a single sample x sampled as $b \xleftarrow{R} \{0,1\}; x \xleftarrow{R} \mathcal{D}_b$ decides b with probability $\frac{1}{2} + \frac{1}{2} \cdot \mathsf{SD}(\mathcal{D}_0, \mathcal{D}_1)$. Moreover, there is no algorithm that decides b with probability $> \frac{1}{2} + \frac{1}{2} \cdot \mathsf{SD}(\mathcal{D}_0, \mathcal{D}_1)$.*

We now quantify what security is possible for Definition 1.

Proposition 2. *Adopt the notation from Definition 1. For all $\mathcal{A}$*

$$\mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}(\mathcal{A}) \leq \frac{1 + \mathsf{SD}(\mathcal{S}, \mathcal{T}) + 2^{-H_\infty(\mathcal{S} \setminus \{x_b\})}}{2} =: \mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}} .$$

Proof. Let B be the event that $\mathcal{A}$ wins by satisfying the first clause of the conjunction ($b' = b$). Let G be the event that the adversary $\mathcal{A}$ wins by satisfying the second clause of the conjunction ($x' = x$). Let $\mathcal{S}_{\bar{x}}$ be the distribution obtained by outputting $y \xleftarrow{R} \mathcal{S} \setminus \{x\}$. Let $\mathcal{T}$ be the distribution obtained by sampling $x' \xleftarrow{R} \mathcal{S}$ and then sampling from $x \leftarrow \mathcal{S}_{\bar{x'}}$ and outputting x. The algorithm first runs the algorithm from Proposition 1 on x_b for $\mathcal{S}$ and $\mathcal{T}$ and returns this decision as b'. This succeeds with $\Pr[B] = \frac{1}{2} + \frac{1}{2} \cdot \mathsf{SD}(\mathcal{S}, \mathcal{T})$. It then picks a most likely element from $\mathcal{S}_{\bar{x}_b}$ and returns this as x'. Conditioned on x_1 being in the support of $\mathcal{S}_{\bar{x}_b}$, i.e. conditioned on $b = 0$, this succeeds with probability $2^{-H_\infty(\mathcal{S}_{\bar{x}_b})}$. Otherwise, this succeeds with probability zero: $\Pr[G] = 1/2 \cdot 2^{-H_\infty(\mathcal{S}_{\bar{x}_b})}$. We have:

$$\begin{aligned} \mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}(\mathcal{A}) &= \Pr[B \cup G] = \Pr[B] + \Pr[G] - \Pr[B \cap G] \\ &\leq \Pr[B] + \Pr[G] = \frac{1 + \mathsf{SD}(\mathcal{S}, \mathcal{T}) + 2^{-H_\infty(\mathcal{S}_{\bar{x}_b})}}{2} . \end{aligned}$$

[11] We could assume bounded-error quantum polynomial time (BQP) throughout without affecting our results, since our construction relies on Minicrypt primitives, after abstracting away an initial key exchange. This distinction does not add anything of technical value here, so we do not make it.

To see that this strategy is optimal, first note that the strategy for $b \stackrel{?}{=} b'$ is optimal as per Proposition 1. Then, whatever $\mathcal{A}$ does for G will not improve its chances of success if $b' = b$. When $b' \neq b$ then $\mathcal{A}$ can only win if $b = 0$, in which case guessing a fresh secret from $\mathcal{S}_{\bar{x_0}}$ maximises its chance of success. □

Remark 1. To make it easier to explore the effect of $\mathcal{S}$ on the advantage in Definition 1, we give a simple implementation in SageMath [S+25] in Appendix E.

2.3 Cryptographic Definitions

We introduce the cryptographic primitives that we use throughout this work. All key spaces that we use throughout this work implicitly depend on the security parameter 1^λ, even when not stated explicitly. All security games are implicitly parametrised by the cryptographic primitive they refer to, e.g. IND-CPA has implicit access to the key space and algorithms of the symmetric encryption scheme Π.

Definition 2 (Pseudorandom function (PRF)). *Given a key space $\mathcal{K}_{\mathsf{PRF}}$, a* pseudorandom function (PRF) *$\mathsf{F}(k, x) \to y$ inputs a key $k \in \mathcal{K}_{\mathsf{PRF}}$ and x of length $|x| = n$, and outputs y of length $|y| = \ell(n)$.*

Game IND-PRF$_{\mathsf{F}}(1^\lambda, \mathcal{A})$	Oracle Derive(x)
1 : $k \xleftarrow{R} \mathcal{K}_{\mathsf{PRF}}$; $f \leftarrow \mathsf{Func}(n, \ell(n))$	1 : $y_0 \leftarrow \mathsf{F}(k, x)$
2 : $b \xleftarrow{R} \{0, 1\}$	2 : $y_1 \leftarrow f(x)$
3 : $b' \leftarrow \mathcal{A}^{\text{Derive}}$	3 : **return** y_b
4 : **if** $b = b'$: **return** 1	
5 : **return** 0	

Figure 1: PRF security game.

Definition 3 (PRF security). *A PRF function* F *with key space $\mathcal{K}_{\mathsf{PRF}}$, input length n and output length $\ell(n)$ is* secure *if for any efficient adversary $\mathcal{A}$ the following holds*

$$\mathsf{Adv}^{\text{IND-PRF}}_{\mathsf{F}}(\mathcal{A}) = \left|\Pr[\text{IND-PRF}(1^\lambda, \mathcal{A}) \Rightarrow 1] - 1/2\right| \leq \mathsf{negl}(\lambda) \ ,$$

where the game IND-PRF *is given in Fig. 1 and $\mathsf{Func}(p, q)$ is the set of all functions mapping p-bit strings to q-bit strings.*

Definition 4 (MAC). *A* message authentication code (MAC) *scheme with key space $\mathcal{K}_{\mathsf{MAC}}$, message space $\mathcal{M}$ and tag space $\mathcal{T}$ comprises the following efficient algorithms:*

- $\mathsf{Tag}(k, m) \to t$ *takes a key $k \in \mathcal{K}_{\mathsf{MAC}}$ and a message $m \in \mathcal{M}$ and returns a tag t.*
- $\mathsf{Ver}(k, m, t) \to \mathsf{acc}$ *takes a key $k \in \mathcal{K}_{\mathsf{MAC}}$, a message $m \in \mathcal{M}$ and a tag $t \in \mathcal{T}$ and returns an acceptance bit $\mathsf{acc} \in \{\mathsf{true}, \mathsf{false}\}$.*

Definition 5 (Key derivation function (KDF)). *Given a key space $\mathcal{K}_{\mathsf{KDF}}$, a* key derivation function *(KDF) scheme consists of a single efficient algorithm $k_1, k_2 \leftarrow \mathsf{KDF}(k)$ that inputs a key $k \in \mathcal{K}_{\mathsf{KDF}}$ and outputs two keys k_1, k_2.*

Definition 6 (IND-KDF security). *A KDF scheme* KDF *provides* indistinguishability (IND-KDF) *if for any efficient adversary $\mathcal{A}$ the following holds:*

$$\mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(\mathcal{A}) = \left|\Pr[\text{IND-KDF}(1^\lambda, \mathcal{A}) \Rightarrow 1] - 1/2\right| \leq \mathsf{negl}(\lambda) \ ,$$

where the game IND-KDF *is given in Fig. 2.*

Game IND-KDF$(1^\lambda, \mathcal{A})$

1 : $k \xleftarrow{R} \mathcal{K}_{\mathsf{KDF}}$
2 : $b \xleftarrow{R} \{0,1\}$
3 : **if** $b = 0$: $(k_1, k_2) \leftarrow \mathsf{KDF}(k)$
4 : **else** : $(k_1, k_2) \xleftarrow{R} \mathcal{K}^2$
5 : $b' \leftarrow \mathcal{A}(k_1, k_2)$
6 : **return** $(b = b')$

Figure 2: IND-KDF security game for a two-output KDF.

Definition 7 (Symmetric encryption). *Given a key space $\mathcal{K}_\Pi$, a* symmetric encryption *scheme Π comprises the following PPT algorithms:*

- $\mathsf{Enc}(k, m) \to \mathsf{ct}$ *takes a key $k \in \mathcal{K}_\Pi$, plaintext m, and returns a ciphertext* ct*.*
- $\mathsf{Dec}(k, \mathsf{ct}) \to m$ *takes a key $k \in \mathcal{K}_\Pi$, ciphertext* ct*, and outputs a plaintext m.*

Definition 8 (Correctness of symmetric encryption). *A symmetric encryption scheme Π is* correct *if for all keys $k \in \mathcal{K}_\Pi$ and all plaintexts m the following holds:*

$$\Pr[\mathsf{Dec}(k, \mathsf{Enc}(k, m)) \to m] = 1 \ ,$$

where the probability is taken over all the random coins used by the algorithms of the scheme.

Definition 9 (IND-CPA security). *A symmetric encryption scheme Π provides* indistinguishability under chosen-plaintext attack (IND-CPA) *if for any efficient adversary $\mathcal{A}$*

$$\mathsf{Adv}^{\text{IND-CPA}}_{\Pi}(\mathcal{A}) = \left|\Pr[\text{IND-CPA}(1^\lambda, \mathcal{A}) \Rightarrow 1] - 1/2\right| \leq \mathsf{negl}(\lambda) \ ,$$

where the game IND-CPA is given in Fig. 3.

Game IND-CPA$(1^\lambda, \mathcal{A})$

1 : $k \leftarrow \mathcal{K}_\Pi$
2 : $b \xleftarrow{R} \{0,1\}$
3 : $b' \leftarrow \mathcal{A}^{\text{Enc}}$
4 : **return** $(b = b')$

Oracle Enc(m_0, m_1)

1 : **if** $|m_0| \neq |m_1|$: **return** $\perp$
2 : **return** $\mathsf{Enc}(k, m_d)$

Figure 3: IND-CPA security game.

Definition 10 (One-time AEAD). *A* one-time authenticated encryption with associated data (AEAD) *scheme* AEAD *defines a key space $\mathcal{K}_{\mathsf{AEAD}}$, an associated data space $\mathcal{AD}$, a message space $\mathcal{M}$, and the following PPT algorithms:*

- $\mathsf{Enc}(k, a, m) \to c$ *takes a key $k \in \mathcal{K}_{\mathsf{AEAD}}$, associated data $a \in \mathcal{AD}$, plaintext $m \in \mathcal{M}$, and returns a ciphertext c.*
- $\mathsf{Dec}(k, a, c) \to (\mathsf{acc}, m)$ *takes a key $k \in \mathcal{K}_{\mathsf{AEAD}}$, ciphertext c, and outputs an acceptance bit* $\mathsf{acc} \in \{\mathsf{true}, \mathsf{false}\}$ *and a plaintext m. We assume that $m \leftarrow \perp$ when* $\mathsf{acc} = \mathsf{false}$*.*

We assume that all randomness stems from the key $k \in \mathcal{K}_{\mathsf{AEAD}}$. We do not need additional randomness or nonces because we only rely on one-time security as we discuss below.

Definition 11 (Correctness of one-time AEAD). *A one-time AEAD scheme* AEAD *is* correct *if for all* $k \in \mathcal{K}_{\mathsf{AEAD}}$*,* $a \in \mathcal{AD}$ *and* $m \in \mathcal{M}$*, the following holds*

$$\Pr[\mathsf{Dec}(k, a, \mathsf{Enc}(k, a, m)) \to m] = 1 \ ,$$

where we compute the probability over the randomness that stems from the key $k \in \mathcal{K}_{\mathsf{AEAD}}$*.*

Definition 12 (OT-IND$-CPA security of one-time AEAD). *A one-time AEAD scheme* $\mathsf{AEAD} = (\mathsf{Enc}, \mathsf{Dec})$ *provides* indistinguishability from random under chosen plaintext attack (OT-IND$-CPA) *if for any PPT adversary* $\mathcal{A}$ *the following holds:*

$$\mathsf{Adv}^{\text{OT-IND\$-CPA}}_{\mathsf{AEAD}}(\mathcal{A}) = \left|\Pr[\text{OT-IND\$-CPA}(1^\lambda, \mathcal{A}) \Rightarrow 1] - 1/2\right| \leq \mathsf{negl}(\lambda) \ ,$$

where the game OT-IND$-CPA *is given in Fig. 4.*

Game OT-IND$-CPA$(1^\lambda, \mathcal{A})$	Oracle $\textsc{Enc}(a, m)$
1 : $k \xleftarrow{R} \mathcal{K}_{\mathsf{AEAD}}$	1 : **if** $m^* \neq \varepsilon$: **return** $\perp$
2 : $b \xleftarrow{R} \{0,1\}$	2 : $m^* \leftarrow m$
3 : $m^* \leftarrow \varepsilon$	3 : $c_1 \leftarrow \mathsf{Enc}(k, a, m)$
4 : $b' \leftarrow \mathcal{A}^{\textsc{Enc}}$	4 : $c_0 \xleftarrow{R} \{0,1\}^{\lvert c_1 \rvert}$
5 : **return** $(b = b')$	5 : **return** c_b

Figure 4: OT-IND$-CPA security game.

Definition 13 (Existential unforgeability of one-time AEAD). *A one-time AEAD scheme* $\mathsf{AEAD} = (\mathsf{Enc}, \mathsf{Dec})$ *provides* existential unforgeability under chosen message attack (OT-EUF-CMA) *if for any PPT adversary* $\mathcal{A}$ *the following holds:*

$$\mathsf{Adv}^{\text{OT-EUF-CMA}}_{\mathsf{AEAD}}(\mathcal{A}) = \left|\Pr[\text{OT-EUF-CMA}(1^\lambda, \mathcal{A}) \Rightarrow 1]\right| \leq \mathsf{negl}(\lambda) \ ,$$

where the game OT-EUF-CMA *is given in Fig. 5.*

Game OT-EUF-CMA$(1^\lambda, \mathcal{A})$	Oracle $\textsc{Enc}(a, m)$
1 : $k \xleftarrow{R} \mathcal{K}_{\mathsf{AEAD}}$; $q \leftarrow \varepsilon$	1 : **if** $q \neq \varepsilon$: **return** $\perp$
2 : $(a, c) \leftarrow \mathcal{A}^{\textsc{Enc}}$	2 : $c \leftarrow \mathsf{Enc}(k, a, m)$
3 : $(\mathsf{acc}, m^*) \leftarrow \mathsf{Dec}(k, a, c)$	3 : $q \leftarrow (a, c)$
4 : **return** $(\mathsf{acc} \wedge (a, c) \neq q)$	4 : **return** c

Figure 5: OT-EUF-CMA security game.

2.4 Ethnography

Ethnography is a field of study originating in social and cultural anthropology, where it serves as "an integration of first-hand empirical investigation and the theoretical and comparative interpretation of social organisation and culture" [HA19, p.1], through fieldwork. Ethnography relies heavily on participant observation, deriving deep and rich knowledge and understanding of people, their lived experiences, perspectives

and practices as these manifest in people's *naturally occurring settings* [Bre00] – often referred to as *the field*. To achieve this level of insight, ethnographers, aided by local gatekeepers and interlocutors,[12] immerse themselves within specific social settings for prolonged periods of time, studying the dynamics and complexities of social life, relations and structures.

By being present in the field, ethnographers capture what people *do* rather than relying solely on what people say they do. Specifically, and as articulated by Herbert [Her00, p.550], ethnography is uniquely placed to "unearth what the group (under study) takes for granted", thereby revealing "the knowledge and meaning structures that provide the blueprint for action".

In essence, ethnography is grounded in a commitment to developing an understanding of other people's daily activities, through a systematic collection, analysis and interpretation of empirical data. Ethnographic data is both messy and extensive and it takes many forms, but generally includes field observations and reflections captured in the form of extensive fieldnotes, interview notes, recordings and transcripts, textual and visual material and objects present in the field as well as materials co-produced through participatory methods of engagement.

Ethnography avoids generalisation. While it is often critiqued for being overly detailed in its descriptions – *thick descriptions* in ethnographic terms [Gee08] – the goal of ethnography is not to gather and relay fragmented narratives and (personal) impressions. Instead, through iterative and systematic analysis, ethnographers strive to make sense of the data to convey meaning, while retaining the complexities of the relations, structures and practices of the field settings. The analytical process often involves extensive coding of large and messy datasets to generate patterns, concepts and categories to support theory building. In Section 3.2, we describe our analytical process in detail.

In most qualitatively driven security research, interviews and focus groups are the dominant research methods. While ethnography is a qualitative research method, there is, in the words of a key figure in ethnography Paul Atkinson, "a world of difference between a commitment to long-term field research [...] and the conduct of a few interviews or focus groups" [Atk15, p.3]. In particular, extended fieldwork aids deeper and richer understanding of how security is experienced and discussed within adversarial settings, ensuring that the research remains sensitive to the higher-risk context while also recognising that establishing rapport and trust in these settings requires time and commitment.

3 Methodology

Here we establish our technique; namely, bringing ethnography to cryptography.[13] Ethnography is a methodology that comprises both *data collection methods* and *analytical steps*. It is through the application of this technique, i.e. ethnography in cryptography, that we arrived at KIND-AtC-AB.

To study cryptography, i.e. computing under adversarial conditions, our research commenced with ethnographers and cryptographers jointly selecting contexts that suggested such conditions – but at a level of violence that does not render cryptography inessential or make ethnography impossible. Our engagement with and analysis of these contexts is then ethnographic (Section 2.4) since the depth of this methodology allows us to *establish* security notions. Our research did not set out to study at-compromise security but this focus emerged from the data through iterative analytical steps (Section 3.2). One team member (hereafter: *the fieldworker*) conducted extensive fieldwork within activist networks and protest settings in Kenya over a period of six and a half months. Concretely, our fieldwork comprised (1) a two-week fieldwork scoping trip to Nairobi in April 2024 (to meet gatekeepers and explore the site), (2) four-month fieldwork in Nairobi (with visits to Mombasa and Kisumu) in June–September 2024 and (3) two-month fieldwork in Nairobi in January–February 2025 (Section 3.1).

Researcher Positionalities. This work requires us to articulate our positionalities with respect to the context under study. First, this is necessary because who the fieldworker is and how they are seen will shape what they

[12] In ethnography, 'interlocutor' refers to a fieldwork-based relation, a person with whom (and about whom) we conduct research. This is different to 'gatekeeper', who is someone that might act as a point of access to specific settings and/or interlocutors. 'Participant', on the other hand, refers to someone who might be involved in interviews but not necessarily part of the researcher's daily life in the field.

[13] We note that our approach deviates from the disciplinary norms of much ethnography where no focus, say, on cryptography, is established *before* entering the field.

will learn through their interactions in the field. The fieldworker is a European researcher and a woman. She is an experienced ethnographer who researches information security as it relates to people in often higher-risk settings. Second, given that our data is ethnographic, it would be neither ethical nor feasible to publish our raw data (Section 3.4). A consequence of this is that greater emphasis needs to be placed on explaining how we analysed this data to present the findings in Section 4; this part of our analysis cannot be replicated.[14] Different pieces of data will catch the attention of different researchers based on their respective experiences. Our individual identities shaped our analysis and interpretation. For example, while we claim that KIND-AtC-AB is correct (Claim 1), we do not claim that it was the sole possible formalisation to emerge from our fieldwork and data. Thus, ethnography – in contrast to positivist concerns of trying to eliminate the researcher's impact on the data – acknowledges that researchers are part of the social world they study [HA19], their presence and interpretations need to be acknowledged but cannot be eliminated. The research team also includes three cryptographers, all with prior work in secure messaging, some with a history of political activism. None of the authors are Kenyan nationals.

3.1 Ethnographic Data Collection

Our ethnographic methods combined active participant observation in activist and protest settings and ethnographic interviews with protesters in Kenya. The fieldwork coincided with the Gen-Z protests in June, July and August 2024, as well as their continued aftermath (Section 1.2). Thus, the research was conducted during a period marked by heightened security among interlocutors and participants as they navigated their own security and safety, developed new protest strategies and planned activities while supporting those injured or arrested.

Active Participant Observation. The fieldworker conducted participant observation at 11 large- and small-scale protests in Nairobi, including 18, 20 and 25 June 2024 (Appendix C), as well as in activist meetings, protest-planning sessions, strategic discussions, community gatherings and organising meetings. Outside such events, the fieldworker spent most days within activist settings in informal settlements, participating in day-to-day activities such as community engagements, hospital and prison visits, memorials and group meetings. The fieldwork took the form of what anthropologist Clifford Geertz [Gee98] refers to as "deep hanging out" – "localized, long-term, close-in, vernacular field research" – with the fieldworker adopting an *active participant* position. This was grounded in informal interactions to uncover the meanings that people attached to their actions during this time, and how they aimed to protect themselves and each other.

Field access was established through local gatekeepers and re-established throughout the fieldwork to ensure openness in all field relations [MU14] which enabled long-term immersion in a specific social movement in Kenya. The fieldworker built rapport with members of this movement, many of whom were involved with or members of other activist groups. Through these field relations, access to a wide network of activists developed organically. Full fieldnotes were taken throughout the fieldwork [EFS11], capturing daily observations and the fieldworker's initial reflections and interpretations.

Ethnographic Interviews. The fieldworker conducted ethnographic interviews with 77 protesters during the fieldwork, including organisers, frontliners, first-time and seasoned protesters, and participants who had been abducted due to their protest involvement. Interviews comprised a mix of individual (51) and group (5) interviews of between three and six participants (Appendix C), depending on participant preferences and to enable observations of group dynamics and shared experiences. They were loosely structured around a topic guide to provide some consistency across interviews, while allowing new and spontaneous topics to emerge [KD23]. However, the topic guide was not a static document, but continued to evolve throughout the fieldwork. All interviews were audio-recorded with the explicit consent of each participant and lasted between 43 minutes and over two hours.

Participants were recruited through interlocutors and day-to-day interactions during the fieldwork. Ethnographic interviews are distinct from other forms of interviews used in qualitative research. They are conducted within the specific field setting and are shaped by an established relationship between the fieldworker and participants, which typically means they are founded in a higher degree of mutual trust [RG22].

[14] This is analogous to, say, a measurement study on the adoption of TLS 1.3; similar results can be obtained but the result cannot be replicated later as adoption changes.

3.2 Data Analysis

Our analysis takes a grounded theoretical approach [GS99], which is common to ethnography and based on inductive reasoning where theory is developed from empirical data – rather than from preconceived theoretical positions or hypotheses. This means we did not set out to test the validity of an intuited security notion, but rather analysed the data to uncover one. In the present work, we specifically report on findings relating to abductions, which also shaped the analytical conversations between ethnography and cryptography.

Throughout, between and after the fieldwork, we iteratively and jointly analysed the data across ethnography and cryptography. Concretely, the fieldworker initially generated insights from the ethnographic fieldwork through high-level written summaries. We then discussed these insights across ethnography and cryptography through analytical sessions, which led to initial observations about 'at-compromise security' contained in the data. This became the point of departure for cryptography, as it enabled some cryptographic work to be initiated. Following these initial steps, a full analysis of the ethnographic data (described below) took place in parallel with the maturing of the cryptographic work. At this stage, the ethnography was also continuously queried to establish if cryptographic assumptions and requirements were satisfied in the ethnographic data. Finally, cryptography reflected back the security notion to ethnography. Through this process the question 'what is at-compromise security in this context' and its answer (AtC-AB) emerged iteratively. This is inherent in ethnography, where analysis is conducted reflexively at several stages of the research.

The ethnographic analytical process was driven by a series of iterative coding cycles:[15] repeated readings of the data used to assign themes and uncover insights, manually coded using qualitative data analysis software NVivo 14 [Lum23]. Given the exploratory nature of our research and the large volume of data, the first coding cycle adopted an *initial coding* approach to break down the data into discrete parts, enabling closer examination while remaining "open to all possible theoretical directions indicated by [our] reading of the data" [Cha06, p.46]. The second coding cycle used *process coding* in combination with *emotion coding* to capture the (sub)processes of action and interaction within the field setting (e.g. planning and action, responses to specific situations) as well as the feelings that accompanied action (e.g. worries, relief). Following these coding cycles the generated codes were grouped into higher-level categories of similar content.

The KIND-AtC-AB game given in Fig. 8 – produced in conversation between cryptography and ethnography, as described above – is a cryptographic formalisation of the insights produced through this analysis.

Bringing AtC-AB *to the Field.* In our final analytical step, we brought our security notion back to six local interlocutors, who were central to our fieldwork in Kenya, and discussed it with them in separate conversations. These lasted between one and three hours. Two of these interlocutors had been abducted in June and July 2024. The other four interlocutors were frontliners and leading organisers in different groups, all of whom had experienced being targeted by the Kenyan security forces and one interlocutor had been moved to a safe house for their protection. This was done to establish the extent to which AtC-AB captured interlocutors' concerns and also the practicalities of our proposal. Specifically, we discussed: giving up confidentiality to raise an alarm during abductions, that every wrong PIN triggers an alarm and who might run the server.

Giving up confidentiality was considered a necessity and a form of protection during an abduction. Referring to their own experience or that of others, interlocutors explained how abductors would use information they already knew about an abducted person to test the extent to which they were telling the truth, i.e. whether they were cooperating. Non-cooperation could (and often would) lead to torture and in some cases death. Thus, in the moment of abduction, protecting confidentiality was a risk they could not take.

All interlocutors raised the concern that with every incorrect PIN triggering an alarm, the likelihood of alarms being triggered by accident was significant. While some suggested an 'alarm PIN', they feared that abductors would then extract two PINs from them. They also voiced the concern that they might not remember a second PIN in the moment of abduction. One of the abducted interlocutors exemplified how they had forgotten their phone unlock PIN when they were captured, which had led to extensive beatings. In our discussions, interlocutors suggested different ways to overcome false positives, which all centred on utilising existing social protocols of check-ins within activist groups.

Our fieldwork uncovered an extensive and highly connected network of organisations supporting activists in Kenya, which played a significant role during and after the protests. For example, they established support

[15] Here *coding* roughly means to 'tag' the data under certain key phrases.

hotlines and a situation room that monitored police violence and dispersed medical and legal teams. The interlocutors pointed to these organisations, trusted among activists, when discussing who might run the server. Establishing what happens once an alarm is triggered is the focus of follow-up co-design work with interlocutors.

3.3 Proof of Feasibility

We establish that KIND-AtC-AB is efficiently achievable from standard assumptions following the standard approach in the literature: a construction accompanied by a proof reducing security to that of standard building blocks. We note that this process, as is usual, produced further refinements to Definition 17. For example, the advantage statement therein is an artefact of this activity.

3.4 Ethics Considerations

We received full ethics approval from the fieldworker's institutional Research Ethics Committee (REC) before the start of the research. Studying activist networks and protest settings in Kenya over extended periods of time meant that the research was considered 'high risk' and an extensive risk assessment was carried out and validated through the fieldworker's institutional Health & Safety (H&S) office. In particular, we put in place safety protocols and risk mitigations specific to the context under study, which included studying live protests *in situ* as well as engaging with activist networks being monitored by the Kenyan security forces, and conducting fieldwork in informal settlements. When the Gen-Z protests became a reality in June 2024, the fieldworker also established a daily check-in protocol with their institutional H&S office. This was in addition to a detailed data management plan, which centred on data minimisation and compartmentalisation within the research team. All data was digitised and securely stored before the fieldworker left the field setting, with the fieldworker not having access to field data while crossing international borders to avoid accidental or forced data leakage.

The research design was developed in close collaboration with local gatekeepers to ensure that it was sensitive to the context under study. This was facilitated through multiple conversations and a scoping trip before the start of the full fieldwork (Section 3), which also informed the participant information sheet and topic guide.

Specific ethics considerations were also given to day-to-day interactions with interlocutors and interviews with participants during the fieldwork. Before *ethnographic interviews*, participants were given an information sheet, while the fieldworker explained the research and what participation would involve. Participants also had the opportunity to ask questions prior to giving informed consent to take part. All participation was limited to persons who were at least 18 years old and no participants received financial compensation to take part. This was important to avoid anyone feeling compelled to partake and possibly accepting increased risks as a result. However, we covered travel expenses and provided refreshments during participant engagements to ensure that no participant was out of pocket as a result of their participation. We refrain from reporting participant demographics to minimise the risk of de-anonymisation and we do not name specific groups, movements or networks of activists. We have also omitted specific locations in the reporting of our findings given the context of heightened state surveillance. We strove to protect participants' anonymity throughout the research and treated their information confidentially at all stages.

During *active participant observation*, interlocutors were informed of the reasons for the fieldworker's presence. In each setting, such as activist meetings, planning sessions, strategic discussions, placard-making sessions, community gatherings and organising meetings, the fieldworker stressed that no contributions would be attributed to an individual or a group, while the specific location would remain undisclosed. While the fieldworker's presence in these settings became more 'natural' during the course of the fieldwork, underpinned by a growing familiarity between the fieldworker and interlocutors, the fieldworker continuously reminded everyone of her researcher position to ensure that everyone was comfortable with her presence. Interlocutors could also inform the fieldworker in private should they wish not to be included in the research; this, however, was never the case. During large- and small-scale protests, in which the fieldworker conducted active participant observation (Table 2), established practices of overt ethnographic research were followed [MU14]. This included being open about the fieldworker's reason for being present in protests, truthfully answering any questions protesters might have and only noting non-identifying details in the fieldnotes.

3.5 Limitations and Future Work

The ethnographic data uncovered security concerns and practices that related to neither abductions nor cryptography. Thus, the social foundations presented in Section 4 are limited in scope. However, the ethnographic grounding necessitates this limitation. Our ethnographic data is extensive and, as is common in ethnography, it is messy, requiring extensive analysis through which diverse findings emerged. We focus on abductions in this work as our analysis brought to light the significance of *at-compromise security* for these people in this setting. Further, all interviews were conducted in English, a language familiar to both participants and the fieldworker. Swahili was the primary language in some group meetings and daily conversations during the fieldwork, of which the fieldworker only had some basic knowledge. Thus, in some situations the fieldworker relied on interlocutors to translate anything that may otherwise have been missed.

We suspect that other AtC notions are possible in other settings, such as during arrests, including notions that maintain confidentiality. We consider analysing those settings and modelling the arising security goals as future work.

Our model ends with detection and does not capture subsequent actions, which included successful public appeals for the release of abducted people (Section 4.3). Understanding and designing such mechanisms is pressing future work.

As per Kerckhoffs' principle, we assume that the adversary *may* know the protocol – strongly, we assume they certainly *do*. We rely on a violent adversary recognising that they must allow a message to go through to realise their intelligence goals and, critically, on this knowledge to dissuade them from escalating their violence further. While we think this condition can be fulfilled in practice, this is an important limitation of our work – and of others modelling cryptography in the presence of violence, sometimes flippantly referred to as 'rubber hose cryptanalysis'. A possible augmentation is for our alert protocol to piggyback on necessary communication, like fetching fresh messages. We omitted this here to focus on the new notion clearly and directly.

We do not address the usability of our construction. In particular, the usability of PINs and other personal secrets is an ongoing area of research, but the stakes of typos are much higher in our protocol than in related work (see Appendix A). More broadly, we do not consider deployment or user experience. Our work reaches the stage of a reference implementation and we leave integration, usability and deployment as directions for future work.

4 Social Foundations

The ethnographic fieldwork uncovered the *social foundations* of the cryptographic notions introduced in Section 5: the security concerns and practices among protesters in Kenya at a time when the use of plain-clothed officers and unmarked police vehicles to forcibly disappear protesters provoked a growing fear among activists. Such concerns were typically voiced in connection with the violence exerted by the DCI and NIS.[16]

4.1 Preparing for Abductions

The fieldwork revealed how the risk of being abducted was not considered a security concern before the Gen-Z protests, while arrests and physical violence were expected during protests. P51, who was involved in the planning of the initial June 2024 protests, exemplified this:

> "No, no, no. There wasn't a concern at that point [...] We have done demonstrations before. The worst that they [the security forces] can do ... they normally arrest you, then they charge you and then give you a cash bail or whatever. It had never reached a point where they could torture people or go beyond that. You see now, for us, this is something that was normal to us. Abductions weren't."

P51 was abducted at the start of the Gen-Z protests, at a time when the threat of abductions became an immediate reality for protesters, particularly those considered to be organisers or frontliners. This was also observed during strategy meetings in activist groups in the lead-up to 25 June. The abductions following

[16] We refer to the DCI (Directorate of Criminal Investigations) and NIS (National Intelligence Service) as 'the security forces' for the remainder of this work.

the 18 and 20 June protests marked a shift in their protest planning and security outlook: "We have never experienced anything like this [abductions] before, so tomorrow we have to stay together" (FN4). Abductions gave rise to security strategies that broadened the collective security; protesters relied on each other and their networks for protection. This was, for example, seen in how they sought to move in groups with established check-in protocols, temporarily moved in with friends or family and avoided staying in one place for more than a few days. Some had a second (clean) phone and several phone numbers which were often linked to someone else's ID, while they relied on support organisations for legal and medical assistance. We exemplify concrete practices developed to protect during abductions in Section 4.3, but here emphasise how the emergence of abductions as an immediate threat to protesters shifted how they approached their security.

A discussion with frontliners in Mombasa in July 2024 revealed how the fear of abduction shaped their daily movements and interactions. P24 expressed how "they [security forces] are looking for frontliners, and as long as they are looking for frontliners they will try to abduct us [...] the moment we relent, they will pick us one by one." This was a concern that many frontliners lived with and tried to navigate every day. P18 stressed that "he [Ruto] will just kill everyone [frontliners] as soon as we stop". This was underscored by the feeling among protesters that the Kenyan Government was "shell shocked" (FN3) by the protests and they devised their security practices within this understanding.

The fear of abduction was echoed across activist groupings and directly shaped how security was devised for the group and the protective measures protesters took. The need to "re-think our security and consider how we better protect ourselves and others" (FN6) was a pivotal point of reflection in most group meetings during and in the immediate aftermath of the June 2024 protests. P9, who was observed to be constantly switching between SIM cards in their phone during a group meeting, shared how "the moment I turn on my SIM card, there they are looking for me [...] the other day, they still tried to abduct me but luckily a woman snatched me first and hid me in her house."

During this time the Kenyan security forces systematically targeted influential protesters, which led to elaborate practices among those who considered themselves a potential target of abduction. P16, whose home was raided by what P16's neighbours assumed to be security operatives, exemplified this:

> "I have numerous pseudo accounts. You can now text me on WhatsApp and I might reply you now. The next minute you text me, that number is not on WhatsApp [...] You might call me now and we converse using this number. The next time you call me, I'm offline and I call you with another number. Now, when you call me and I'm offline, I'll get a notification that you have tried to call me. If I know you, I'll call you back [...] So, I'm the only person who is in the position to call you."

Such practices were often developed in response to protesters' own experiences of being tracked. For example, P37, who was abducted in the early hours of 25 June, explained how they were "living in fear for about a week" prior to their abduction: "I was entirely in hiding, but they found me even though my phone had been off for, like, two days." P37 had stopped using their phone to protect against being tracked through their phone number, while they also started to use cash instead of the, in Kenya, dominant mobile-phone based payment system M-PESA. Instead, P37 used their laptop: "I had my apps on my laptop." P60 and P72, who were both abducted, also spoke about how they had been trailed by what they considered to be unmarked DCI vehicles in the days and hours leading up to their abductions; others experienced receiving threatening phone calls and messages from unknown numbers in the time before being taken.

4.2 The Contours of Abductions

Throughout the fieldwork reports of abductions circulated through activist networks and on platforms such as X, where calls for their release quickly spread (Section 4.3). While some abducted people were taken to a police station after a couple of days and released, others were 'dumped' in unknown locations and found alive by locals. Some were also killed, as starkly captured in an FN5 entry:

> "Over the weekend, five bodies were retrieved from swamps and quarries, people who had been missing since the first protests and most with bullet wounds. Bodies of missing protesters have also been found in some morgues."

Our data contains the experiences of eight abducted persons who were taken at different points in the protests. Many chose not to speak publicly about their abductions out of fear for potential repercussions. Identifying and engaging with abductees was made possible because the fieldworker was embedded in prominent activist networks. P22 and P51 were abducted at the start of the protests, while P31, P37, P49 and P61 were captured during a series of coordinated abductions in the early hours of 25 June where security forces abducted up to 50 protesters [Ose24]. P60 was abducted during a protest in July and P72 was taken during intensified crackdowns on activists in the aftermath of the protests. P22, P31, P37, P49, P61 and P72 were taken to unknown locations where they were held for days, while P51 and P60 were driven around by their abductors. Their experiences shed light on the observed behaviours, motivations and capabilities of the adversary during these abductions. As is the case with ethnographic work, our data is focused on the period in which the fieldwork took place (Section 3.1).

The Abduction. The experiences of those abducted reveal a clear pattern that characterised their abductions. They all spoke of how they were taken by between four and eight masked individuals, who did not identify themselves and, in some cases, arrived in a convoy of unmarked vehicles. For most abductees (except P51 and P60) their abductors captured them at or near their home, and most under the cover of darkness. They had their phones taken and asked to unlock them (⧫ 1),[17] except P72 who smashed theirs (⧫ 2). They were blindfolded, their hands were tied and they were shuffled into one of the vehicles, where they sat between two of their abductors. They drove around for between 20 minutes and three hours before arriving at an unknown location. They were dumped in a room, while still blindfolded and with their hands tied. For P22, who was one of the first to be abducted, it came as "a surprise" when they opened their door and saw "four guys in balaclavas":

> "[...] I was being beaten. The other guy went for my laptop and my phone [...] He wanted me to unlock my phone, but I refused (⧫ 3). And the whole time I was asking why they were here and what they wanted but they silenced me with more beatings [...] I was blindfolded and put into a car and we drove around for maybe two hours [...] After some time the car stopped and I was being taken out of the car [...] I was taken down some stairs into a very cold room. They told me to lie on my stomach."

Only interrupted by the interrogations, P22 was left in this position for two days and nights. The initial handcuffs were changed to cable straps that were gradually tightened, making their hands numb. P22's abduction resembles that of several other protesters who were abducted during this period. They all spoke about being blindfolded and tied up while being left either on a concrete floor or a mattress in a cold room (in what "felt like a house" (P37)), not knowing what would happen or where they were. P72 was captured near their home in the aftermath of the protests:

> "They just dragged me off, you know, I was blindfolded, thrown in the backseat and they started driving [...] At one point, I tried asking them what do they want, and where they're taking me. And at one point, [they] put a gun in my mouth until I shut up. So, they didn't talk to me the whole time."

P72's abduction lasted two days. All abductees expressed the same fear: they would be killed if they did not cooperate with their abductors (⧫ 4). P60, who had been arrested numerous times in previous protests, expressed being "very afraid" when they were put into the backseat of their abductors' vehicle – a vehicle which they had seen "trail me for almost four days"(⧫ 5). P60 was taken to an isolated location and beaten, reflecting that "if I do anything now to upset these guys, I'm dead." P51 was placed in the boot of their abductors' vehicle after they had asked where they were being taken: "So I'm there now. I don't know what's happening. Just there in the dark. Just praying. I'm shaking. Thinking that I had asked a wrong question and now paying for it." P72 "thought this was the end" for them when they were unable to answer specific questions about their protest mobilisation.

[17] We use ⧫ to mark findings we explicitly refer back to later.

The Interrogation. Following their capture, abductees were "handed over" to the "interrogators" who led the questioning. These interrogations included threats and physical violence (♦6). P37 relayed how they had been "tortured to say things". The use of torture to extract information was exemplified by all abductees. P61 shared how the interrogators had told them: "We are going to ask you questions and you're going to answer us; this is not a police station so they might not find you." P72 still had scars on their arms from being tortured:

> "[I]t's psychological torture, because they [interrogators] tell you: 'Just remember, you are not going back [...] so it depends on how you answer us, how we will handle you depends on how you answer our questions. Who is financing you? What are you planning?' [...] They were using a tool. I don't know whether it was an iron or wood, hitting me on the arm. So, I still have injuries [rolling up their sleeve to show the scars]."

All abductees talked about their fear of answering questions in a way that would lead to further torture (♦7). Many spoke of their confusion over having been taken, with P61 sharing how they "spent the whole night trying to think why I might be here". When one of their friends was taken, P37 was convinced that "they would come for me next". The escalating abductions – and their coordination – and the continued crackdowns on activist networks, left many fearing that the security forces were in possession of extensive information about them. The interrogations further underscored this. P61 explained how the interrogators asked questions about their protest activities that P61, in that moment, assumed they had learned from other abductees. P49 referred to "psychological mind games" where their interrogators knew "everything" about them (♦8).

Although P22 had refused to provide the PIN for their phone in the moment of abduction, the interrogations "broke" them (♦9). When they, during their first interrogation, were told to unlock their phone again, P22 gave them their PIN, because "I was so scared what would happen if I didn't". P49 shared how they had been forced to open their phone when "they [the interrogators] laughed and said they'd just cut off my thumb if I didn't do it".[18] Abductees talked about how the interrogators (in the cases of P22, P31, P37, P49 and P61) and the abductors (in the cases of P51 and P60), after having gained access to their phones, searched them for protest-related content, including financial information (scrolling through WhatsApp messages and M-PESA statements[19] and information about their protest activities. When P60 initially refused to share their phone PIN, they were beaten by their abductors:

> "I never knew a human can be that strong, that a jab can be that heavy, you know. I don't even know how I found myself on the ground because I was seated [...] at first I wondered what had brought the ground to my hand, only to realise that it is me who had fallen down. So I just gave them my password now [...] They told me 'if it doesn't work, now we are finishing you'. But, you know, I had given them the right password."

While P60 gave their abductors the "right password", P31 "ended up forgetting all of them in that situation", i.e. in the moment of capture (♦10). They also forgot phone numbers they had agreed in their group they should contact in the case of arrest. The interrogators eventually got access to P31's phone. P51 was beaten by their abductors until they provided the login details. While P51 was concerned that unlocking their phone would reveal information about the other members in their "inner circle of organisers" and their plans, they believed that their life depended on them giving up their phone PIN (♦11):

> "[The abductors] stopped the car, and they removed me out of the car. And now they started beating me and telling me to open my phone which they had. There were also questions like 'who is funding you?' [...] I told them: 'Nobody is funding me'. So they beat me up. They said: 'You are being funded'. That was their main line of questioning [...] [T]hey sent me to the boot. They put me there and then they closed and drove."

[18] Most protesters engaged through the fieldwork used PINs but some used biometrics.

[19] M-PESA is a mobile money transfer, payments and micro-financing service in Kenya (and in East Africa more broadly), operated by Safaricom which is the largest mobile network operator in Kenya.

All abductees highlighted how the interrogations focused on funding (♦12): "They wanted to know who was funding these protests, I told them that no-one was but that people were doing this themselves" (P49). The question of protest funding is not arbitrary in the context of Kenya. The fieldwork uncovered how protests have historically been funded in some form, often through external funding, where some protesters have been incentivised with the promise of money – nominally to cover travel costs. P28, a young protester from a Nairobi slum, exemplified how they had previously been given money to attend protests: "We used to go [to protests] because you only need the money, there's no motivation, no personal motivation." P34 also shared how their movement received external funding for different campaigns: "It comes in millions, you know". In contrast, Gen-Z protesters went to the street without such incentives. However, the fact that such financial undercurrents of previous protests were known across activist and political networks, with support coming from national and international organisations, it is plausible that this informed the questions during the interrogations that the abductees endured.

4.3 Protection during Abductions

Our ethnographic data shows how the fear of abduction emerged as a specific security concern in *these* protests, directly shaping protesters' security goals and protective practices. In Section 4.1 we showed how frontliners developed extensive practices to try to protect from abductions. Here, we discuss how abductees sought to protect themselves *in the moment of abduction.*

Raising the Alarm (♦13). The overshadowing security goal for abductees at the time of abduction was to make their disappearance publicly known. The fieldwork identified examples of how abductees were released by their abductors after they learned that news of the abduction had spread across social media. Our data identifies distinct practices adopted by abductees to achieve this goal.

'Raising the alarm' was an established practice among activist groups in Kenya before the Gen-Z protests, developed as a protection strategy during arrests. This was underpinned by the mantra *don't get arrested silently* which was repeated during all protest meetings. P36, who had been arrested six times in previous protests, explained: "The first thing we have been doing when arrested is to raise the alarm. Get to the group and text: 'Comrades, I'm arrested, they're taking me to Central Police Station.' That's enough. Comrades will be there." This practice relied on the security provided through close-knit groups of activists, exemplified by P41 who was arrested during the 18 June protest; before their phone was taken they managed to send out an "I've been arrested" message to their group. P41, along with around 400 other protesters on this day, were released with the support of lawyers from the Law Society of Kenya. Thus, before the June 2024 protests the practice of alerting others in the event of compromise was devised to protect during arrests; the emergence of abductions as an immediate threat to protesters led to its adaptation.

Our data contains distinct examples of how those who considered themselves a likely target of abductions prepared for this potentiality (see Section 4.1) and how abductees tried to protect themselves during their captivity. This took different forms but was grounded in communicating a distress signal that would 'let the world know' that they had been taken. While some composed a message to go out on X in the event of their abduction, some shared their login details with a trusted contact to enable them to "shout should they be abducted" (FN3). During the abductions in the early hours of 25 June, one of the abductees tweeted "Guys are outside where I am", after which their X account went silent while the post went viral (FN4) (♦14). P37 had prepared a message that they planned to send in the moment of abduction: "Before that time, before I was taken, I had planned tweets on my laptop that I would send out in the event that I would be taken, but I didn't even have time to send that out when they came for me" (♦15). P72 had established a practice with their group members that if they did not pick up or their phone was off and they could not be located, then "an alarm is raised on Twitter" (♦16). In the moment of abduction, P72 took a picture of their abductors' number plate and shared it with their partner who posted "the details online and people could start shouting [...] So, I think it became too much." P72 concluded: "This is why I'm still here."

Our data comprises several examples of how the ability to *shout* directly led to the release of the abductee. P51, while in the boot of their abductors' car, felt their second phone, undiscovered by their abductors, vibrate in their pocket:

> "I declined the call and I wrote to him a message. Very brief message: 'Hey, I'm being abducted. I don't know where I am' [...] After I had texted him, he took the screenshot [of the message] and

> posted it on X. So that is how it spread [...] funnily enough, after like 30 minutes or so, as they [the abductors] were driving me around, the car stopped again. And then they came out. They opened the trunk, and then they started beating me up. And then they asked me: 'Where is that phone?' So they knew that I had another phone because my friend shared my screenshot [...] I think that is the only thing that saved me. Because if it wasn't for that message, these guys had another motive. Because nobody was aware of my abduction, apart from that one message. So they could have killed me and taken me somewhere, nobody could have known. So the fact that I was now out saved me" (⧫ 17).

The abductors tried to 'offload' P51 at different police stations. However, with P51's abduction being widely known, support organisations such as the Law Society of Kenya sent representatives to most police stations. P51 was not prepared for potentially being abducted and therefore the *one message* they managed to send, and which they considered to be what saved them, was largely coincidental. P51 was eventually dumped in an isolated location. In the case of P22's abduction, one of their friends who had observed their abduction sounded the alarm on X which led to extensive calls for their release. The interrogators had told P22 that their friends "are making noise all over", which led P22 to surmise: "I think that was what helped me to remain alive right now." P22 was released after two days in captivity.

Our data contains several examples of how abductors either released or tried to disburden themselves of the abductee at times when the knowledge of the abduction had spread publicly. This was also the case for P60, whose abduction was observed by a fellow protester, which led to "a lot of online pressure for my release and almost all police stations in the country were visited by comrades who were looking for me". The "online pressure" for the release of abductees continued until visual proof of their release was public, e.g. on X (⧫ 18).

Security Infrastructures (⧫ 19). The practice of raising the alarm at the time of abduction relied on a network of well-connected activists, who, by posting about a given abduction, would ensure that the abduction was shared widely – and quickly. This was exemplified by P18, whose friend had been abducted: "[T]he day I was with [name redacted] when [they were] abducted [...] I posted about [them] being taken. I DM'ed those people with influence and told them what had happened so they could shout from their platform." X, which was considered to be outside the control of the security forces, served as the main platform for these objectives; as an alert function and a network of protesters at scale. For example, more than 52,000 people joined an X Space organised to call for the release of the first abductee Billy Simani (a.k.a. Crazy Nairobian) (FN3), while posters of missing protesters were circulated across X networks.

The fieldwork revealed how the coordination between independent support organisations was instrumental to the security of protesters – and to abductees once the alarm of their abduction had been raised. This included the Kenya National Commission on Human Rights, the Law Society of Kenya, Defenders Coalition and Medics for Kenya who set up a 'situation room' during the protests, from where they coordinated on-the-ground support. Fieldnotes taken during the 18 June protest exemplify this coordination:

> "Outside the police station representative from support organisations such as LSK [Law Society of Kenya], Defenders Coalition and other organisations that offer legal and medical assistance are present; negotiating with the OCS [Officer Commanding Station], documenting those who have been injured or have had things confiscated. We have examples of both this evening." (FN3)

Thus, although the Gen-Z protests appeared to be spontaneous with masses of protesters flooding streets across Kenya, they relied on highly organised and networked activists and established practices. Dedicated hotlines were set up for legal and medical assistance, while a protest-specific USSD code was obtained to enable access to protest-related information without the internet (FN4), as data was unaffordable for many[20] (⧫ 20). Limited data meant that many protesters used regular phone calls and SMS, also during protests; yet, many organisers experienced how their calls and messages would be blocked during protest days:

> "[...] We are going for the demonstrations and my phone would stop working the whole day until the following day at noon. I don't know what they do with my phone [...] Literally, you can't call me. You can't text me. I can't receive any calls or texts. And it's on [...] And it's on full charge. You can see it. And it's not the network thing. The network bars are full." (P77)

[20] USSD codes are widely used in Kenya to access e.g. payments and checking balances.

P14 shared similar experiences: "My phone was on but when they [group members] call, I'm not available; when we met up they told me 'we called you, like, 10 times', but I had not received any calls." Both P14's and P77's experiences were specific to regular phone calls and texts and not e.g. WhatsApp (♦21).

Protesters also relied on protocols developed over years of street protests in Kenya, and were agreed upon during group meetings in response to the evolving adversarial context. This, for example, included check-in/check-out protocols to ensure all group members were accounted for (♦22). It also included the provision of 'safe houses' by support organisations for those considered particularly at risk, while large water containers were placed in shoe shine stands across central protest areas. The initial stages of the protests were marked by widespread public support, with coffee shops being used as gathering spaces for protesters, with private security guards protecting protesters and with boda boda (motorcycle taxis) riders carrying water and face masks to the frontline of the protests.

5 At-Compromise Alert Blindness

A key finding in Section 4 is that targets of abductions aimed to send an alert as they were taken without their abductors noticing. In this section, we thus formalise *alert blindness* as a *compromise* occurs: AtC-AB. For this, we model a person separately from their device, jointly referred to as the 'client',[21] which enables us to rely on personal secrets or PINs to trigger an alert. Like forward secrecy and post-compromise security, this goal can be formalised in different ways depending on the protocol it applies to. Thus, to formally define alert blindness, in Section 5.1 we first introduce the *alert blindness with one message* (ABOM) protocol relying on strong cryptographic secrets stored on a device and (presumably low-entropy) personal secrets. In Section 5.3 we then define KIND-AtC-AB which models a general confidentiality goal of ABOM: the adversary cannot learn information about the client's key. We also define IND-CPA-AtC-AB which composes ABOM with an IND-CPA-secure encryption scheme to illustrate a concrete composition of ABOM with another protocol. That is, based on the findings in Section 4, the notion explicitly gives up confidentiality (formalised via KIND) in order to achieve AtC-AB. The core idea is that running the ABOM protocol is required to learn information about the client's key. In Section 5.4 we reflect these notions back to the social foundations established earlier. We also define correctness (Section 5.2) and obliviousness (Section 5.5), the latter capturing privacy against a compromised server.

5.1 Protocol Syntax and Correctness

We introduce the ABOM (sub-)protocol (Definition 14), which enables an *at-compromise* client to alert the server without detection. ABOM has a *setup phase* – client and server establish a secure channel and share the client's key – and an *alert phase* – the client recovers the key via communication with the server and may alert it.

KeyGen generates a shared secret, abstracting authenticated key exchange. The client runs ClientSetup with a protocol key κ, an application key k (for another protocol or application) and a personal secret ps (the 'real' secret) to set the device state ds, and produce a client message cm. On receiving cm, the server runs ServerSetup to set its state ss and generate the server message sm. The client then runs ClientDone, using sm and ds, to confirm setup with an acceptance bit acc and to update ds, ending the setup phase.

In the alert phase, the client recovers the key k and may send an alert by running Request, which updates ds and generates cm. The server processes cm via Response, deciding acceptance, updating its state, and setting dtct as needed. If verification fails, the server neither updates its state nor sends a response. The flag dtct is set only if the client runs Request using a secret in $\mathcal{S}$ other than the real secret ps from the setup phase. The client then runs GetKey to extract k and updates ds. Finally, Clear prunes ds to prevent key reconstruction before the client runs Request again. We formally define the ABOM protocol in Definition 14, while Fig. 6 depicts the protocol's message flow.

Definition 14. *Given a personal secret distribution $\mathcal{S}$ and a key space $\mathcal{K}$, an* alert blindness with one message (ABOM) *protocol comprises the following PPT algorithms:*

- $\mathsf{KeyGen}(1^\lambda) \to \kappa$ *takes a security parameter λ expressed in unary and outputs a key κ.*

[21] In contrast, 'server' refers to a literal server.

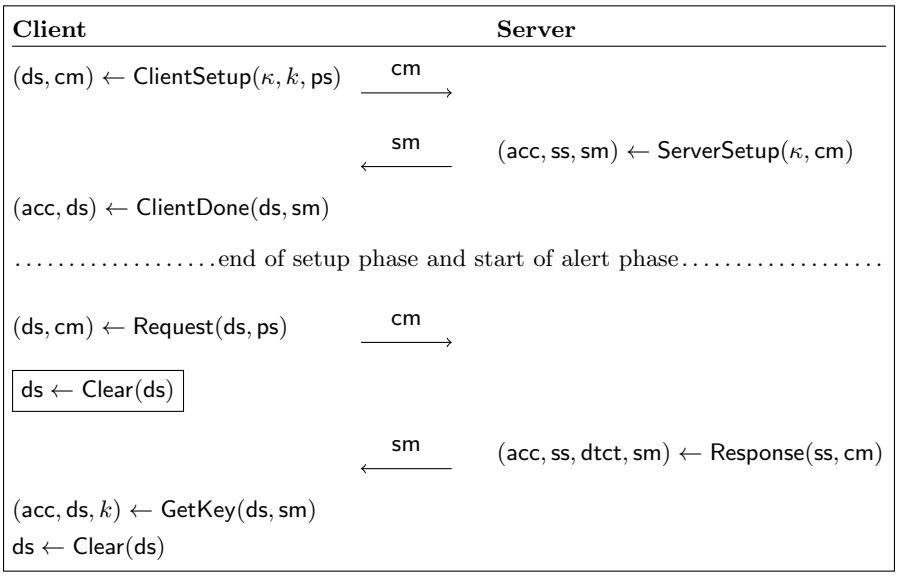


Figure 6: Flow of ABOM during the setup phase (above the dotted line) and the alert phase (below). The client executes the code within the box only if it does not receive a message from the server within a specified timeout period.

- $\mathsf{ClientSetup}(\kappa, k, \mathsf{ps}) \to (\mathsf{ds}, \mathsf{cm})$ *takes a protocol key* κ*, an application key* k *(drawn from* $\mathcal{K}$*) and a personal secret* ps *(drawn from the distribution* $\mathcal{S}$*) and outputs a device state* ds *and a client message* cm*.*
- $\mathsf{ServerSetup}(\kappa, \mathsf{cm}) \to (\mathsf{acc}, \mathsf{ss}, \mathsf{sm})$ *takes a protocol key* κ *and a client message* cm*, and outputs an acceptance bit* $\mathsf{acc} \in \{\mathsf{true}, \mathsf{false}\}$*, a server state* ss *and a server message* sm*. We assume that* $\mathsf{sm} \leftarrow \perp$ *and* $\mathsf{ss} \leftarrow \perp$ *when* $\mathsf{acc} = \mathsf{false}$*.*
- $\mathsf{ClientDone}(\mathsf{ds}, \mathsf{sm}) \to (\mathsf{acc}, \mathsf{ds}')$ *takes a device state* ds *and a server message, and returns an acceptance bit* $\mathsf{acc} \in \{\mathsf{true}, \mathsf{false}\}$ *and outputs an updated device state* ds'*. We assume that* $\mathsf{ds}' \leftarrow \mathsf{ds}$ *when* $\mathsf{acc} = \mathsf{false}$*.*
- $\mathsf{Request}(\mathsf{ds}, \mathsf{ps}) \to (\mathsf{ds}', \mathsf{cm})$ *takes a device state* ds *and a personal secret* $\mathsf{ps} \in \mathcal{S}$*, and outputs an updated device state* ds' *and a client message* cm*.*
- $\mathsf{Response}(\mathsf{ss}, \mathsf{cm}) \to (\mathsf{acc}, \mathsf{ss}', \mathsf{dtct}, \mathsf{sm})$ *takes a server state* ss *and a client message* cm*, and outputs an acceptance bit* $\mathsf{acc} \in \{\mathsf{true}, \mathsf{false}\}$*, an updated server state* ss'*, a detection bit* $\mathsf{dtct} \in \{\mathsf{true}, \mathsf{false}\}$ *and a server message* sm*. We assume that* $\mathsf{sm} \leftarrow \perp$ *and* $\mathsf{ss}' \leftarrow \mathsf{ss}$ *when* $\mathsf{acc} = \mathsf{false}$*.*
- $\mathsf{GetKey}(\mathsf{ds}, \mathsf{sm}) \to (\mathsf{acc}, \mathsf{ds}', k)$ *takes a device state* ds *and a server message* sm*, and returns an acceptance bit* $\mathsf{acc} \in \{\mathsf{true}, \mathsf{false}\}$ *an updated device state* ds *and a key* k*. We assume that* $k \leftarrow \perp$ *and* $\mathsf{ds}' \leftarrow \mathsf{ds}$ *when* $\mathsf{acc} = \mathsf{false}$*.*
- $\mathsf{Clear}(\mathsf{ds}) \to \mathsf{ds}'$ *takes a device state* ds *and outputs a new device state* ds'*.*

For notational convenience we also define $\mathsf{GeneralSetup}(1^\lambda, k, \mathsf{ps}) \to (\mathsf{ds}, \mathsf{ss})$*:*

Algorithm $\mathsf{GeneralSetup}(1^\lambda, k, \mathsf{ps})$

1 : $\kappa \leftarrow \mathsf{KeyGen}(1^\lambda)$
2 : $(\mathsf{ds}, \mathsf{cm}) \leftarrow \mathsf{ClientSetup}(\kappa, k, \mathsf{ps})$
3 : $(\cdot, \mathsf{ss}, \mathsf{sm}) \leftarrow \mathsf{ServerSetup}(\kappa, \mathsf{cm})$
4 : $(\cdot, \mathsf{ds}) \leftarrow \mathsf{ClientDone}(\mathsf{ds}, \mathsf{sm})$
5 : **return** $(\mathsf{ds}, \mathsf{ss})$

Game CORRECT$(1^\lambda, k, \mathsf{ps}_1, \mathsf{sched})$

$(\mathsf{ds}, \mathsf{ss}) \leftarrow \mathsf{GeneralSetup}(1^\lambda, k, \mathsf{ps}_1)$
$\mathsf{cleared} \leftarrow \mathsf{false};\ \mathsf{cm}[\cdot], \mathsf{sm}[\cdot] \leftarrow \bot;\ \mathsf{sent}_*[\cdot], \mathsf{recv}_*[\cdot], \mathsf{real}[\cdot] \leftarrow \mathsf{false};\ \mathsf{last}_* \leftarrow 0$
for $i = 1$ **to** $\mathsf{length}(\mathsf{sched})$:
  if $\mathsf{sched}[i]$ parses as $(\text{“request”}, \mathsf{ps})$ for $\mathsf{ps} \in \mathcal{S}$:
    $(\mathsf{ds}, \mathsf{cm}) \leftarrow \mathsf{Request}(\mathsf{ds}, \mathsf{ps})$
    if $\mathsf{ps} = \mathsf{ps}_1$: $\mathsf{real}[i] \leftarrow \mathsf{true}$
    $\mathsf{cleared} \leftarrow \mathsf{false};\ \mathsf{sent}_{\mathsf{cm}}[i] \leftarrow \mathsf{true};\ \mathsf{cm}[i] \leftarrow \mathsf{cm}$
  elseif $\mathsf{sched}[i]$ parses as $(\text{“response”}, j)$ for $j \in \mathbb{N}$:
    if $\neg\mathsf{sent}_{\mathsf{cm}}[j] \vee \mathsf{recv}_{\mathsf{cm}}[j] \vee \mathsf{last}_{\mathsf{cm}} > j$: **continue**
    $(\mathsf{acc}, \mathsf{ss}, \mathsf{dtct}, \mathsf{sm}) \leftarrow \mathsf{Response}(\mathsf{ss}, \mathsf{cm}[j])$
    if $\neg\mathsf{acc} \vee (\mathsf{real}[j] = \mathsf{dtct})$: **return** 1
    $\mathsf{recv}_{\mathsf{cm}}[j] \leftarrow \mathsf{true};\ \mathsf{sent}_{\mathsf{sm}}[i] \leftarrow \mathsf{true};\ \mathsf{sm}[i] \leftarrow \mathsf{sm};\ \mathsf{last}_{\mathsf{cm}} \leftarrow j$
  elseif $\mathsf{sched}[i]$ parses as $(\text{“getkey”}, j)$ for $j \in \mathbb{N}$:
    if $\neg\mathsf{sent}_{\mathsf{sm}}[j] \vee \mathsf{recv}_{\mathsf{sm}}[j] \vee \mathsf{last}_{\mathsf{sm}} > j$: **continue**
    $(\mathsf{acc}, \mathsf{ds}, k') \leftarrow \mathsf{GetKey}(\mathsf{ds}, \mathsf{sm}[j])$
    if $(\neg\mathsf{cleared} \wedge (\neg\mathsf{acc} \vee k' \neq k)) \vee (\mathsf{cleared} \wedge \mathsf{acc})$: **return** 1
    $\mathsf{recv}_{\mathsf{sm}}[j] \leftarrow \mathsf{true};\ \mathsf{last}_{\mathsf{sm}} \leftarrow j$
  else: $\mathsf{sched}[i]$ parses as (“clear”):
    $\mathsf{ds} \leftarrow \mathsf{Clear}(\mathsf{ds})$
    $\mathsf{cleared} \leftarrow \mathsf{true}$
return 0

Figure 7: Correctness game for the ABOM protocol, where we highlight the essential correctness requirements.

5.2 Correctness

Informally, an ABOM protocol is *correct* if the GetKey algorithm always returns the key k that was provided during the setup phase, the real personal secret ps_1 does not trigger an alert, all other personal secrets $\mathsf{ps} \in \mathcal{S} \setminus \{\mathsf{ps}_1\}$ do trigger an alert and a call to Clear prunes the information that the client needs to compute the key until a fresh Request call. We insist an ABOM protocol is robust against dropped and out-of-order messages, i.e. can fast forward to messages with a greater index than the one expected and silently discards stale or late messages without causing desynchronisation. We formally define correctness in Definition 15, based on the CORRECT game of Fig. 7. This game accepts a security parameter, a key k, a personal secret $\mathsf{ps} \in \mathcal{S}$ and a schedule sched, which models the sequence of calls to the protocol's algorithm. Specifically, sched is an ordered list of instruction of the form (1) $(\text{“request”}, \mathsf{ps})$, (2) $(\text{“response”}, j)$, (3) $(\text{“getkey”}, j)$, (4) (“clear”), where $j \in \mathbb{N}$ indicates the messages that Response and GetKey must process.

Definition 15 (ε-Correctness). *Consider the game* CORRECT *in Fig. 7. An* ABOM *protocol is* ε-correct *if, for a personal secret distribution* $\mathcal{S}$*, for all* $\lambda \in \mathbb{N}$*, for all* $k \in \mathcal{K}$*, for all* $\mathsf{ps}_1 \in \mathcal{S}$ *and all sequences* sched *with elements of the form* $(\text{“request”}, \mathsf{ps})$*,* $(\text{“response”}, j)$*,* $(\text{“getkey”}, j)$*,* (“clear”)*, for* $\mathsf{ps} \in \mathcal{S}$*,* $j \in \mathbb{N}$*, we have*

$$\Pr[\text{CORRECT}(1^\lambda, \mathsf{sched}) \Rightarrow 1] \leq \varepsilon + \mathsf{negl}(\lambda) \ .$$

When $\varepsilon \in \mathsf{negl}(\lambda)$ *we say an* ABOM *protocol is* correct.

5.3 KIND and IND-CPA with At-Compromise Alert Blindness

This section introduces our main security notion: KIND with At-Compromise Alert Blindness (KIND-AtC-AB) in Definition 16. This is meant to abstractly capture confidentiality. We also introduce IND-CPA

with At-Compromise Alert Blindness (IND-CPA-AtC-AB) in Definition 17 to illustrate a more concrete composition with another protocol, here an IND-CPA-secure protocol Π. To illustrate the flexibility of AtC-AB, in Appendix B we also define an unforgeability-style game where the adversary wins if they forge an 'all-clear' message.

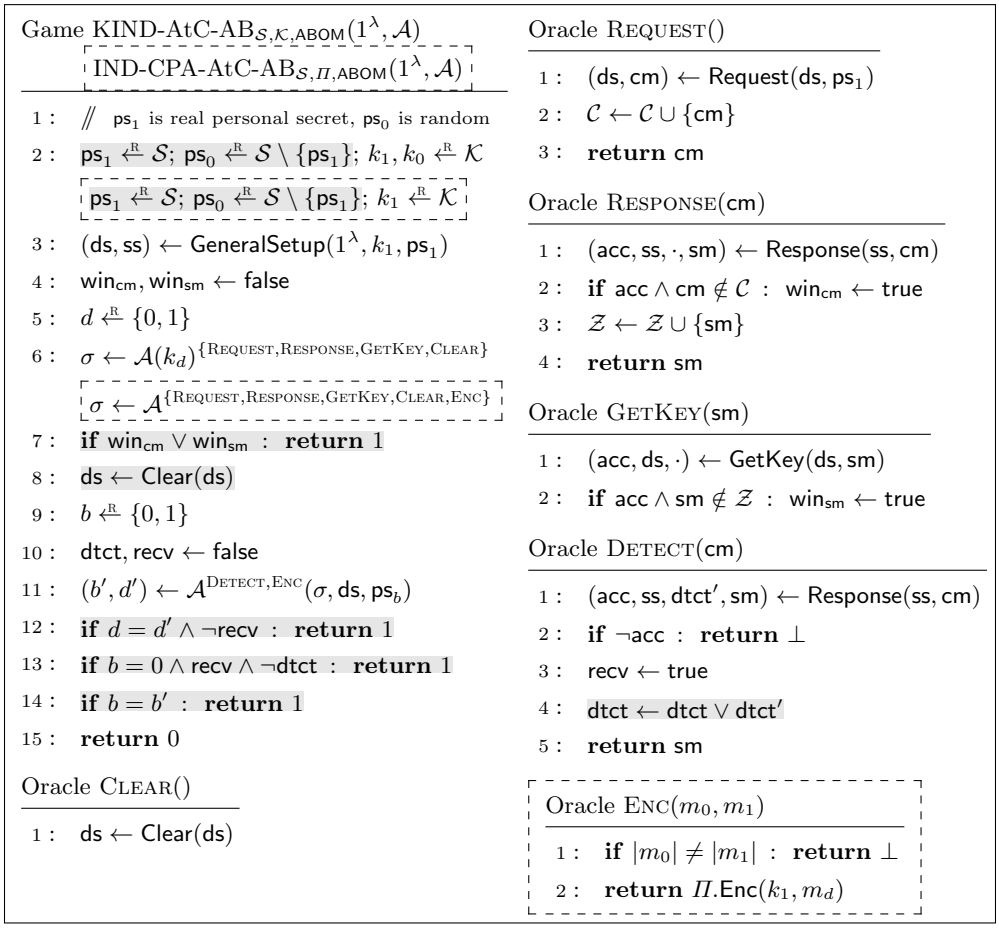


Figure 8: KIND and IND-CPA with At-Compromise Alert Blindness. We highlight code that is grounded in Section 4.

We discuss the KIND-AtC-AB game. The challenger samples a real secret ps_1 and a random secret $\mathsf{ps}_0 \neq \mathsf{ps}_1$ from $\mathcal{S}$ and a key k from $\mathcal{K}$. It runs GeneralSetup and initialises the game's variables. In the first phase, $\mathcal{A}$ interacts with the oracles $\{\text{Request}, \text{Response}, \text{GetKey}, \text{Clear}\}$.[22] This phase models the pre-compromise state: $\mathcal{A}$ observes the network and exchanges messages, aiming to forge cm or sm without access to ds or ss. The challenger checks $\mathsf{win}_{\mathsf{cm}}$ and $\mathsf{win}_{\mathsf{sm}}$ which model forgeries. Then Clear is run, pruning ds of any key-reconstructing information without a fresh cm. Without Clear, $\mathcal{A}$ could replay ds and recover k via GetKey. Our protocol ensures second-phase security only if Clear is called, explicitly or via a timeout.

In the second phase, $\mathcal{A}$ has access to Detect,[23] its state σ, the device state ds and ps_b for $b \xleftarrow{R} \{0,1\}$. This phase models compromise. $\mathcal{A}$ wins in three ways (lines 12 to 14). First, $d = d' \wedge \neg\mathsf{recv}$ enforces that the server received a valid message (clause $\neg\mathsf{recv}$) to distinguish the KIND bit (clause $d = d'$). Second, $b = 0 \wedge \mathsf{recv} \wedge \neg\mathsf{dtct}$ ensures the *alert* succeeds. $\mathcal{A}$ can get k: they run Request to get cm, query Detect for sm and use GetKey with ds and sm. But detection must trigger (clauses $b = 0$ and $\neg\mathsf{dtct}$). Thus, for any

[22] This is not a restriction since $\mathcal{A}$ wins at this stage by forging *any* accepted message.
[23] At this stage $\mathcal{A}$ has ds to emulate Request, Clear and GetKey.

ps_0, $\mathcal{A}$ can learn the KIND bit only by triggering detection. Third, $b = b'$ enforces *blindness*: even with ds, $\mathcal{A}$ cannot distinguish the two secrets. In particular, that a message has to go out to reconstruct the key is known, whether this message carries an alarm is not.

The IND-CPA-AtC-AB game proceeds similarly, but the adversary does not get the challenge key k_d and instead has access to the IND-CPA oracle Enc.

Definition 16 (KIND-AtC-AB**).** *Let $\lambda \in \mathbb{N}$ be a security parameter. Let* KIND-AtC-AB$_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}$ *be the* KIND with At-Compromise Alert Blindness *game defined in Fig. 8, with respect to a personal secret distribution $\mathcal{S}$, a key distribution $\mathcal{K}$ and an* ABOM *protocol. Let $\mathcal{T}$ be the distribution obtained by sampling $x' \xleftarrow{R} \mathcal{S}$ and then sampling and outputting $x \xleftarrow{R} \mathcal{S} \setminus \{x'\}$. Let $\mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}$ as per Proposition 2:*

$$\mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}} = \frac{1 + \mathsf{SD}(\mathcal{S},\mathcal{T}) + 2^{-H_\infty(\mathcal{S}\setminus\{x_b\})}}{2} \ .$$

We define the advantage of an adversary $\mathcal{A}$ in winning the KIND-AtC-AB$_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}$ *game as*

$$\mathsf{Adv}^{\text{KIND-AtC-AB}}_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) = \Pr[\text{KIND-AtC-AB}_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}(1^\lambda,\mathcal{A}) \Rightarrow 1] - \frac{1 + \mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}}{2} \ .$$

We say that ABOM *is* KIND-AtC-AB*-secure when the advantage is negligible in the security parameter λ.*

Definition 17 (IND-CPA-AtC-AB**).** *Let $\lambda \in \mathbb{N}$ be a security parameter. Let* IND-CPA-AtC-AB$_{\mathcal{S},\Pi,\mathsf{ABOM}}$ *be the* IND-CPA-AtC-AB with At-Compromise Alert Blindness *game defined in Fig. 8, with respect to a personal secret distribution $\mathcal{S}$, an* ABOM *protocol and a symmetric encryption scheme Π (with key distribution $\mathcal{K}$). Let $\mathcal{T}$ be the distribution obtained by sampling $x' \xleftarrow{R} \mathcal{S}$ and then sampling and outputting $x \xleftarrow{R} \mathcal{S} \setminus \{x'\}$. Let $\mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}$ as per Proposition 2:*

$$\mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}} = \frac{1 + \mathsf{SD}(\mathcal{S},\mathcal{T}) + 2^{-H_\infty(\mathcal{S}\setminus\{x_b\})}}{2} \ .$$

We define the advantage of an adversary $\mathcal{A}$ in winning the IND-CPA-AtC-AB$_{\mathcal{S},\Pi,\mathsf{ABOM}}$ *game as*

$$\mathsf{Adv}^{\text{IND-CPA-AtC-AB}}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) = \Pr[\text{IND-CPA-AtC-AB}_{\mathcal{S},\Pi,\mathsf{ABOM}}(1^\lambda,\mathcal{A}) \Rightarrow 1] - \frac{1 + \mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}}{2} \ .$$

We say that the composition of ABOM *and Π is* IND-CPA-AtC-AB*-secure when if the advantage is negligible in the security parameter λ.*

Remark 2. We define the advantage in the two definitions above as doing better than guessing d, b or ps_1. In particular, let B be the event of $\mathcal{A}$ distinguishing b (line 14) by distinguishing $\mathcal{S}$ and $\mathcal{T}$; let G be the event of $\mathcal{A}$ forging a message not triggering dtct (line 13) by guessing the personal secret; and let D be the event of $\mathcal{A}$ guessing the bit d (line 12), then

$$\Pr[D \cup B \cup G] = \Pr[D] + \Pr[B \cup G] - \Pr[D \cap (B \cup G)] = \frac{1}{2} + \frac{\Pr[B \cup G]}{2} \ .$$

Moreover, we have that

$$\mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}(\mathcal{A}) = \Pr[B \cup G] \leq \frac{1 + \mathsf{SD}(\mathcal{S},\mathcal{T}) + 2^{-H_\infty(\mathcal{S}\setminus\{x_b\})}}{2} = \mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}$$

for $\mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}(\mathcal{A})$ defined in Definition 1 and the bound established in Proposition 2. We do not take the absolute value because it is easy to construct an adversary that performs much worse than $\mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}$.

5.4 Realism of KIND-AtC-AB

A key contribution of this work is that it establishes the realism of the newly introduced notion. In Section 4 we established *what* security at compromise meant within the context we consider. Here, we reflect back how our *constructed* notion relates to these findings. We stress that our model only relies on empirical data to establish the security goals of honest parties but not to establish adversarial strategies. We assume worst-case PPT adversaries but also recognise these adversaries as violent. Ethnography establishes $\exists$ claims and not $\forall$ claims (Section 3.1).

Table 1: Outcomes of the KIND-AtC-AB game and their justifications.

b	KIND	dtct	recv	Win?	Justification
0	$d = d'$	dtct	recv	✗	The adversary distinguishes the KIND challenge bit, but in doing so dtct is set: the abducted person surrenders the key (4), but raised the alarm (13).
0	$d = d'$	dtct	¬recv		This case is impossible.
0	$d = d'$	¬dtct	recv	✓	The adversary distinguishes the KIND challenge bit, but dtct is not set: the abducted person surrenders the key, but does not raise an alarm, thereby achieving $\mathcal{A}$'s goal.
0	$d = d'$	¬dtct	¬recv	✓	The adversary distinguishes the KIND challenge bit, but dtct is not set because the adversary did not use the DETECT oracle (since recv = false).
0	$d \neq d'$	dtct	recv	✗	The adversary is incompetent: by construction it can win the KIND game, but $\mathcal{A}$ outputs a wrong decision d'.
0	$d \neq d'$	dtct	¬recv		This case is impossible.
0	$d \neq d'$	¬dtct	recv	✓	The adversary, while incompetent, manages a forgery and we reward it.
0	$d \neq d'$	¬dtct	¬recv	✗	The adversary does not call DETECT and does not win the KIND game.
1	$d = d'$	¬dtct	recv	✗	The abducted person surrenders their real personal secret ps_1: cryptography cannot help. We exclude it as a trivial win.
1	$d = d'$	¬dtct	¬recv	✓	The adversary wins the KIND game without using the DETECT oracle.
1	$d \neq d'$	¬dtct	recv	✗	An incompetent adversary called DETECT yet failed to win the KIND game
1	$d \neq d'$	¬dtct	¬recv	✗	The adversary does not call DETECT and does not win the KIND game.
1	...	dtct	...		We only check dtct when $b = 0$.

Claim 1 *Consider* KIND-AtC-AB *as defined in Definition 17. Model "confidentiality" as* KIND *security. Then* KIND-AtC-AB *correctly responds to security needs of some organisers and frontliners in Kenya in 2024.*

Justification. Most code in Fig. 8 performs bookkeeping, maintains consistency or exists for complexity-theoretic reasons. We do not discuss this here further. We only consider highlighted code.

Our game models the interaction of a client – a person and their device – and a server. The latter reflects collective approaches to security – "Comrades will be there" (P36) – and observed security infrastructures (19). Thus, organisations/networks exist that can run such a server. The use of personal secrets (line 2) captures the distinction of compromising people or their devices. The use of more than one personal secret models that the compromise considered here is detectable by at least one honest party, the abducted person: all abductees who still had their device were forced to unlock it (1).[24] We parameterise the game and advantage statement by the personal secret distribution $\mathcal{S}$ (and $\mathcal{T}$ which is a function of $\mathcal{S}$) since we consider $\mathcal{S}$ to be given by the context.

The winning condition $\mathsf{win}_{\mathsf{cm}} \vee \mathsf{win}_{\mathsf{sm}}$ in line 7 models the adversary's control over the network, i.e. we want to rule out forgeries against such an adversary. On the one hand, this is a standard cryptographic assumption. On the other hand, we observed that the adversary in Kenya indeed exploited its ability to control the network, at least for denial of service: "they told me 'we called you, like, 10 times', but I had not received any calls" (P14) (21).[25]

Clear can be realised with a timeout (15) or manually (14),(2).[26] Line 4 of DETECT models that an alert is maintained until a sign of life (18).

The main task is to justify the three winning conditions in lines 12 to 14. These model the adversarial goal of intelligence gathering (12) and the security goal of confidentiality by activists (11), which we capture by the KIND bit $d = d'$. Line 12 only considers the adversary successful when ¬recv. This captures that a violent adversary can win the KIND game: all abductees feared for their life if they did not cooperate with the adversary (4),(7) and were subjected to threats and violence (6) (P22 initially refused (3) but cooperated

[24] The decision to trigger an alarm for any $\mathsf{ps}_0 \neq \mathsf{ps}_1$ rules out a 'trivial win' where an adversary extracts two PINs from an abducted person to boost their chances of success. It also alleviates the need to remember a specific "alert" personal secret (10).

[25] However, this control was incomplete, cf. Appendix B.

[26] P72 smashed their phone *before* the abduction.

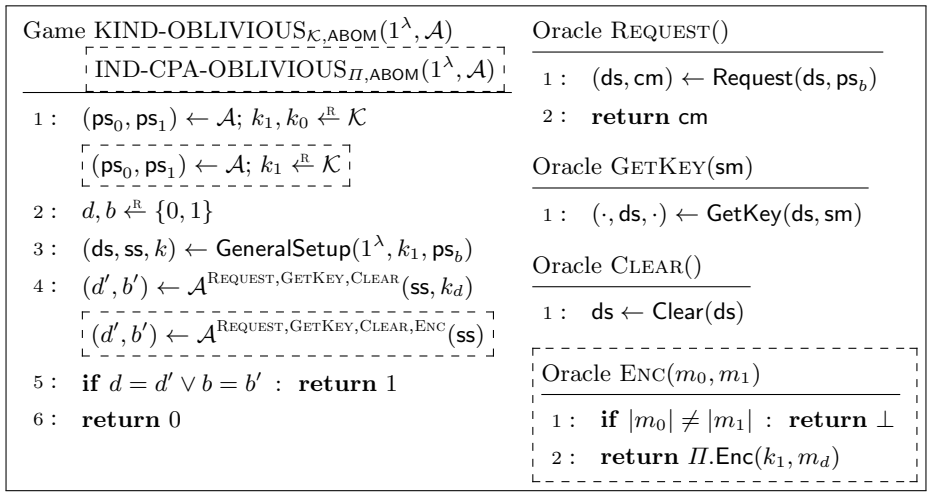


Figure 9: KIND and IND-CPA obliviousness for ABOM.

later (9)). The game enforces that breaking KIND this way requires at least one message to be distributed to the server.

This message then realises the security goal to raise the alarm (13). If a wrong personal secret was provided ($b=0$) and a message was allowed through (recv) by $\mathcal{A}$, it only wins if no alert is raised ($\neg$dtct).

The blindness bit ($b=b'$) in line 14 captures that non-cooperation remains hidden from the adversary until protective mechanisms can be deployed based on $\mathsf{dtct}=\mathsf{true}$ being set on the server (4). (The adversary detected the alert (17) for P51 but this happened *after* the protective mechanism – broad public awareness of the abduction – triggered.)

The indistinguishability of b necessitates returning the correct key k_1 to rule out trivial wins. Returning $k'\neq k_1$ on $\mathsf{dtct}=\mathsf{true}$ to preserve KIND security at compromise, would not be correct here: "They told me 'if it doesn't work, now we are finishing you'. [...] I had given them the right password" (P60).
We summarise these three winning conditions in Table 1.

Remark 3. Blindness would also be violated by returning a key that decrypts to some other plaintext, e.g. assuming non-committing encryption [CFGN96]. For example, P61 and P49 reported how their interrogators had significant knowledge of their personal and political lives (5),(8). That is, what messages to equivocate ciphertexts *to* is not known.

5.5 Obliviousness

The *obliviousness* property informally states that a malicious server learns 1) neither the client's key nor 2) the personal secret that the person selects at setup (i.e. the real personal secret). Under our assumption of an honest setup phase, this corresponds to a server that is compromised only at a later time. We formally define $\mathfrak{G}$-OBLIVIOUS in Definitions 18 and 19, for $\mathfrak{G}\in\{\text{KIND},\text{IND-CPA}\}$. This game defines a two-stage adversary, which first outputs a pair of personal secrets. We then run the setup algorithm and ask the adversary to guess either the blindness bit or the KIND (resp. IND-CPA) bit. This notion, minimising data leakage to the server, is justified by that the sever might be seized or otherwise compromised.

Definition 18 (KIND-OBLIVIOUS)**.** *Let $\lambda\in\mathbb{N}$ be a security parameter. Let* KIND-OBLIVIOUS$_{\mathcal{K},\mathsf{ABOM}}$ *be the* KIND obliviousness *game in Fig. 9, with respect to an* ABOM *protocol (Definition 14) and a key distribution $\mathcal{K}$. We define the advantage of $\mathcal{A}$ winning the* KIND-OBLIVIOUS$_{\mathcal{K},\mathsf{ABOM}}$ *game as*

$$\mathsf{Adv}^{\text{KIND-OBLIVIOUS}}_{\mathcal{K},\mathsf{ABOM}}(\mathcal{A})=\left|\Pr[\text{KIND-OBLIVIOUS}_{\mathcal{K},\mathsf{ABOM}}(1^\lambda,\mathcal{A})\Rightarrow 1]-3/4\right|\ .$$

We say that ABOM *is* KIND-OBLIVIOUS*-secure if the advantage is negligible in the security parameter λ.*

Definition 19 (IND-CPA-OBLIVIOUS**).** *Let $\lambda \in \mathbb{N}$ be a security parameter. Moreover, let* IND-CPA -OBLIVIOUS$_{\Pi,\mathsf{ABOM}}$ *be the* IND-CPA obliviousness *game in Fig. 9, with respect to an* ABOM *protocol (Definition 14) and a symmetric encryption scheme Π with key distribution $\mathcal{K}$ (Definition 7). We define the advantage of $\mathcal{A}$ winning the* IND-CPA-OBLIVIOUS$_{\Pi,\mathsf{ABOM}}$ *game as*

$$\mathsf{Adv}^{\text{IND-CPA-OBLIVIOUS}}_{\Pi,\mathsf{ABOM}}(\mathcal{A}) = \left| \Pr[\text{IND-CPA-OBLIVIOUS}_{\Pi,\mathsf{ABOM}}(1^\lambda, \mathcal{A}) \Rightarrow 1] - 3/4 \right| .$$

We say that the composition of ABOM *and Π is* IND-CPA-OBLIVIOUS*-secure if the advantage is negligible in the security parameter λ.*

6 Construction

In Fig. 10 we instantiate the ABOM protocol. We then prove that this instantiation is correct, $\mathfrak{G}$-AtC-AB-secure and achieves $\mathfrak{G}$-OBLIVIOUS security for $\mathfrak{G} \in \{\text{KIND}, \text{IND-CPA}\}$.

At a high level, the construction enables the client to send commitments to personal secrets to the server over a forward-secure channel while ensuring that the application key k can always be reconstructed. The forward-secure channel is obtained from an AEAD scheme combined with KDF-derived per-ratchet keys. The application key is secret-shared as $k \leftarrow \varkappa_0 \oplus \varkappa_1$ between the client (holding $\varkappa_0$) and the server (holding $\varkappa_1$), so that either share alone is information-theoretically useless. During setup, the client computes a commitment $\mathsf{com} \leftarrow \mathsf{F}(k_{\mathsf{com}}, \mathsf{ps})$ using a PRF F under a device-held key k_{com} and transmits com to the server. The server stores com and, in the alert phase, checks a fresh submission com' from the client against it: a mismatch raises an alert, while a match does not. Crucially, the server always returns its correct share $\varkappa_1$, enabling the client to reconstruct the application key as $k \leftarrow \varkappa_0 \oplus \varkappa_1$. We next describe the protocol details and present the security proofs in Section 6.1.

We discuss the three helper functions in Fig. 10 . The encrypt-then-MAC (EtM) and verify-then-decrypt (VtD) functions are standard. The forward-to-key (FtK) function takes as input a ratchet state rs and two counters, cnt and cnt', and advances the ratchet to level cnt', returning the corresponding key, updated state and counter.

We describe the setup phase of ABOM in Fig. 10. In this phase, the client first runs the $\mathsf{ClientSetup}$ function, which takes the shared secret κ, the application key k and the person's *real* personal secret ps. The procedure uses the shared secret to derive two ratchet states $\mathsf{rs}_{\mathsf{cm}}$ and $\mathsf{rs}_{\mathsf{sm}}$, which server as the initial states for ratchets that produce the keys to encrypt client and server messages, respectively. The function then computes a commitment over the person's *real* personal secret ps and secret-shares the application key k as $\varkappa_0 \oplus \varkappa_1$. Finally, it encrypts the server's share $\varkappa_1$ using the encrypt-then-MAC function (EtM in Fig. 10) and outputs the device state ds and the client message.

Upon receiving the client message cm, the server runs $\mathsf{ServerSetup}$. This function initialises the ratchet states and counters, parses the client messages and advances the client-message ratchet using the counter $\mathsf{cnt}_{\mathsf{cm}}$. After forwarding the ratchet and deriving the corresponding key, the function uses the verify-then-decrypt function (VtD in Fig. 10) to decrypt the client message. If decryption fails, the function returns an error. Such failures occur for old messages, since the server does not store previous keys or ratchet states, but it suffices for our purposes to decrypt only the most recent message. The server then stores the commitment to the *real* personal secret and its key share $\varkappa_1$, advances the server-message ratchet, and acknowledges reception to the client.

The function $\mathsf{ClientDone}$ concludes the setup phase by decrypting the server message and advancing the ratchets accordingly. As before, the function decrypts only the most recent message and rejects possibly delayed messages.

We discuss the alert phase in Fig. 10. The functions of this phase closely mirror those of the setup phase. The $\mathsf{Request}$ function enables the client to send a message to the server containing a commitment to the personal secret ps. Upon receiving the client message, the server executes the $\mathsf{Response}$ function, which sets $\mathsf{dtct} \leftarrow \mathsf{true}$ if the commitment *differs* from the commitment to the real personal secret that the server stored during the setup phase. Thus, *all* personal secrets from the personal secret space $\mathcal{S}$ *except* the real one yield $\mathsf{dtct} \leftarrow \mathsf{true}$. The server then sends the key share $\varkappa_1$ to the client. Importantly, the server *always* sends the key share to the client and the commitment to ps only affects the value of dtct. Upon receiving $\varkappa_1$, the client reconstructs the application key $k \leftarrow \varkappa_0 \oplus \varkappa_1$.

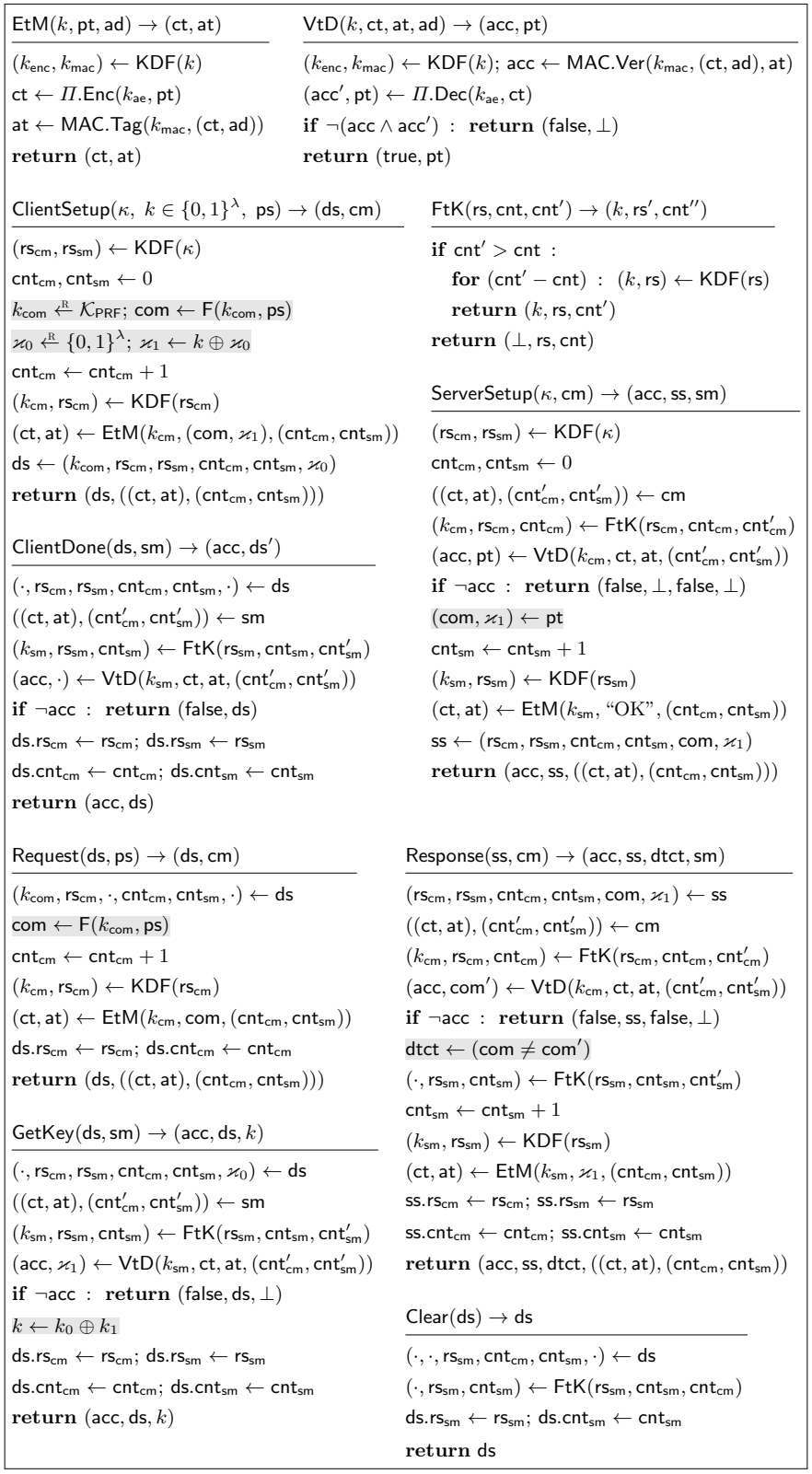


Figure 10: Proof-of-feasibility construction. Functions EtM (encrypt-then-MAC), VtD (verify-then-decrypt) and FtK (forward-to-key) serve as auxiliary helpers. We highlight the important steps.

The Clear function plays a crucial role in achieving our security notion. When the alert phase is active, the device state ds and a server message sm suffice to reconstruct the application key k. In particular, the adversary can compute the key *without* sending a message, thus preventing the client from raising an alert. The purpose of Clear is to block this scenario: it advances the *server* ratchets so that an old server message sm cannot be decrypted and is therefore useless for reconstruction. We envision Clear being called after a timeout if the client does not call GetKey, and it may also be invoked manually to clear keying material before a potentially critical event, such as a protest.

6.1 Security Proofs

We prove that the construction of the ABOM protocol of Fig. 10 is correct, KIND-AtC-AB, IND-CPA-AtC-AB, KIND-OBLIVIOUS and IND-CPA-OBLIVIOUS secure by reducing its security to that of its building blocks. Throughout, we build on prior work [BN08, BDJR97, BR00] to assume that the pair of algorithms $(\mathsf{EtM}, \mathsf{VtD})$ is a correct one-time AEAD scheme (Definition 10) that is OT-IND$-CPA-secure (Definition 12) and OT-EUF-CMA-secure (Definition 13). We rely on F being $(\mathcal{S}, \varepsilon)$-injective as per Definition 20. In our proofs below, we will consider a setting where the adversary has the key k so that we cannot solely rely on F being a PRF, motivating our definition:[27]

Definition 20 ($(\mathcal{S}, \varepsilon)$-injective Functions). F *is $(\mathcal{S}, \varepsilon)$-injective if*

$$\Pr_{k \leftarrow\$ \mathcal{K}}[\,\exists\, x \neq y \in \mathcal{S} : \mathsf{F}(k, x) = \mathsf{F}(k, y)] \leq \varepsilon \ .$$

Theorem 1 (Correctness). *Given a correct one-time AEAD scheme* $(\mathsf{EtM}, \mathsf{VtD})$ *(Definition 11) and a $(\mathcal{S}, \varepsilon)$-injective* F*, the construction in Fig. 10 is* ε-correct *as per Definition 15.*

Proof. We prove that the ABOM protocol of Fig. 10 is ε-correct by induction.

Base case ($i = 1$). We differentiate two cases:

1. If $\mathsf{sched}[1]$ parses as $(\text{“request”}, \mathsf{ps})$ for $\mathsf{ps} \in \mathcal{S}$ or as (“clear”) the game does not return 1.
2. If $\mathsf{sched}[1]$ parses as $(\text{“response”}, j)$ or as $(\text{“getkey”}, j)$ for $j \in \mathbb{N}$ the game does not return 1 because $\neg\mathsf{sent}_{\mathsf{cm}}[j] = \mathsf{true}$ and $\neg\mathsf{sent}_{\mathsf{sm}}[j] = \mathsf{true}$ since the game changes the values in the sent_* maps only if $\mathsf{sched}[j]$ parses as $(\text{“request”}, \mathsf{ps})$ for $\mathsf{ps} \in \mathcal{S}$, which is false by assumption. Therefore in this case the game does not return 1 since either the clause on line 9 is true or the clause on line 14 is true.

Therefore correctness holds at the first step of the loop in the CORRECT game.

Induction step. We assume that the protocol is correct up to $i = k - 1$ and we prove that correctness holds also for the k-th step of the loop, for $k \leq \mathsf{length}(\mathsf{sched})$. We focus on the values to which $\mathsf{sched}[k]$ parses that can break correctness by differentiating the following two cases:

1. Assume $\mathsf{sched}[k]$ parses as $(\text{“response”}, j)$ for $j \in \mathbb{N}$. We assume that the clause $\neg\mathsf{sent}_{\mathsf{cm}}[j] \vee \mathsf{recv}_{\mathsf{cm}}[j] \vee \mathsf{last}_{\mathsf{cm}} > j$ is false, otherwise correctness of this case holds by construction. Therefore we have $\neg\mathsf{sent}_{\mathsf{cm}}[j] = \mathsf{false}$, $\mathsf{recv}_{\mathsf{cm}}[j] = \mathsf{false}$ and $\mathsf{last}_{\mathsf{cm}} \leq j$, which implies that $\mathsf{sent}_{\mathsf{cm}}[j] = \mathsf{true}$ and that $\mathsf{last}_{\mathsf{cm}} < j$. The latter inequality holds because the case $\mathsf{last}_{\mathsf{cm}} = j$ is ruled out by $\mathsf{recv}_{\mathsf{cm}}[j] = \mathsf{false}$, i.e. the CORRECT game does not allow to call Response twice on the same message. By induction hypothesis we know that $\mathsf{cm}[j]$ is the correct output of a call to Request on line 5 of the game. We now show that all the conjunctions on line 11 of the CORRECT game are false, which implies that the entire disjunction is false.
   - $\underline{\neg\mathsf{acc}}$. We know that $\mathsf{last}_{\mathsf{cm}} < j$, so $\mathsf{cm}[j]$ has a $\mathsf{cnt}_{\mathsf{cm}[j]}$ greater than any cm that Response on line 10 of the game has already processed (see line 3 of the Request procedure in Fig. 10). Therefore the condition on line 1 of the FtK function in Fig. 10 holds and the key is advanced to $\mathsf{cnt}_{\mathsf{cm}[j]}$, i.e. the counter that the correct call to Request for $\mathsf{cm}[j]$ sets (in particular, this counter is denoted $\mathsf{cnt}'_{\mathsf{cm}}$ in the Response procedure and it is the last input to FtK in line 3). Therefore, by correctness of the one-time AEAD scheme $(\mathsf{EtM}, \mathsf{VtD})$ (Definition 11), we conclude that $\mathsf{acc} = \mathsf{true}$ and therefore $\neg\mathsf{acc} = \mathsf{false}$.

[27] We assume $\log|\mathcal{S}| \ll \lambda$ so that we do not rely on computational collision resistance.

- $\underline{(\mathsf{real}[j] = \mathsf{dtct})}$. To show that this conjunction is indeed false, we prove that $\mathsf{real}[j] \implies \neg\mathsf{dtct}$. Assume $\mathsf{real}[j] = \mathsf{true}$ which implies that the call to Request on line 5 of the CORRECT game used $\mathsf{ps} = \mathsf{ps}_1$ (see line 6 of the game), therefore $\mathsf{com}' = \mathsf{com}$ in the Response procedure in Fig. 10 which implies that $\mathsf{dtct} = \mathsf{false}$ (see line 6 of the Response procedure). The implication in the other direction follows from the $(\mathcal{S}, \varepsilon)$-injectivity of F and $k_{\mathsf{com}} \leftarrow\!\$\, \mathcal{K}_{\mathsf{PRF}}$, i.e. with probability $1-\varepsilon$ over the randomness in k_{com} there are no collisions on $\mathcal{S}$ under $\mathsf{F}(k_{\mathsf{com}}, \cdot)$. Therefore $\mathsf{real} \wedge \mathsf{dtct} = \mathsf{false}$ with probability $1-\varepsilon$.

We conclude that the boolean expression $\neg\mathsf{acc} \vee (\mathsf{real} \wedge \mathsf{dtct})$ is false, which implies that $\mathsf{sched}[k]$ that parses as $(\text{“response”}, j)$ for $j \in \mathbb{N}$ does not break correctness given correctness of the one-time AEAD scheme $(\mathsf{EtM}, \mathsf{VtD})$.

2. Assume $\mathsf{sched}[k]$ parses as $(\text{“getkey”}, j)$ for $j \in \mathbb{N}$. We assume that the clause $\neg\mathsf{sent}_{\mathsf{sm}}[j] \vee \mathsf{recv}_{\mathsf{sm}}[j] \vee \mathsf{last}_{\mathsf{sm}} > j$ is false, otherwise correctness of this case holds by construction. Therefore we have $\neg\mathsf{sent}_{\mathsf{sm}}[j] = \mathsf{false}$, $\mathsf{recv}_{\mathsf{sm}}[j] = \mathsf{false}$ and $\mathsf{last}_{\mathsf{cm}} \leq j$, which implies that $\mathsf{sent}_{\mathsf{sm}}[j] = \mathsf{true}$ and that $\mathsf{last}_{\mathsf{sm}} < j$. The latter inequality holds because the case $\mathsf{last}_{\mathsf{sm}} = j$ is ruled out by $\mathsf{recv}_{\mathsf{sm}}[j] = \mathsf{false}$, i.e. the CORRECT game does not allow to call GetKey twice on the same message. By induction hypothesis we know that $\mathsf{sm}[j]$ is the correct output of a call to Response on line 10 of the game. We now show that all the conjunctions on line 16 of the CORRECT game are false, which implies that the entire disjunction is false.

- $\underline{(\neg\mathsf{cleared} \wedge (\neg\mathsf{acc} \vee k' \neq k))}$. To prove that this conjunction is false, we prove that

$$\neg\mathsf{cleared} \implies \neg(\neg\mathsf{acc} \vee k' \neq k),$$

  which by De Morgan's law is equivalent to

$$\neg\mathsf{cleared} \implies (\mathsf{acc} \wedge k' = k).$$

  We assume $\neg\mathsf{cleared} = \mathsf{true}$, which means $\mathsf{cleared} = \mathsf{false}$. This means that there exists a $\ell < k$ such that $\mathsf{sched}[\ell]$ parses as $(\text{“request”}, \mathsf{ps})$ for $\mathsf{ps} \in \mathcal{S}$ and that no entry that parses as (“clear”) exists in the entries from $\mathsf{sched}[\ell]$ to $\mathsf{sched}[k]$. Moreover, we know that $\mathsf{last}_{\mathsf{sm}} < j$, so we know that GetKey on line 15 did not process $\mathsf{sm}[j]$ yet. Moreover, $\mathsf{last}_{\mathsf{sm}} < j$, implies that $\mathsf{sm}[j]$ has a $\mathsf{cnt}_{\mathsf{sm}}[j]$ greater than any sm that GetKey on line 15 of the game has already processed (see line 8 of the Response procedure in Fig. 10). Therefore the condition on line 1 of the FtK function in Fig. 10 holds and the key is advanced to $\mathsf{cnt}_{\mathsf{cm}}[j]$ (see line 3 of GetKey in Fig. 10), i.e. the counter that the correct call to Response for $\mathsf{sm}[j]$ sets. Therefore, by correctness of the one-time AEAD scheme $(\mathsf{EtM}, \mathsf{VtD})$ (Definition 11), we have that $\mathsf{acc} = \mathsf{true}$. By construction of the GetKey procedure (see line 6) we know that when the procedure accepts a message (that is, $\mathsf{acc} = \mathsf{true}$), then it returns $k \leftarrow k_0 \oplus k_1$, and therefore $k' = k$.
- $\underline{(\mathsf{cleared} \wedge \mathsf{acc})}$. To prove that this conjunction is false, we prove that

$$\mathsf{cleared} \implies \neg\mathsf{acc}.$$

  We assume $\mathsf{cleared} = \mathsf{true}$. This means that there exists a $\ell < k$ such that $\mathsf{sched}[\ell]$ parses as (“clear”) and that no entry that parses as $(\text{“request”}, \mathsf{ps})$ for $\mathsf{ps} \in \mathcal{S}$ exists in the entries from $\mathsf{sched}[\ell]$ to $\mathsf{sched}[k]$. Moreover, we know that $\mathsf{last}_{\mathsf{sm}} < j$, so we know that GetKey on line 15 did not process $\mathsf{sm}[j]$ yet. The call to Clear of $\mathsf{sched}[\ell]$ advances the ratchet $\mathsf{rs}_{\mathsf{sm}}$ and the associated counter $\mathsf{cnt}_{\mathsf{sm}}$ to the level that $\mathsf{cnt}_{\mathsf{cm}}$ represents. By assumption we know that the protocol is correct up to this entry $\mathsf{sched}[k]$, therefore $\mathsf{cnt}_{\mathsf{cm}}$ in the Clear procedure that $\mathsf{sched}[\ell]$ triggers corresponds to the greater counter that the procedure Request sets., i.e. after the FtK call in Clear we have $\mathsf{cnt}_{\mathsf{sm}} = \mathsf{cnt}_{\mathsf{cm}}$. Since by construction we have that $\mathsf{cnt}_{\mathsf{sm}} \leq \mathsf{cnt}_{\mathsf{cm}}$, i.e. the server always catches up on client messages, condition on line 1 of the FtK procedure that the GetKey procedure calls on line 3 does not hold and FtK returns $k_{\mathsf{sm}} = \perp$, which implies that $\mathsf{acc} = \mathsf{false}$ and concludes the proof for this case.

Since $\mathsf{sched}[k]$ does not break correctness except with probability ε over k_{com}, we conclude by induction that the protocol is ε-correct. □

Theorem 2 (KIND-AtC-AB**).** *Consider the construction in Fig. 10. Let $\mathcal{S}$ be a personal secret distribution and let F be $(\mathcal{S}, \varepsilon)$-injective (Definition 20). For any PPT adversary $\mathcal{A}$ in the* KIND-AtC-AB *game (Fig. 8 and Definition 16) it holds:*

$$\begin{aligned}\mathsf{Adv}^{\text{KIND-AtC-AB}}_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) \leq\ & (4 + 3\,n_{\mathsf{cm}} + 2\,n_{\mathsf{sm}}) \cdot \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda) \\ & + (2\,n_{\mathsf{cm}} + n_{\mathsf{sm}}) \cdot \mathsf{Adv}^{\text{OT-IND\$-CPA}}_{\mathsf{AEAD}}(1^\lambda) \\ & + (n_{\mathsf{cm}} + n_{\mathsf{sm}}) \cdot \mathsf{Adv}^{\text{OT-EUF-CMA}}_{\mathsf{AEAD}}(1^\lambda) + \varepsilon\ .\end{aligned}$$

Proof. The proof proceeds through a sequence of games, bounding the advantage of an efficient adversary $\mathcal{A}$ in the KIND-AtC-AB game (Fig. 8 and Definition 16). In what follows we split our analysis into four cases:

- **Case 1**: $\mathcal{A}$ wins by causing $\mathsf{win}_{\mathsf{cm}} \leftarrow \mathsf{true}$ or $\mathsf{win}_{\mathsf{sm}} \leftarrow \mathsf{true}$;
- **Case 2**: $\mathcal{A}$ wins by causing $(\mathsf{recv} \wedge \neg\mathsf{dtct}) \leftarrow \mathsf{true}$ while $b = 0$.
- **Case 3**: $\mathcal{A}$ determines the bit d.
- **Case 4**: $\mathcal{A}$ determines the bit b.

Case 1. GAME G_0. We inline the construction (Fig. 10) into the KIND-AtC-AB game (Fig. 8). Therefore

$$\mathsf{Adv}^{\text{KIND-AtC-AB}}_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) = \mathsf{Adv}^{\mathrm{G}_0}_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) \ .$$

GAME G_1. In this game we replace the computation of the keys $(\mathsf{rs}_{\mathsf{cm}}, \mathsf{rs}_{\mathsf{sm}}) \leftarrow \mathsf{KDF}(\kappa)$ with uniformly random and independent strings. We do so by reduction $\mathcal{B}_1$ to a KDF challenger – specifically, when $\mathcal{B}_1$ begins the experiment, it initialises a KDF challenger $\mathcal{C}_{\mathsf{kdf}}$ and immediately calls it. Otherwise $\mathcal{B}_1$ acts exactly as the challenger $\mathcal{C}$ in the KIND-AtC-AB security experiment. We note that $\mathcal{A}$ cannot learn the value κ, which itself is a uniformly random string, and thus this replacement is sound. If the bit b sampled by $\mathcal{C}_{\mathsf{kdf}}$ is 0, then $\mathsf{rs}'_{\mathsf{cm}}, \mathsf{rs}'_{\mathsf{sm}} \leftarrow \mathsf{KDF}(\kappa)$, and we are in G_0. Otherwise, $\mathsf{rs}'_{\mathsf{cm}}, \mathsf{rs}'_{\mathsf{sm}} \xleftarrow{\mathrm{R}} \{0,1\}^\lambda$ and we are in G_1. Any adversary $\mathcal{A}$ that can distinguish between G_0 and G_1 with non-negligible probability can be used to break the KDF security of the underlying KDF primitive. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_0}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_1}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda, \mathcal{B}_1) \ .$$

GAME G_2. In this game we replace the computation of all keys $(k_{\mathsf{cm}}, \mathsf{rs}_{\mathsf{cm}}) \leftarrow \mathsf{KDF}(\mathsf{rs}_{\mathsf{cm}})$ with uniformly random and independent strings before the adversary outputs σ. Let n_{cm} be the number of times the value $\mathsf{rs}_{\mathsf{cm}}$ is ratcheted forward, i.e. $\max(n_{crq}, n_{\mathsf{cnt}'})$ where n_{crq} is the number of times the adversary queries REQUEST and $n_{\mathsf{cnt}'}$ is the highest $\mathsf{cm}_{\mathsf{cnt}'}$ sent by the attacker in either RESPONSE(cm) or DETECT(cm). We do so by reduction $\mathcal{B}_2$ to a KDF challenger – specifically, when $\mathcal{B}_2$ computes $(k_{\mathsf{cm}}, \mathsf{rs}_{\mathsf{cm}}) \leftarrow \mathsf{KDF}(\mathsf{rs}'_{\mathsf{cm}})$ for the first time, it initialises a KDF challenger $\mathcal{C}_{\mathsf{kdf}}$ and immediately calls it on the empty string $\emptyset$. $\mathcal{B}_2$ then replaces $(k_{\mathsf{cm}}, \mathsf{rs}_{\mathsf{cm}})$ with $(k'_{\mathsf{cm}}, \mathsf{rs}'_{\mathsf{cm}})$, the output of $\mathcal{C}_{\mathsf{kdf}}$. We note here that each call to $\mathcal{C}_{\mathsf{kdf}}$ replaces a computation of $\mathsf{KDF}(\mathsf{rs}'_{\mathsf{cm}})$, where $\mathsf{rs}'_{\mathsf{cm}}$ is already a uniformly random and independent value (by G_1 or a previous replacement in G_2), and thus this replacement is sound. Otherwise $\mathcal{B}_2$ acts exactly in G_1. If the bit b sampled by $\mathcal{C}_{\mathsf{kdf}}$ is 0, then $k'_{\mathsf{cm}}, \mathsf{rs}'_{\mathsf{cm}} \leftarrow \mathsf{KDF}(\mathsf{rs}'_{cm})$, and we are in G_1. Otherwise, $k'_{\mathsf{cm}}, \mathsf{rs}'_{\mathsf{cm}} \xleftarrow{\mathrm{R}} \{0,1\}^\lambda$ and we are in G_2. Any adversary $\mathcal{A}$ that can distinguish between G_1 and G_2 with non-negligible probability can be used to break the KDF security of the underlying KDF primitive. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_1}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_2}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{cm}} \cdot \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda, \mathcal{B}_2) \ .$$

GAME G_3. In this game, we introduce an abort event that triggers the first time the following condition in RESPONSE (line 2) evaluates true: if $\mathsf{acc} = \mathsf{true}$, but $\mathsf{cm} \notin \mathcal{C}$, $\mathsf{win} \leftarrow \mathsf{true}$. Specifically, we begin by guessing the key k'_{cm} (replaced in G_2) that is used when the event above is triggered. There are n_{cm} possible keys replaced in G_2, which places an upper bound on this guess. We define a reduction $\mathcal{B}_3$ that, when it computes k'_{cm}, initialises an OT-EUF-CMA AEAD challenger $\mathcal{C}_{\text{OT-EUF-CMA}}$. Whenever $\mathcal{B}_3$ is required to encrypt using k'_{cm}, it instead queries $(\mathsf{pt}, \mathsf{ad})$ to the $\mathcal{C}_{\text{OT-EUF-CMA}}$ encrypt oracle, and replaces the computation of $(\mathsf{ct}, \mathsf{at})$ with ct', the output of the $\mathcal{C}_{\text{OT-EUF-CMA}}$. Note that whenever it does so, $\mathcal{B}_3$ adds $(\mathsf{ct}', \mathsf{at}', \mathsf{ad})$ to C, the set of honestly generated ciphertexts. $\mathcal{B}_3$ must also maintain a lookup table $CT[k'_{\mathsf{cm}}] \leftarrow (\mathsf{ct}', \mathsf{at}', \mathsf{ad}, \mathsf{pt})$, which $\mathcal{B}_3$ can use to 'decrypt' ciphertexts. Whenever $\mathcal{B}_3$ is required to decrypt ct^* using k'_{cm} it instead uses the lookup table. If $(\mathsf{ct}^*, \mathsf{at}^*, \mathsf{ad}^*, \mathsf{pt}) \in CT[k'_{\mathsf{cm}}]$, then $\mathcal{B}_3$ replaces the decryption of ct^* with pt, otherwise $\mathcal{B}_3$ returns $\perp$. Note that if our abort event triggers, this means a ciphertext $(\mathsf{ct}^*, \mathsf{at}^*, \mathsf{ad}^*)$ was never the output of $\mathcal{C}_{\text{OT-EUF-CMA}}$, but decrypted successfully, thus breaking the OT-EUF-CMA security of the AEAD scheme. Thus, triggering the abort event immediately is turned into an attacker against the AEAD scheme and we have

$$\mathsf{Adv}^{\mathrm{G}_2}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_3}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{cm}} \cdot \mathsf{Adv}^{\text{OT-EUF-CMA}}_{\mathsf{AEAD}}(1^\lambda, \mathcal{B}_3) \ .$$

We note that since we abort in G_3 instead of triggering $\mathsf{win}_{\mathsf{cm}} \leftarrow \mathsf{true}$, this condition on line 7 never triggers.

Our proof strategy continues identically with $\mathsf{win}_{\mathsf{sm}} \leftarrow \mathsf{true}$.

GAME G_4. In this game we replace the computation of all keys $(k_{\mathsf{sm}}, \mathsf{rs}_{\mathsf{sm}}) \leftarrow \mathsf{KDF}(\mathsf{rs}_{\mathsf{sm}})$ with uniformly random and independent strings before the adversary outputs σ. Let n_{sm} be the number of times the value $\mathsf{rs}_{\mathsf{sm}}$ is ratcheted forward. We do so by reduction $\mathcal{B}_4$ to a KDF challenger similarly to G_2 (up to a change in notation) with the same bounds. Any adversary $\mathcal{A}$ that can distinguish between G_3 and G_4 with non-negligible probability can be used to break the KDF security of the underlying KDF primitive. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_3}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_4}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{sm}} \cdot \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda, \mathcal{B}_4) \ .$$

GAME G_5. In this game, we introduce an abort event that triggers the first time the following condition in GETKEY (line 2) evaluates true: if $\mathsf{acc} = \mathsf{true}$, but $\mathsf{sm} \notin \mathcal{Z}$, $\mathsf{win} \leftarrow \mathsf{true}$. We do so by reduction $\mathcal{B}_5$ to a OT-EUF-CMA AEAD challenger similarly to G_3 (up to a change in notation) with the same bounds. Any adversary that can trigger the abort event immediately is turned into an attacker against the AEAD scheme and we have

$$\mathsf{Adv}^{\mathrm{G}_4}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_5}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{sm}} \cdot \mathsf{Adv}^{\text{OT-EUF-CMA}}_{\mathsf{AEAD}}(1^\lambda, \mathcal{B}_5) \ .$$

We note that in G_5 the adversary cannot trigger either winning condition on Line 7 and thus summing the advantages above we find our bound.

Case 2. In this case we address the winning condition 'if $b = 0 \wedge \mathsf{recv} \wedge \neg\mathsf{dtct}$'. Note that it is mutually exclusive of all other winning conditions: if the winning condition of line 7 is triggered then the experiment cannot reach line 13, and similarly line 14 cannot be reached when line 13 triggers. Line 12 and line 13 are also mutually exclusive: line 12 requires $\neg\mathsf{recv} = \mathsf{true}$ and line 13 requires $\mathsf{recv} = \mathsf{true}$.

GAME G_0. We inline the construction (Fig. 10) into the KIND-AtC-AB game (Fig. 8). Therefore

$$\mathsf{Adv}^{\text{KIND-AtC-AB}}_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) = \mathsf{Adv}^{\mathrm{G}_0}_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) \ .$$

GAME G_1. In this game we replace the computation of the keys $(\mathsf{rs}_{\mathsf{cm}}, \mathsf{rs}_{\mathsf{sm}}) \leftarrow \mathsf{KDF}(\kappa)$ with uniformly random and independent strings. We do so by reduction $\mathcal{B}_6$ to a KDF challenger that proceeds identically to the reduction $\mathcal{B}_1$ in G_1 of **Case 1**. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_0}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_1}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda, \mathcal{B}_6) \ .$$

GAME G_2. In this game we replace the computation of the keys $(k_{\mathsf{cm}}, \mathsf{rs}_{\mathsf{cm}}) \leftarrow \mathsf{KDF}(\mathsf{rs}'_{\mathsf{cm}})$ with uniformly random and independent strings. We do so by reduction $\mathcal{B}_7$ to a KDF challenger that proceeds identically to the reduction $\mathcal{B}_2$ in G_2 of **Case 1**. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_1}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_2}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{cm}} \cdot \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda, \mathcal{B}_7) \ .$$

GAME G_3. In this game, we replace the ciphertext output of REQUEST with uniformly random strings before the adversary outputs σ. We do so by defining a reduction $\mathcal{B}_8$ that, when it computes k'_{cm}, also initialises an OT-IND$-CPA AEAD challenger $\mathcal{C}_{\text{OT-IND\$-CPA}}$. Whenever $\mathcal{B}_8$ is required to encrypt using k'_{cm} (replaced in G_2), it instead queries $(\mathsf{pt}, \mathsf{ad})$ to the $\mathcal{C}_{\text{OT-IND\$-CPA}}$ encrypt oracle, and replaces the computation of $(\mathsf{ct}, \mathsf{at})$ with ct', the output of the $\mathcal{C}_{\text{OT-EUF-CMA}}$. $\mathcal{B}_8$ must also maintain a lookup table $CT[k'_{\mathsf{cm}}] \leftarrow (\mathsf{ct}', \mathsf{at}', \mathsf{ad}, \mathsf{pt})$, which $\mathcal{B}_8$ can use to 'decrypt' ciphertexts. Whenever $\mathcal{B}_8$ is required to decrypt ct^* using k'_{cm} it instead uses the lookup table. If $(\mathsf{ct}^*, \mathsf{at}^*, \mathsf{ad}^*, \mathsf{pt}) \in CT[k'_{\mathsf{cm}}]$, then $\mathcal{B}_8$ replaces the decryption of ct^* with pt, otherwise $\mathcal{B}_8$ returns $\perp$. There are n_{cm} possible keys replaced in G_2, which places an upper bound on the number of AEAD challengers initialised in this way. If the bit b sampled by $\mathcal{C}_{\text{OT-IND\$-CPA}}$ for any given key k'_{cm} is 0, then we encrypt normally and we are in G_2. Otherwise, $\mathcal{B}_8$ proceeds as in G_3. Thus, any adversary able to distinguish between G_2 and G_3 can be turned into an efficieny adversary against the OT-IND$-CPA security of the AEAD scheme and thus we have:

$$\mathsf{Adv}^{\mathrm{G}_2}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_3}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{cm}} \cdot \mathsf{Adv}^{\text{OT-IND\$-CPA}}_{\mathsf{AEAD}}(1^\lambda, \mathcal{B}_8) \ .$$

We note that after G_3 the adversary is never given any output that contains com (or something derived from com). In order for the winning condition to hold, the adversary must call DETECT with cm such that:

- cm decrypts validly
- $\mathsf{dtct}' \leftarrow (\mathsf{com} \neq \mathsf{com}') = \mathsf{false}$

By definition, $b = 0$ and thus $\mathcal{A}$ does not have ps such that $\mathsf{com} = F(k_{\mathsf{com}}, \mathsf{ps})$. Thus, $\mathcal{A}$ must either guess com or ps.

GAME G_4. In this game we introduce an abort event that triggers if, after $\mathcal{A}$ outputs σ, they query DETECT(cm) such that cm was not output by REQUEST and $\mathsf{dtct} = \mathsf{false}$. As reasoned above, $\mathcal{A}$ must have guessed the ps or com. Now, note that $\mathsf{F}(\cdot,\cdot)$ is $(\mathcal{S}, \varepsilon)$-injective, $k_{\mathsf{com}} \leftarrow\!\!\$\ \mathcal{K}_{\mathsf{com}}$. With probability $1 - \varepsilon$ there are no collisions in $\mathsf{F}(k_{\mathsf{com}}, \mathcal{S})$ and the guessing advantage for com is at most the advantage for guessing ps_1 given ps_0, i.e. $\Pr[G]$ in the notation of Remark 2. Recall that we defined the advantage of $\mathcal{A}$ as doing better than $\Pr[G]$ and we obtain:

$$\mathsf{Adv}^{\mathrm{G}_3}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_4}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + \varepsilon \ .$$

Since we now abort if $\mathcal{A}$ guesses correctly and uses com to evade detection, $\mathsf{Adv}^{\mathrm{G}_4}_{\mathcal{S},\Pi}(\mathcal{A}) = 0$ and we have our advantage statement.

Case 3. In this case we bound the probability of $\mathcal{A}$ guessing the bit d. Based on the winning condition on line 12, we note that $\mathcal{A}$ can never receive an output from DETECT: recv is set to true before any output is returned. Thus, in what follows we will prevent the adversary from learning any information about the server-maintained $\varkappa_1$. Then, we replace all encryptions using k with uniformly random strings and argue that the bit d is, in this case, indistinguishable.

GAME G_0. We inline the construction (Fig. 10) into the KIND-AtC-AB game (Fig. 8). Therefore

$$\mathsf{Adv}^{\text{KIND-AtC-AB}}_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) = \mathsf{Adv}^{\mathrm{G}_0}_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) \ .$$

GAME G_1. In this game we replace the computation of the keys $(\mathsf{rs}_{\mathsf{cm}}, \mathsf{rs}_{\mathsf{sm}}) \leftarrow \mathsf{KDF}(\kappa)$ with uniformly random and independent strings. We do so by reduction $\mathcal{B}_9$ to a KDF challenger that proceeds identically to the reduction $\mathcal{B}_1$ in G_1 of **Case 1**. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_0}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_1}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda, \mathcal{B}_9) \ .$$

GAME G_2. In this game we replace the computation of all keys $(k_{\mathsf{sm}}, \mathsf{rs}_{\mathsf{sm}}) \leftarrow \mathsf{KDF}(\mathsf{rs}_{\mathsf{sm}}$ with uniformly random and independent strings before the adversary outputs σ. We do so by reduction $\mathcal{B}_{10}$ to a KDF challenger that proceeds identically to the reduction $\mathcal{B}_1$ in G_4 of **Case 1**. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_1}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_2}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{sm}} \cdot \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda, \mathcal{B}_{10}) \ .$$

GAME G_3. In this game we replace the ciphertext output of RESPONSE with a uniformly random string before the adversary outputs σ. We do so by defining a reduction $\mathcal{B}_{11}$ as in G_3 of **Case 2**, which proceeds identically up to a change in notation. Thus, any adversary able to distinguish between G_2 and G_3 can be turned into an efficient adversary against the OT-IND\$-CPA AEAD game and thus we have:

$$\mathsf{Adv}^{\mathrm{G}_2}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_3}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{sm}} \cdot \mathsf{Adv}^{\text{OT-IND\$-CPA}}_{\mathsf{AEAD}}(1^\lambda, \mathcal{B}_{11}) \ .$$

GAME G_4. We note that by G_3, $\mathcal{A}$ never learns k_1 itself, only $\varkappa_0$ that ds contains, which is one-time pad encryption of k_1 using $\varkappa_1$, i.e. the server-maintained share, which is never communicated to the adversary. Thus, k_1 is itself a uniformly random and independent string. Recall from Remark 2 that $\Pr[D]$ is the probability that an adversary guessing d given only k_d. By the arguments that we outline above, $\mathcal{A}$ never learns k_1 (nor anything derived from k_1). We note that in G_3, the key k_1 is not sent (nor anything derived from k_1) to the adversary. All outputs from RESPONSE are now uniformly random and independent of the protocol flow. Thus the adversary can do no better than guessing d given k_d: $\mathsf{Adv}^{\mathrm{G}_3}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) = 0$ since we defined the advantage for this case as doing better than guessing d given ps_b.

Case 4. In this case, we bound the probability of our adversary guessing the bit b. We note that the behaviour of the oracles only differ based on the bit b during the $\mathsf{Request}$ and $\mathsf{GeneralSetup}$ algorithms. In the $\mathsf{Request}$ algorithm the client will always use ps_1 to compute the commitment. In what follows, we hide this information from the adversary. All other interactions proceed identically regardless of the value ps used.

GAME G_0. We inline the construction (Fig. 10) into the KIND-AtC-AB game (Fig. 8). Therefore

$$\mathsf{Adv}^{\text{KIND-AtC-AB}}_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) = \mathsf{Adv}^{\mathrm{G}_0}_{\mathcal{S},\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) \ .$$

GAME G_1. In this game we replace the computation of the keys $(\mathsf{rs}_{\mathsf{cm}}, \mathsf{rs}_{\mathsf{sm}}) \leftarrow \mathsf{KDF}(\kappa)$ with uniformly random and independent strings. We do so by reduction $\mathcal{B}_{13}$ to a KDF challenger that proceeds identically to the reduction $\mathcal{B}_1$ in G_1 of **Case 1**. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_0}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_1}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda, \mathcal{B}_{13}) \ .$$

GAME G_2. In this game we replace the computation of the keys $(k_{\mathsf{cm}}, \mathsf{rs}_{\mathsf{cm}}) \leftarrow \mathsf{KDF}(\mathsf{rs}'_{\mathsf{cm}})$ with uniformly random and independent strings. We do so by reduction $\mathcal{B}_{13}$ to a KDF challenger that proceeds identically to the reduction $\mathcal{B}_2$ in G_2 of **Case 1**. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_1}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_2}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{cm}} \cdot \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda, \mathcal{B}_{14}) \ .$$

GAME G_3. In this game, we replace the ciphertext output of REQUEST with uniformly random strings before the adversary outputs σ. We do so by defining a reduction $\mathcal{B}_{14}$ that proceeds identically to the reduction $\mathcal{B}_8$ in G_3 of **Case 2**. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_2}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_3}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{cm}} \cdot \mathsf{Adv}^{\text{OT-IND\$-CPA}}_{\mathsf{AEAD}}(1^\lambda, \mathcal{B}_{15}) \ .$$

Recall from Remark 2 that $\Pr[B]$ is the probability that an adversary guessing b given only ps_b. We note that in G_3, the personal secret ps_1 not sent (nor anything derived from ps_1) to the adversary. All outputs from REQUEST are now uniformly random and independent of the protocol flow. All outputs from RESPONSE are computed identically regardless of whether ps_0 or ps_1 is used. Thus the adversary can do no better than guessing b given ps_b: $\mathsf{Adv}^{\mathrm{G}_3}_{\mathcal{S},\Pi}(\mathcal{A}) = 0$ since we defined the advantage for this case as doing better than guessing b given ps_b. □

Theorem 3 (IND-CPA-AtC-AB**).** *Consider the construction in Fig. 10. Let $\mathcal{S}$ be a personal secret distribution and let F be $(\mathcal{S}, \varepsilon)$-injective (Definition 20). For any PPT adversary $\mathcal{A}$ in the* IND-CPA-AtC-AB *game (Fig. 8 and Definition 17) it holds:*

$$\begin{aligned}\mathsf{Adv}^{\text{IND-CPA-AtC-AB}}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) \leq\ & (4 + 3\,n_{\mathsf{cm}} + 2\,n_{\mathsf{sm}}) \cdot \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda) \\ & + (2\,n_{\mathsf{cm}} + n_{\mathsf{sm}}) \cdot \mathsf{Adv}^{\text{OT-IND\$-CPA}}_{\mathsf{AEAD}}(1^\lambda) \\ & + (n_{\mathsf{cm}} + n_{\mathsf{sm}}) \cdot \mathsf{Adv}^{\text{OT-EUF-CMA}}_{\mathsf{AEAD}}(1^\lambda) \\ & + \mathsf{Adv}^{\text{IND-CPA}}_{\Pi}(1^\lambda) + \varepsilon \ .\end{aligned}$$

Proof. The proof proceeds through a sequence of games, bounding the advantage of an efficient adversary $\mathcal{A}$ in the IND-CPA-AtC-AB game (Fig. 8 and Definition 17). In what follows we split our analysis into four cases:

- **Case 1**: $\mathcal{A}$ wins by causing $\mathsf{win}_{\mathsf{cm}} \leftarrow \mathsf{true}$ or $\mathsf{win}_{\mathsf{sm}} \leftarrow \mathsf{true}$;
- **Case 2**: $\mathcal{A}$ wins by causing $(\mathsf{recv} \wedge \neg\mathsf{dtct}) \leftarrow \mathsf{true}$ while $b = 0$.
- **Case 3**: $\mathcal{A}$ determines the bit d.
- **Case 4**: $\mathcal{A}$ determines the bit b.

Case 1, **Case 2** and **Case 4**, proceed identically to the proof of Theorem 2. We therefore focus on **Case 3**.

Case 3. We bound the probability of $\mathcal{A}$ guessing the bit d. Based on the winning condition on line 12, we note that $\mathcal{A}$ can never receive an output from DETECT: recv is set to true before any output is returned. Thus, in what follows we will prevent the adversary from learning any information about the server-maintained $\varkappa_1$. Then, we replace all encryptions using k with uniformly random strings and argue that the bit d is, in this case, indistinguishable.

GAME G_0. We inline the construction (Fig. 10) into the IND-CPA-AtC-AB game (Fig. 8). Therefore

$$\mathsf{Adv}^{\text{IND-CPA-AtC-AB}}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) = \mathsf{Adv}^{\mathrm{G}_0}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) \ .$$

GAME G_1. In this game we replace the computation of the keys $(\mathsf{rs}_{\mathsf{cm}}, \mathsf{rs}_{\mathsf{sm}}) \leftarrow \mathsf{KDF}(\kappa)$ with uniformly random and independent strings. We do so by reduction $\mathcal{B}_9$ to a KDF challenger that proceeds identically to the reduction $\mathcal{B}_1$ in G_1 of **Case 1** in the proof of Theorem 2. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_0}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_1}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) + \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda, \mathcal{B}_9) \ .$$

GAME G_2. In this game we replace the computation of all keys $(k_{\mathsf{sm}}, \mathsf{rs}_{\mathsf{sm}}) \leftarrow \mathsf{KDF}(\mathsf{rs}_{\mathsf{sm}}$ with uniformly random and independent strings before the adversary outputs σ. We do so by reduction $\mathcal{B}_{10}$ to a KDF challenger that proceeds identically to the reduction $\mathcal{B}_1$ in G_4 of **Case 1** in the proof of Theorem 2. Thus we have:

$$\mathsf{Adv}^{\mathrm{G}_1}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_2}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{sm}} \cdot \mathsf{Adv}^{\text{IND-KDF}}_{\mathsf{KDF}}(1^\lambda, \mathcal{B}_{10}) \ .$$

GAME G_3. In this game we replace the ciphertext output of RESPONSE with a uniformly random string before the adversary outputs σ. We do so by defining a reduction $\mathcal{B}_{11}$ as in G_3 of **Case 2** in the proof of Theorem 2, which proceeds identically up to a change in notation. Thus, any adversary able to distinguish between G_2 and G_3 can be turned into an efficient adversary against the OT-IND$-CPA AEAD game and thus we have:

$$\mathsf{Adv}^{\mathrm{G}_2}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_3}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) + n_{\mathsf{sm}} \cdot \mathsf{Adv}^{\text{OT-IND\$-CPA}}_{\mathsf{AEAD}}(1^\lambda, \mathcal{B}_{11}) \ .$$

GAME G_4. By G_3, $\mathcal{A}$ never learns k_1 itself, only $\varkappa_0$, which is one-time pad encryption of k_1 using $\varkappa_1$, which is never communicated to the adversary. Thus, k_1 is itself a uniformly random and independent string. We define a reduction $\mathcal{B}_{12}$ that, when the game begins, initialises an IND-CPA challenger $\mathcal{C}_{\text{IND-CPA}}$. Whenever $\mathcal{A}$ issues a $\textsc{Enc}(m_0, m_1)$ query, $\mathcal{B}_{12}$ simply forwards the query to the IND-CPA challenger. When $\mathcal{A}$ terminates and outputs d (and $\mathsf{recv} = \mathsf{false}$, as in the definition of this case), we return d to the IND-CPA challenger. We note that in this case, our advantage is exactly equal to the advantage in breaking the IND-CPA game and thus we find:

$$\mathsf{Adv}^{\mathrm{G}_3}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{G}_4}_{\mathcal{S},\Pi,\mathsf{ABOM}}(\mathcal{A}) + \mathsf{Adv}^{\text{IND-CPA}}_{\Pi}(1^\lambda, \mathcal{B}_{12}) \ .$$

Theorem 4 (KIND-OBLIVIOUS**).** *Consider the construction in Fig. 10. Let* F *be a function. For any PPT adversary* $\mathcal{A}$ *in the* KIND-OBLIVIOUS *game (Fig. 9 and Definition 18) we have that:*

$$\mathsf{Adv}^{\text{KIND-OBLIVIOUS}}_{\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\text{IND-PRF}}_{\mathsf{F}}(1^\lambda) \ .$$

Proof. Let E_w be the event that the adversary outputs $w = w'$ for $w \in \{d, b\}$. By the union bound, we can upper-bound the advantage of the adversary $\mathcal{A}$ in winning the OBLIVIOUS game as

$$\begin{aligned}\mathsf{Adv}^{\text{KIND-OBLIVIOUS}}_{\mathcal{K},\mathsf{ABOM}}(\mathcal{A}) &= \left|\Pr[\text{KIND-OBLIVIOUS}_{\mathcal{K},\mathsf{ABOM}}(1^\lambda, \mathcal{A}) \Rightarrow 1] - 3/4\right| \\ &\leq |\Pr[\mathsf{E}_d] - 1/2| + |\Pr[\mathsf{E}_b] - 1/2|,\end{aligned}$$

since $0 \leq \Pr[\mathsf{E}_d \wedge \mathsf{E}_b] \leq 1/4$. We first prove that $|\Pr[\mathsf{E}_d] - 1/2| = 0$ and then that $|\Pr[\mathsf{E}_b] - 1/2|$ is negligible.

Observer that $\mathcal{A}$ never learns k_1 itself, but only $\varkappa_1$ that is contained in the server state ss, which is a one-time pad encryption of k_1 using $\varkappa_0$, which is never communicated to the adversary. More precisely, in the alert phase of the ABOM protocol (Fig. 10), the client never sends the share $\varkappa_0$ to the server, so ss is independent of the key k_1 as it only contains the random share $\varkappa_1$. Moreover, the outputs of $\mathsf{Request}$ are

independent of k_1 and the same holds for the output of Clear. The ds output of GetKey is also independent of the key k_1. Therefore, $\mathcal{A}$ can only randomly guess the KIND bit d, which implies that

$$|\Pr[\mathsf{E}_d] - 1/2| = |1/2 - 1/2| = 0 \ .$$

We now tackle the second term, i.e. $|\Pr[\mathsf{E}_b] - 1/2|$. Let G_1 be the same as the KIND-OBLIVIOUS game of Fig. 9 but with the PRF replaced by a random function $f \xleftarrow{\mathrm{R}} \mathsf{Func}(n, \ell(n))$ (we define $\mathsf{Func}(n, \ell(n))$ in Definition 3). In particular, we change line 3 of Setup as $\mathsf{com} \leftarrow f(\mathsf{ps})$ and line 2 of $\mathsf{Request}$ as $\mathsf{com} \leftarrow f(\mathsf{ps})$. Assume a distinguisher $\mathcal{D}$ that can differentiate the two games with a non-negligible probability ν. We build an adversary $\mathcal{B}'$ that plays the $\mathsf{IND\text{-}PRF}$ game (Definition 3). The adversary $\mathcal{B}'$ simulates all the oracles of the two games, which are the same. Whenever $\mathcal{B}'$ needs the commitment on ps_b (to run $\mathsf{GeneralSetup}$ and to simulate the REQUEST oracle, which always use the same ps_b), $\mathcal{B}'$ uses their own DERIVE oracle (Fig. 1). Finally, $\mathcal{B}'$ outputs the bit that the distinguisher outputs. Therefore we have

$$|\Pr[\mathcal{D}(\mathrm{G}_0) \Rightarrow 1] - \Pr[\mathcal{D}(\mathrm{G}_1) \Rightarrow 1]| \leq \mathsf{Adv}^{\text{IND-PRF}}_{\mathsf{F}}(\mathcal{B}') \ .$$

Finally, note that in the game G_1 the commitment com is a random bit string independent of b and therefore the adversary can only randomly guess the bit b. Therefore we conclude that

$$|\Pr[\mathsf{E}_b] - 1/2| \leq \mathsf{Adv}^{\text{IND-PRF}}_{\mathsf{F}}(\mathcal{B}') \ . \qquad \square$$

Theorem 5 (IND-CPA-OBLIVIOUS**).** *Consider the construction in Fig. 10. Let Π be an encryption scheme and F be a function. For any PPT adversary $\mathcal{A}$ in the* IND-CPA-OBLIVIOUS *game (Fig. 9 and Definition 19) we have that:*

$$\mathsf{Adv}^{\text{IND-CPA-OBLIVIOUS}}_{\Pi,\mathsf{ABOM}}(\mathcal{A}) \leq \mathsf{Adv}^{\text{IND-PRF}}_{\mathsf{F}}(1^\lambda) + \mathsf{Adv}^{\text{IND-CPA}}_{\Pi}(1^\lambda) \ .$$

Proof. Let E_w be the event that the adversary outputs $w = w'$ for $w \in \{d, b\}$. By the union bound, we can upper-bound the advantage of the adversary $\mathcal{A}$ in winning the OBLIVIOUS game as

$$\begin{aligned}\mathsf{Adv}^{\text{IND-CPA-OBLIVIOUS}}_{\Pi,\mathsf{ABOM}}(\mathcal{A}) &= \left|\Pr[\text{IND-CPA-OBLIVIOUS}_{\Pi,\mathsf{ABOM}}(1^\lambda, \mathcal{A}) \Rightarrow 1] - 3/4\right| \\ &\leq |\Pr[\mathsf{E}_d] - 1/2| + |\Pr[\mathsf{E}_b] - 1/2| \ ,\end{aligned}$$

since $0 \leq \Pr[\mathsf{E}_d \wedge \mathsf{E}_b] \leq 1/4$. We prove that each addend is negligible by starting with $|\Pr[\mathsf{E}_d] - 1/2|$ and then addressing $|\Pr[\mathsf{E}_b] - 1/2|$.

Assume $|\Pr[\mathsf{E}_d] - 1/2|$ is not negligible. We build an adversary $\mathcal{B}$ that plays the multi-query IND-CPA game against Π (Definition 9). Adversary $\mathcal{B}$ runs $\mathcal{A}$ as a subroutine. Whenever $\mathcal{A}$ calls the ENC oracle, $\mathcal{B}$ forwards m_0 and m_1 to the IND-CPA challenger. Adversary $\mathcal{B}$ simulates all the other oracles honestly. For this, $\mathcal{B}$ samples a random bit $b \xleftarrow{\mathrm{R}} \{0, 1\}$ and runs $\mathsf{GeneralSetup}(1^\lambda, \mathsf{ps}_b)$, then simulates all the oracles. Crucially, all these oracles are independent of the IND-CPA key k, to which clearly $\mathcal{B}$ does not have access to. In the alert phase of the ABOM protocol (Fig. 10), the client never sends the share $\varkappa_0$ to the server, so ss is independent of the key k as it only contains the random share $\varkappa_1$. Moreover, the outputs of $\mathsf{Request}$ are independent of k and the same holds for the output of Clear. The ds output of GetKey is also independent of the key k. Therefore $\mathcal{B}$'s simulation of the oracles is perfect. Finally, $\mathcal{B}$ outputs $\mathcal{A}$'s guess as their own. Hence, we have

$$|\Pr[\mathsf{E}_d] - 1/2| \leq \mathsf{Adv}^{\text{IND-CPA}}_{\Pi}(\mathcal{B}) \ .$$

We now tackle the second term, i.e. $|\Pr[\mathsf{E}_b] - 1/2|$. Let G_1 be the same as the IND-CPA-OBLIVIOUS game of Fig. 9 but with the PRF replaced by a random function $f \xleftarrow{\mathrm{R}} \mathsf{Func}(n, \ell(n))$ (we define $\mathsf{Func}(n, \ell(n))$ in Definition 3). In particular, we change line 3 of Setup as $\mathsf{com} \leftarrow f(\mathsf{ps})$ and line 2 of $\mathsf{Request}$ as $\mathsf{com} \leftarrow f(\mathsf{ps})$. Assume a distinguisher $\mathcal{D}$ that can differentiate the two games with a non-negligible probability ν. We build an adversary $\mathcal{B}'$ that plays the $\mathsf{IND\text{-}PRF}$ game (Definition 3). The adversary $\mathcal{B}'$ simulates all the oracles of the two games, which are the same. Whenever $\mathcal{B}'$ needs the commitment on ps_b (to run $\mathsf{GeneralSetup}$ and to simulate the REQUEST oracle, which always use the same ps_b), $\mathcal{B}'$ uses their own DERIVE oracle (Fig. 1). Finally, $\mathcal{B}'$ outputs the bit that the distinguisher outputs. Therefore we have

$$|\Pr[\mathcal{D}(\mathrm{G}_0) \Rightarrow 1] - \Pr[\mathcal{D}(\mathrm{G}_1) \Rightarrow 1]| \leq \mathsf{Adv}^{\text{IND-PRF}}_{\mathsf{F}}(\mathcal{B}') \ .$$

Finally, note that in the game G_1 the commitment com is a random bit string independent of b and therefore the adversary can only randomly guess the bit b. Therefore we conclude that

$$|\Pr[\mathsf{E}_b] - 1/2| \leq \mathsf{Adv}^{\text{IND-PRF}}_{\mathsf{F}}(\mathcal{B}') \ . \qquad \square$$

6.2 Implementation

We give a proof-of-concept implementation in Appendix F and https://github.com/si-co/abom; the repository contains additional test files. As mentioned before, each message is at most 96 bytes in the alert phase and 112 bytes in the setup phase. All messages thus fit into the 140 bytes that a single SMS can carry (20).

Acknowledgements

This work would not exist without the many contributions from the protesters and activists in Kenya, who trusted us with their experiences. We thank Daniel Collins, Eamonn Postlethwaite, Fernando Virdia, Joël Felderhoff, Kenny Paterson and Lenka Mareková for discussions on an earlier draft.

References

ABJM21. Martin R. Albrecht, Jorge Blasco, Rikke Bjerg Jensen, and Lenka Mareková. Collective information security in large-scale urban protests: the case of hong kong. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 3363–3380. USENIX Association, August 2021.

ADJ24. Martin R. Albrecht, Benjamin Dowling, and Daniel Jones. Device-oriented group messaging: A formal cryptographic analysis of matrix' core. In *2024 IEEE Symposium on Security and Privacy*, pages 2666–1685. IEEE Computer Society Press, May 2024.

ADJ25. Martin R. Albrecht, Benjamin Dowling, and Daniel Jones. Formal analysis of multi-device group messaging in WhatsApp. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 242–271. Springer, Cham, May 2025.

AL10. Yonatan Aumann and Yehuda Lindell. Security against covert adversaries: Efficient protocols for realistic adversaries. *Journal of Cryptology*, 23(2):281–343, April 2010.

App24. Apple. Check In on iPhone. https://support.apple.com/en-gb/guide/iphone/iphc143bb7e9/ios, 2024.

Atk15. Paul Atkinson. *For Ethnography*. Sage, 2015.

BBB+19. Olivier Blazy, Angèle Bossuat, Xavier Bultel, Pierre-Alain Fouque, Cristina Onete, and Elena Pagnin. SAID: Reshaping Signal into an identity-based asynchronous messaging protocol with authenticated ratcheting. In *2019 IEEE European Symposium on Security and Privacy*, pages 294–309. IEEE Computer Society Press, June 2019.

BBC25. BBC World Service. Blood Parliament, 2025. Accessed: 2025-06-29.

BBL+23. Olivier Blazy, Ioana Boureanu, Pascal Lafourcade, Cristina Onete, and Léo Robert. How fast do you heal? A taxonomy for post-compromise security in secure-channel establishment. In Calandrino and Troncoso [CT23], pages 5917–5934.

BCC+23. Khashayar Barooti, Daniel Collins, Simone Colombo, Loïs Huguenin-Dumittan, and Serge Vaudenay. On active attack detection in messaging with immediate decryption. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 362–395. Springer, Cham, August 2023.

BDG+24. Matilda Backendal, Hannah Davis, Felix Günther, Miro Haller, and Kenneth G. Paterson. A formal treatment of end-to-end encrypted cloud storage. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part II*, volume 14921 of *LNCS*, pages 40–74. Springer, Cham, August 2024.

BDJR97. Mihir Bellare, Anand Desai, Eric Jokipii, and Phillip Rogaway. A concrete security treatment of symmetric encryption. In *38th FOCS*, pages 394–403. IEEE Computer Society Press, October 1997.

BFH+23. Jonathan Bootle, Sebastian H. Faller, Julia Hesse, Kristina Hostáková, and Johannes Ottenhues. Generalized fuzzy password-authenticated key exchange from error correcting codes. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VIII*, volume 14445 of *LNCS*, pages 110–142. Springer, Singapore, December 2023.

BJSL11. Ali Bagherzandi, Stanislaw Jarecki, Nitesh Saxena, and Yanbin Lu. Password-protected secret sharing. In Yan Chen, George Danezis, and Vitaly Shmatikov, editors, *ACM CCS 2011*, pages 433–444. ACM Press, October 2011.

Bla12. Jean-François Blanchette. *Burdens of Proof: Cryptographic Culture and Evidence Law in the Age of Electronic Documents*. The MIT Press, Cambridge, MA, 2012.

BN08. Mihir Bellare and Chanathip Namprempre. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. *Journal of Cryptology*, 21(4):469–491, October 2008.

BR00. Mihir Bellare and Phillip Rogaway. Encode-then-encipher encryption: How to exploit nonces or redundancy in plaintexts for efficient cryptography. In Tatsuaki Okamoto, editor, *ASIACRYPT 2000*, volume 1976 of *LNCS*, pages 317–330. Springer, Berlin, Heidelberg, December 2000.

Bre00. John D. Brewer. Ethnography, 2000.

Bri24. British High Commission Nairobi. Joint statement by Ambassadors and High Commissioners in Kenya on public protests, 2024. Accessed: 2025-05-29.

BRT12. Mihir Bellare, Thomas Ristenpart, and Stefano Tessaro. Multi-instance security and its application to password-based cryptography. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 312–329. Springer, Berlin, Heidelberg, August 2012.

BS23. Mihir Bellare and Laura Shea. Flexible password-based encryption: Securing cloud storage and provably resisting partitioning-oracle attacks. In Mike Rosulek, editor, *CT-RSA 2023*, volume 13871 of *LNCS*, pages 594–621. Springer, Cham, April 2023.

CAA+16. Rahul Chatterjee, Anish Athayle, Devdatta Akhawe, Ari Juels, and Thomas Ristenpart. pASSWORD tYPOS and how to correct them securely. In *2016 IEEE Symposium on Security and Privacy*, pages 799–818. IEEE Computer Society Press, May 2016.

CCG16. Katriel Cohn-Gordon, Cas J. F. Cremers, and Luke Garratt. On post-compromise security. In Michael Hicks and Boris Köpf, editors, *CSF 2016 Computer Security Foundations Symposium*, pages 164–178. IEEE Computer Society Press, 2016.

CFGN96. Ran Canetti, Uriel Feige, Oded Goldreich, and Moni Naor. Adaptively secure multi-party computation. In *28th ACM STOC*, pages 639–648. ACM Press, May 1996.

CFKN20. Cas Cremers, Jaiden Fairoze, Benjamin Kiesl, and Aurora Naska. Clone detection in secure messaging: Improving post-compromise security in practice. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1481–1495. ACM Press, November 2020.

Cha06. Kathy Charmaz. *Constructing grounded theory: A practical guide through qualitative analysis*. SAGE, 2006.

CJN23. Cas Cremers, Charlie Jacomme, and Aurora Naska. Formal analysis of session-handling in secure messaging: Lifting security from sessions to conversations. In Calandrino and Troncoso [CT23], pages 1235–1252.

CMN25. Cas Cremers, Niklas Medinger, and Aurora Naska. Impossibility results for post-compromise security in real-world communication systems. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 4391–4405. IEEE Computer Society Press, May 2025.

CT23. Joseph A. Calandrino and Carmela Troncoso, editors. *USENIX Security 2023*. USENIX Association, August 2023.

CWP+17. Rahul Chatterjee, Joanne Woodage, Yuval Pnueli, Anusha Chowdhury, and Thomas Ristenpart. The TypTop system: Personalized typo-tolerant password checking. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 329–346. ACM Press, October / November 2017.

DGP22. Benjamin Dowling, Felix Günther, and Alexandre Poirrier. Continuous authentication in secure messaging. In Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng, editors, *ESORICS 2022, Part II*, volume 13555 of *LNCS*, pages 361–381. Springer, Cham, September 2022.

DH20. Benjamin Dowling and Britta Hale. There can be no compromise: The necessity of ratcheted authentication in secure messaging. Cryptology ePrint Archive, Report 2020/541, 2020.

DH21. Benjamin Dowling and Britta Hale. Secure messaging authentication against active man-in-the-middle attacks. In *2021 IEEE European Symposium on Security and Privacy*, pages 54–70. IEEE Computer Society Press, September 2021.

DHP+18. Pierre-Alain Dupont, Julia Hesse, David Pointcheval, Leonid Reyzin, and Sophia Yakoubov. Fuzzy password-authenticated key exchange. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 393–424. Springer, Cham, April / May 2018.

EFS11. Robert M Emerson, Rachel I Fretz, and Linda L Shaw. *Writing ethnographic fieldnotes*. University of Chicago press, 2011.

EHM17. Ksenia Ermoshina, Harry Halpin, and Francesca Musiani. Can Johnny build a protocol? Co-ordinating developer and user intentions for privacy-enhanced secure messaging protocols. In *European Workshop on Usable Security*, 2017.

FHH+24. Sebastian H. Faller, Tobias Handirk, Julia Hesse, Máté Horváth, and Anja Lehmann. Password-protected key retrieval with(out) HSM protection. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2445–2459. ACM Press, October 2024.

Gee98. Clifford Geertz. Deep Hanging Out. *The New York Review of Books*, 45(16):69–72, 1998.

Gee08. Clifford Geertz. Thick description: Toward an interpretive theory of culture. In *The Cultural Geography Reader*, pages 41–51. Routledge, 2008.

GGSW13. Sanjam Garg, Craig Gentry, Amit Sahai, and Brent Waters. Witness encryption and its applications. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 467–476. ACM Press, June 2013.

GS99. Barney Glaser and Anselm Strauss. *Discovery of grounded theory: Strategies for qualitative research*. Routledge, 1999.

HA19. Martyn Hammersley and Paul Atkinson. *Ethnography: Principles in Practice*. Routledge Ltd, London, 4th ed. edition, 2019.

Her00. Steve Herbert. For ethnography. *Progress in Human Geography*, 24(4):550–568, 2000.

Hum24. Human Rights Watch. Kenya: Security Forces Abducted, Killed Protesters, 2024. Accessed: 2025-06-30.

KD23. Leah R Kimber and Emilie Dairon. Ethnographic Interviews. In *International Organizations and Research Methods*, pages 48–55. University of Michigan Press, 2023.

Ken24a. Kenya National Commission on Human Rights. Statement on Mukuru Murders and Updates on the Anti-Finance Bill Protests, 2024. Accessed: 2025-07-01.

Ken24b. Kenya National Commission on Human Rights. Statement on the Recent Surge of Abductions/Enforced Disappearances in Kenya, 2024. Accessed: 2025-07-01.

Ken25. Kenya National Commission on Human Rights. KNCHR Update on the 25th June 2025 Demonstrations, 2025. Accessed: 2025-07-01.

LS20. Jasper Lee and Shamay Sameul. Lecture 11: Distinguishing (Discrete) Distributions. https://cs.brown.edu/courses/csci1951-w/lec/lec%2011%20notes.pdf, 2020.

Lum23. Lumivero. Nvivo 14, 2023. Accessed: 2025-06-29.

Lun19. Joshua Lund. Technology preview for secure value recovery, 2019. Accessed: 2025-05-28.

MBNP24. Larry Madowo, Stephanie Busari, Catherine Nicholls, and Sugam Pokharel. Kenya's president calls protests 'treasonous' after police fire live rounds at demonstrators, 2024. Accessed: 2025-06-29.

Mbu25. Judy Mbugua. Why kenya's gen z has taken to the streets, 2025. Accessed: 2025-09-12.

MU14. Patrick McCurdy and Julie Uldam. Connecting participant observation positions: Toward a reflexive framework for studying social movements. *Field Methods*, 26(1):40–55, 2014.

Nan24. Nanjala Nyabola. The world is scrambling to understand Kenya's historic protests – this is what too many are missing, 2024. Accessed: 2025-09-12.

Off24. Office of the President of Kenya. President Ruto declines to sign Finance Bill, calls for its withdrawal, 2024. Accessed: 2025-06-29.

Ose24. Gordon Osen. Wave of abductions fuel protest as police condemned, 2024. Accessed: 2025-07-14.

RG22. Rachel Rinaldo and Jeffrey Guhin. How and why interviews work: Ethnographic interviews and meso-level public culture. *Sociological Methods & Research*, 51(1):34–67, 2022.

S$^+$25. William Stein et al. *Sage Mathematics Software Version 10.6*. The Sage Development Team, 2025. http://www.sagemath.org.

SL21. Sena Sahin and Frank Li. Don't forget the stuffing! Revisiting the security impact of typo-tolerant password authentication. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 252–270. ACM Press, November 2021.

Ver25. VeraCrypt. *Hidden Volume*, May 2025. https://veracrypt.io/en/Hidden%20Volume.html.

A Usability of Passwords

As discussed in Section 3.5, our work does not address usability concerns of an instantiation of our protocol achieving our security notion. While, on the one hand, we consider such considerations premature at this stage – the introduction of a novel security notion – we stress and discuss here that there are significant usability questions that would need to be addressed.

Namely, during 'normal' operation, our protocol relies critically on the person depending on it not to mistype their password: *any* $\mathsf{ps} \neq \mathsf{ps}_1$ triggers an alert.

On the one hand, this can be alleviated at the design stage of a protocol that would respond to $\mathsf{dtct} = \mathsf{true}$. In particular, while our protocol is inspired by widespread public calls triggered by an alarm, it is possible another protocol is invoked first, e.g. checking in with trusted contacts if they can reach the person who triggered the alarm. This was also suggested by the interlocutors with whom we discussed our model. As also mentioned in Section 3.5 we consider this a pressing topic for future work.

On the other hand, password typos and how to handle them is an active area of research, both in the broader information security literature and, more narrowly, in cryptography, e.g. [CAA+16, CWP+17, SL21, BFH+23]. Similar to prior work [CWP+17] it might be possible to have a learning phase where common typos are learned server-side (by storing more commitments), but we did not analyse the security impact of such a system.

We stress that in our model, based on our data, the adversary is remote until line 8. However, while this is true for this, life-threatening, adversary – William Ruto's security forces – we stress that this does not necessarily imply that these security forces are always the primary adversaries that organisers, frontliners and other activists were concerned with. In Appendix D we present some data from security trainings the fieldworker ran with activist groups in Kenya. The data shows that social and intimate partners were commonly identified as potential threats to confidentiality.

Thus, we observe, on the one hand, the need for 'traditional' PIN-based access control mechanisms to maintain confidentiality against social and intimate partners. We observe, on the other hand, the need for an AtC-AB mechanism which surrenders access control for raising an alert. This tension can be resolved by a two-tiered approach. Traditional PIN-based accept/deny decisions at, e.g. the OS level, say, to unlock a device. Then, an AtC-AB-based solution for accessing certain secure messaging or data storage applications.

More broadly, we note that a fully working solution would not only have to respond to the threat modelled and discussed here but would have to take the broader context and the subjectivity of the people it is meant to protect into account [**?**].

B A Forgery Style Model of Alert Blindness

Our central security notion in Definition 17 requires us to send a message to a server to raise an alarm. There, we accomplish this by leveraging the adversary's interest in allowing a message to pass to realise its intelligence goal. Here, in Definition 21, we realise AtC-AB by coding the absence of a message as this alarm. This captures strategies in our data such as social check-in protocols or scheduled X messages which were deleted once someone returned from a higher risk setting.[28] Overall, the game is quite similar to Definition 17 except that there is no bit d.

Definition 21 ($\text{AtC-AB}^{\emptyset}_{\mathcal{S},\mathsf{ABOM}}$). *Let $\lambda \in \mathbb{N}$ be a security parameter and let $\text{AtC-AB}^{\emptyset}_{\mathcal{S},\mathsf{ABOM}}$ be the game defined in Fig. 11, with respect to a personal secret distribution $\mathcal{S}$ and an* ABOM *protocol. Let $\mathcal{T}$ be the distribution obtained by sampling from $x' \xleftarrow{R} \mathcal{S}$ and then sampling and outputting $x \xleftarrow{R} \mathcal{S} \setminus \{x'\}$. Let $\mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}}$ as per Proposition 2:*

$$\mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}} = \frac{1 + \mathsf{SD}(\mathcal{S},\mathcal{T}) + 2^{-H_\infty(\mathcal{S}\setminus\{x_b\})}}{2} \ .$$

We define the advantage of an adversary $\mathcal{A}$ in winning this game as

$$\mathsf{Adv}^{\text{AtC-AB}^{\emptyset}}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A}) = \Pr[\text{AtC-AB}^{\emptyset}_{\mathcal{S},\mathsf{ABOM}}(1^\lambda,\mathcal{A}) \Rightarrow 1] - \mathsf{Adv}^{\mathrm{SG}}_{\mathcal{S}} \ .$$

We say that ABOM *is* $\text{AtC-AB}^{\emptyset}$*-secure if $\mathsf{Adv}^{\text{AtC-AB}^{\emptyset}}_{\mathcal{S},\mathsf{ABOM}}(\mathcal{A})$ is negligible in the security parameter λ.*

[28] More broadly, strategies along the lines of "if I miss this meeting, assume something has happened" appear to be common and e.g. Apple has implemented a digital version of this strategy [App24].

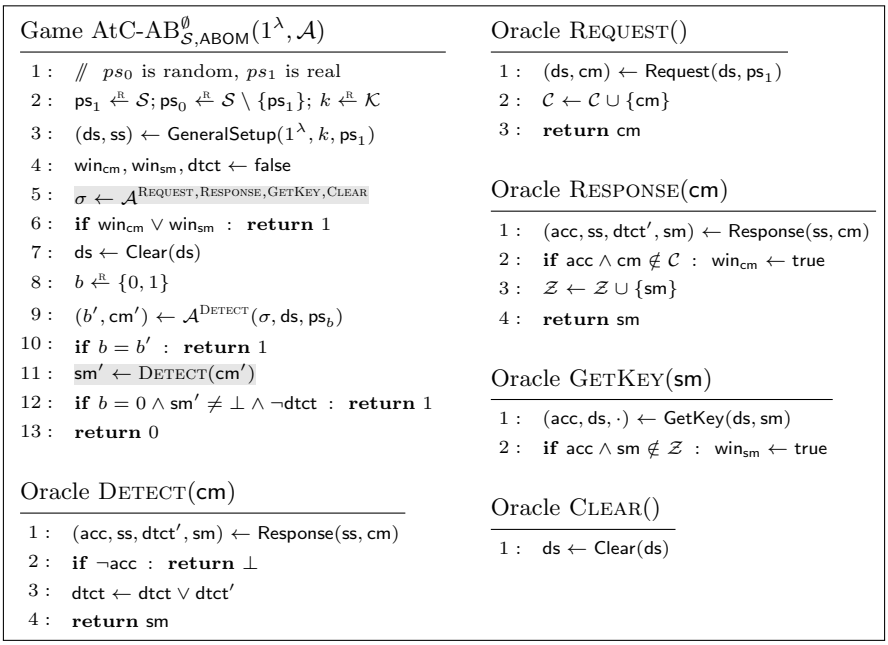


Figure 11: Zero-Message At-Compromise Alert Blindness (AtC-AB$^{\emptyset}_{\mathcal{S},\mathsf{ABOM}}$).

Remark 4. The advantage statement follows from the same observation as in Remark 2 except that there is no KIND bit in this game.

Claim 2 *Consider* AtC-AB$^{\emptyset}$ *as defined in Definition 21. Assume the adversary does not consistently control the network. Then the* AtC-AB$^{\emptyset}$ *model correctly responds to security needs of some organisers and frontliners in Kenya in 2024.*

Justification. The winning conditions, the call to Clear and line $\mathsf{dtct} \leftarrow \mathsf{dtct} \vee \mathsf{dtct}'$ are justified exactly as in Claim 1. Here, we need to justify the forced Detect call in line 11. This call models scheduled messages, e.g. on X, (16) and check-in/check-out protocols within groups (22). Thus, the absence of a message is the distress signal, forcing the adversary to send/forge a cm' message with the intention of not raising an alarm. That is, in our unforgability style game, failure to produce an accepting message realises AtC-AB.

However, this then, at the very least, complicates our use of oracles {Request, Response, GetKey, Clear} in line 5 which allow the adversary to schedule cm and sm messages. Given this power, the adversary can trivially trigger a false positive by simply refusing to distribute some cm. Our model captures the observation from our data that while the adversary in Kenya indeed exploited its ability to suppress messages, "they told me 'we called you, like, 10 times', but I had not received any calls" (P14), this control was incomplete (21) with e.g. WhatsApp unaffected. We reflect this by allowing the adversary to control the network (which in turn allows us to reason about the cryptographic unforgability properties of cm and sm). However, we *do not model* the adversary winning by triggering false positives.

Remark 5. Our *assumption* that the adversary does not consistently control the network is somewhat aligned with the assumption in PCS that eventually the adversary becomes passive, i.e. observes but does not interfere in the network. However, in PCS the adversary loses if it briefly does not control the network, here the adversary triggers a false alarm if it controls the network for a period of time.

C Fieldwork Components

Table 2: Overview of protests that took place during the fieldwork and which the fieldworker attended.

Date	**Location**	**Name**	**High-level notes**
18 June	Nairobi	*Occupy Parliament*	Large scale, mass arrests (approx. 400).
20 June	Nairobi Major towns	*Occupy Parliament Reloaded*	Large-scale protests across Kenya.
25 June	Nairobi Major towns	*Total Shutdown*	Storming of Parliament.
27 June	Nairobi Major towns	*Occupy Statehouse*	Smaller-scale protests, violent street battles.
2 July	Nairobi Major towns	*Occupy CBD*	Smaller-scale protests, violent street battles.
4 July	Nairobi Major towns	*Occupy CBD*	Smaller-scale protests, violent street battles.
7 July	Uhuru Park, Nairobi	*Saba Saba Memorial concert*	All-day memorial concert organised by known activists.
16 July	Nairobi Major towns	*Occupy Everywhere*	Smaller-scale protests, violent street battles.
23 July	Nairobi Major towns	*Fixing the Nation*	Boda boda riders hired to disrupt protests, two riders killed by protesters
25 July	Nairobi	*Mashujaa Wetu (Our Heroes) March*	Protest to mark one month after 25 June, organised by leading activists.
8 August	Nairobi	*Nane Nane*	Brutal confrontations between protesters and police.

CBD refers to Central Business District.

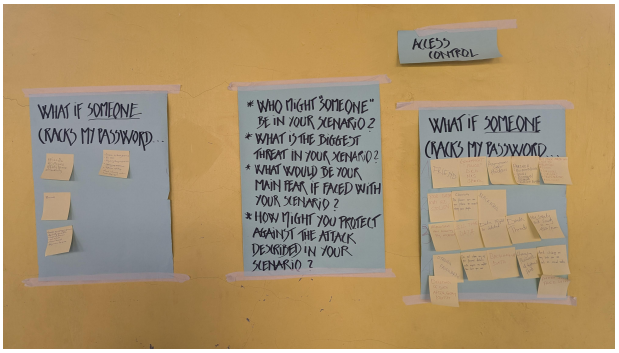
Table 3: High-level information on interview and group discussion participants.

PID	GID	Location	Role	Type	Dur.	PID	GID	Location	Role	Type	Dur.
P1	–	Kisumu	Organiser	Interview	93	P40	–	Nairobi	Protester	Interview	54
P2	–	Kisumu	Protester	Interview	78	P41	–	Nairobi	Frontliner	Interview	80
P3	–	Mombasa	Protester	Interview	53	P42	–	Nairobi	Organiser	Interview	104
P4	–	Kisumu	Organiser	Interview	82	P43	K	Nairobi	Organiser	Group	88
P5	–	Mombasa	Protester	Interview	63	P44	K	Nairobi	Organiser	Group	88
P6	C	Mombasa	Frontliner	Group	97	P45	K	Nairobi	Frontliner	Group	88
P7	W	Kisumu	Frontliner	Group	152	P46	K	Nairobi	Frontliner	Group	88
P8	C	Mombasa	Frontliner	Group	97	P47	K	Nairobi	Frontliner	Group	88
P9	K	Nairobi	Frontliner	Group	88	P48	K	Nairobi	Frontliner	Group	88
P10	W	Kisumu	Protester	Group	152	P49	–	Nairobi	Abductee	Interview	64
P11	C	Mombasa	Frontliner	Group	97	P50	–	Nairobi	Frontliner	Interview	104
P12	–	Mombasa	Protester	Interview	55	P51	–	Nairobi	Abductee	Interview	131
P13	W	Kisumu	Protester	Group	152	P52	–	Nairobi	Organiser	Interview	43
P14	Q	Mombasa	Organiser	Group	157	P53	–	Nairobi	Frontliner	Interview	54
P15	W	Kisumu	Organiser	Group	152	P54	–	Nairobi	Frontliner	Interview	61
P16	Q	Mombasa	Frontliner	Group	157	P55	–	Nairobi	Frontliner	Interview	55
P17	W	Kisumu	Frontliner	Group	152	P56	–	Nairobi	Frontliner	Interview	71
P18	C	Mombasa	Frontliner	Group	97	P57	–	Nairobi	Frontliner	Interview	63
P19	W	Kisumu	Protester	Group	152	P58	–	Nairobi	Organiser	Interview	45
P20	–	Mombasa	Organiser	Interview	62	P59	–	Nairobi	Protester	Interview	66
P21	–	Kisumu	Organiser	Interview	56	P60	–	Kisumu	Abductee	Interview	53
P22	–	Nairobi	Abductee	Interview	78	P61	–	Nairobi	Abductee	Interview	128
P23	Q	Mombasa	Protester	Group	157	P62	–	Nairobi	Oganiser	Interview	105
P24	C	Mombasa	Frontliner	Group	97	P63	F	Nairobi	Organiser	Group	92
P25	–	Kisumu	Protester	Interview	67	P64	F	Nairobi	Organiser	Group	92
P26	–	Mombasa	Protester	Interview	78	P65	F	Nairobi	Organiser	Group	92
P27	–	Nairobi	Protester	Interview	61	P66	F	Nairobi	Protester	Group	92
P28	–	Nairobi	Protester	Interview	43	P67	F	Nairobi	Protester	Group	92
P29	–	Nairobi	Protester	Interview	61	P68	–	Nairobi	Organiser	Interview	62
P30	–	Nairobi	Protester	Interview	67	P69	–	Nairobi	Protester	Interview	58
P31	–	Nairobi	Abductee	Interview	69	P70	–	Nairobi	Frontliner	Interview	77
P32	–	Nairobi	Organiser	Interview	59	P71	–	Nairobi	Frontliner	Interview	116
P33	–	Mombasa	Organiser	Interview	59	P72	–	Nairobi	Abductee	Interview	123
P34	–	Nairobi	Organiser	Interview	50	P73	–	Nairobi	Protester	Interview	62
P35	–	Nairobi	Frontliner	Interview	56	P74	–	Nairobi	Protester	Interview	71
P36	–	Nairobi	Frontliner	Interview	60	P75	–	Nairobi	Protester	Interview	64
P37	–	Nairobi	Abductee	Interview	71	P76	–	Nairobi	Organiser	Interview	81
P38	–	Nairobi	Protester	Interview	59	P77	–	Nairobi	Frontliner	Interview	53
P39	–	Nairobi	Organiser	Interview	94						

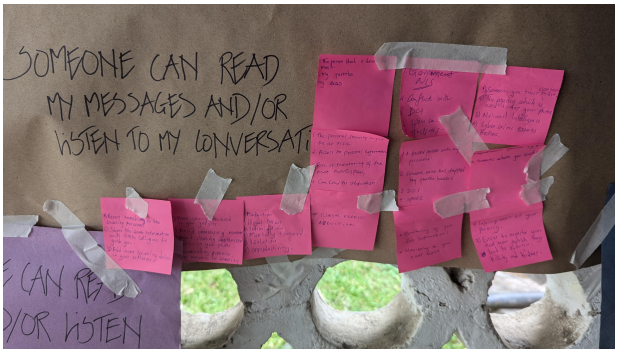
Notes: **PID**: Participant ID. **Location**: Nairobi, Mombasa or Kisumu. **Role**: *Organiser* is a known activist and/or leading role in groups, *Frontliner* is a not known activist, but a mobiliser and experience of confrontation with and targeted by the Kenyan security forces and some of them had been taken to safe houses, *Protester* is someone who had participated in at least one of the protests, *Abductee* is someone who had been forcefully disappeared. **Type**: Individual interview or a group discussion. **Dur.**: Duration is given in minutes. **GID**: ID of the group discussion where this is applicable.

D Social and Intimate Threats

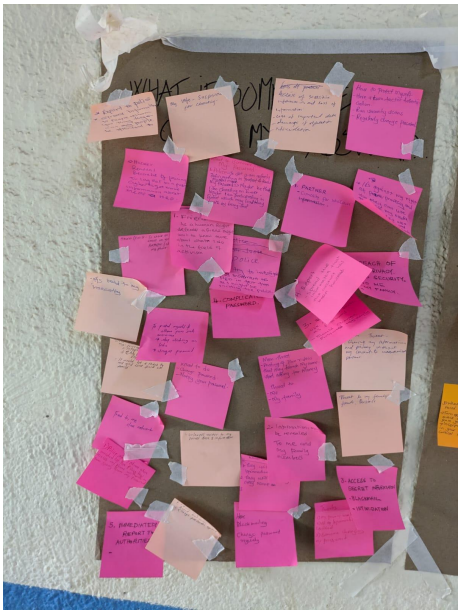
During the fieldwork, the fieldworker engaged groups of activists through security trainings, which were grounded in the immediate context of the Gen-Z protests and the threats facing protesters at the time. This involved group discussions focusing on *what if ...* scenarios: *What if ...* (1) someone cracks my password, (2) someone can read my messages and/or listen to my conversations, (3) someone can see which groups I am in and who the other group members are. Asked to identify who the most likely *someone*, i.e. the most likely adversary, was in their scenario, participant contributions show how social and intimate relations such as spouses, partners, parents and friends were considered a likely – and often the most prominent – threat in scenarios (1) and (2) in particular as we exemplify below. When asked to elaborate, participants gave several examples of how they would hide their devices from people they lived with because of their activism; this included parents and partners who did not agree with their activism or participation in protests. Many had also experienced how close relations such as family members and trusted friends had revealed information about their whereabouts and activism to local security forces – sometimes because they had been threatened to do so – and therefore protecting information from intimate relations was considered important. It is also important to note that many lived in informal settlements and in makeshift housing, with neighbours living in close proximity to each other, often with only a curtain or metal sheet separating living spaces. The struggle for survival sometimes also led to what participants referred to as "betrayal by people you trust" (FN8) as well as "daily violence" (FN5), highlighting how it was sometimes easy for local security forces to obtain information about them and their activities as they were known in their communities. See Fig. 12.



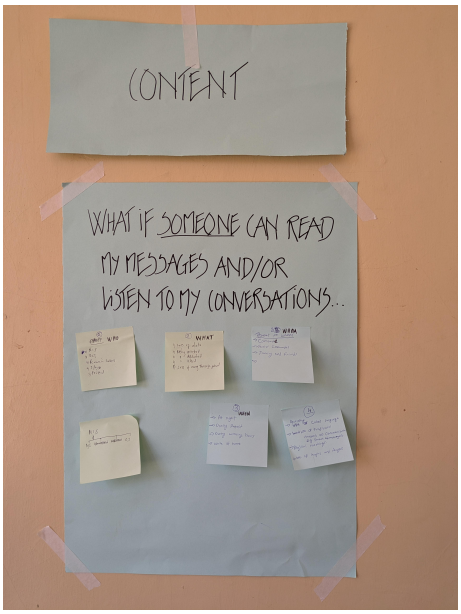
(a) **Nairobi, August 2024.** Adversaries listed: friend, government, police, DCI, NIS, anonymous cyber hackers, spouse/partner, fiancé, any other person who is not you, relatives, colleagues, victims, survivors.



(b) **Mombasa, July 2024.** Adversaries listed: the person that I trust the most, my parents, my boss, government, NIS, DCI, spouses, close friends, the agency which is used to tap your phone, hackers, national intelligence.



(c) **Kisumu, July 2024.** Adversaries listed: partner, my wife, police, friends, hacker.



(d) **Nairobi, January 2025.** Adversaries listed: NIS, DCI, hackers, spouse, friend.

(e) **Nairobi, September 2024.** Adversaries listed: any other person, government infiltration, cyber hackers, spouses (partner/friend).

Figure 12: Examples of participant contributions from security trainings run by the fieldworker during the fieldwork.

E Password Guessing Experiments

```
"""
Run

```
python -m sage.doctest pw.py
```

- P :: A list of probabilities describing a distribution
- S :: The password distribution
- S_{¬x} :: Output y given x :: y ← S / {x}
- T :: Output y s.t. x ← S; y ← S_{-x}

"""

from sage.all import GeneralDiscreteDistribution, randint, RR

class Dist(GeneralDiscreteDistribution):
    def __init__(self, P, seed=None):
        self.P = P
        self.seed = seed
        GeneralDiscreteDistribution.__init__(self, P=P, seed=seed)

    def negx(self, x, seed=None):
        """
        Return S_{¬x}

        :param x: An element in S

        EXAMPLE ::

            sage: P = [0.1, 0.2, 0.3, 0.4]
            sage: S = Dist(P, seed=1312)
            sage: trials = 2^16

            sage: S0 = S.negx(0, seed=1337)
            sage: L0 = [S0.get_random_element() for _ in range(trials)]
            sage: R0 = [S.get_random_element_negx(0) for _ in range(trials)]
            sage: [round(trials*p) for p in S0.P]
            [0, 14564, 21845, 29127]
            sage: [L0.count(i) for i in range(4)]
            [0, 14518, 21700, 29318]
            sage: [R0.count(i) for i in range(4)]
            [0, 14509, 22000, 29027]

            sage: S2 = S.negx(2, seed=1337)
            sage: L2 = [S2.get_random_element() for _ in range(trials)]
            sage: R2 = [S.get_random_element_negx(2) for _ in range(trials)]
            sage: [round(trials*p) for p in S2.P]
            [9362, 18725, 0, 37449]
            sage: [L2.count(i) for i in range(4)]
            [9365, 18572, 0, 37599]
            sage: [R2.count(i) for i in range(4)]
            [9260, 18723, 0, 37553]

        """
        T = [0.0 for _ in range(len(self.P))]
        P_ = [p for p in self.P]
        P_[x] = 0
        P_ = Dist.normalize(P_)
        for i in range(len(self.P)):
            if i == x:
                continue
            T[i] += P_[i]
        T = Dist.normalize(T)
        return Dist(tuple(T), seed=seed)

    def get_random_element_negx(self, x):
        """
        Sample from S_{¬x}.

        :param x: An element in S
        """

        while True:
            y = self.get_random_element()
            if y != x:
                return y
```

```
    def T(self, seed=None):
        """
        Return T

        EXAMPLE ::

            sage: P = [0.1, 0.2, 0.3, 0.4]
            sage: S = Dist(P, seed=1312)
            sage: trials = 2^16

            sage: T = S.T(seed=1337)
            sage: L = [T.get_random_element() for _ in range(trials)]
            sage: R = [S.get_random_element_t() for _ in range(trials)]
            sage: [round(trials*p) for p in T.P]
            [8816, 15812, 20207, 20701]
            sage: [L.count(i) for i in range(4)]
            [8845, 15708, 20038, 20945]
            sage: [R.count(i) for i in range(4)]
            [8767, 15677, 20476, 20616]

        """
        T = [0.0 for _ in range(len(self.P))]
        for i in range(len(self.P)):
            P_ = [p for p in self.P]
            P_[i] = 0
            P_ = Dist.normalize(P_)
            for j in range(len(self.P)):
                T[j] += self.P[i] * P_[j]
        T = Dist.normalize(T)
        return Dist(T, seed)

    def get_random_element_t(self):
        """
        Sample from T.
        """
        x = self.get_random_element()
        return self.get_random_element_negx(x)

    @classmethod
    def normalize(cls, P):
        """
        Enforce that the probabilities sum to 1.

        :param P: A list of probabilities describing a distribution

        EXAMPLE ::

            sage: Dist.normalize([0.1, 0.4])
            (0.200000000000000, 0.800000000000000)

        """

        s = sum(P)
        return tuple([p / s for p in P])

    def most_likely_element(self):
        """
        Return an element with highest likelihood.

        """

        return self.P.index(max(self.P))


def advf(S, x):
    """
    The optimal adversary against our game.

    :param S: A distribution.
    :param x: A candidare element.

    """
    T = S.T()
    V = S.negx(x)

    b_ = bool(S.P[x] > T.P[x])
    x_ = V.most_likely_element()

    return b_, x_
```

```
def advf_bad_S(S, x):
    """
    A bad adversary that guesses form S instead of S_{¬x}

    :param S: A distribution.
    :param x: A candidare element.

    """
    T = S.T()

    b_ = bool(S.P[x] >= T.P[x])
    x_ = S.most_likely_element()

    return b_, x_


def advf_bad_real(S, x):
    """
    The 'abductor' adversary which will use the ‘x‘ it was given if it believes it to be real.

    :param S: A distribution.
    :param x: A candidare element.

    """
    T = S.T()
    V = S.negx(x)

    b_ = bool(S.P[x] >= T.P[x])
    if b_ == 1:
        x_ = x
    else:
        x_ = V.most_likely_element()

    return b_, x_


def advf_trivial(_, x):
    """
    This adversary motivates the ‘b == 0‘ check in the conjunction.

    :param S: A distribution.
    :param x: A candidare element.

    """

    # real game: b=0 ⇒ win, b=1 ⇒ lose
    # No b==0: b=0 ⇒ win, b=1 ⇒ win

    b_ = 0
    x_ = x
    return b_, x_


def game(P, adv, seed=None):
    S = Dist(P, seed=seed)
    x = [0, 0]
    x[1] = S.get_random_element()
    x[0] = S.get_random_element_negx(x[1])
    b = randint(0, 1)
    b_, x_ = adv(Dist(S.P), x[b])
    return b_ == b or ((b == 0) and (x_ == x[1]))

def game_b(P, adv, seed=None):
    S = Dist(P, seed=seed)
    x = [0, 0]
    x[1] = S.get_random_element()
    x[0] = S.get_random_element_negx(x[1])
    b = randint(0, 1)
    b_, _ = adv(Dist(S.P), x[b])
    return (b_ == b)

def game_g(P, adv, seed=None):
    S = Dist(P, seed=seed)
    x = [0, 0]
    x[1] = S.get_random_element()
    x[0] = S.get_random_element_negx(x[1])
    b = randint(0, 1)
    _, x_ = adv(Dist(S.P), x[b])
    return (b == 0) and (x_ == x[1])
```

```
def game_b_and_g(P, adv, seed=None):
    S = Dist(P, seed=seed)
    x = [0, 0]
    x[1] = S.get_random_element()
    x[0] = S.get_random_element_negx(x[1])
    b = randint(0, 1)
    b_, x_ = adv(Dist(S.P), x[b])
    # print(b_, b == 0, x_ == x[1])
    return (b_ == b) and (b == 0) and (x_ == x[1])

def test_adv(P, adv, trials=16, game=game, seed=None):
    """
    Test `adv` in winning for `P` over `trial` trials.

    :param P: A list of probabilities.
    :param adv: An adversary against the guessing game.
    :param trials: Number of trials.
    :param seed: Explicit seed (optional).
    :returns: Rate of success.

    EXAMPLES:

        sage: set_random_seed(1312)
        sage: P = [0.1, 0.2, 0.3, 0.4]
        sage: test_adv(P, advf, trials=2^16, seed=1337)
        0.627700805664062
        sage: test_adv(P, advf_bad_real, trials=2^16, seed=1337)
        0.540374755859375
        sage: test_adv(P, advf_bad_S, trials=2^16, seed=1337)
        0.540222167968750
        sage: test_adv(P, advf_trivial, trials=2^16, seed=1337)
        0.500167846679688

        sage: set_random_seed(1312)
        sage: P = [0.25, 0.25, 0.25, 0.25]
        sage: test_adv(P, advf, trials=2^16, seed=1312)
        0.499435424804688
        sage: test_adv(P, advf_bad_real, trials=2^16, seed=1312)
        0.496841430664062
        sage: test_adv(P, advf_bad_S, trials=2^16, seed=1312)
        0.624633789062500
        sage: test_adv(P, advf_trivial, trials=2^16, seed=1312)
        0.500167846679688

        sage: set_random_seed(1312)
        sage: P = [0.99, 0.01]
        sage: test_adv(P, advf, trials=2^16, seed=1234)
        0.995132446289062
        sage: test_adv(P, advf_bad_real, trials=2^16, seed=1234)
        0.990127563476562
        sage: test_adv(P, advf_bad_S, trials=2^16, seed=1234)
        0.990127563476562
        sage: test_adv(P, advf_trivial, trials=2^16, seed=1312)
        0.500167846679688

    """

    if seed is None:
        seed = randint(0, 2**31)

    return sum([int(game(P, adv, seed=seed + i)) for i in range(trials)]) / RR(trials)
```

F Construction Code

F.1 Client Code

```
package scheme

// DeviceState holds the client's local state. All fields are unexported;
// callers interact via the exported functions in this package.
//
//   - k0: client share used to reconstruct k (k = k0 xor k1)
//   - k_com: PRF key for commitments com = PRF(k_com, ps)
//   - rs_cm, rs_sm: ratchet states for client messages (rs_cm) and server
//     messages (rs_sm)
//   - cnt_cm, cnt_sm: counters for client messages and server messages
//
// Field names use underscores for parity with the spec.
type DeviceState struct {
        k0    []byte
        k_com []byte

        rs_cm, rs_sm   []byte
        cnt_cm, cnt_sm uint32
}

// ClientSetup (Figure 4) initializes a fresh DeviceState, which it returns
// with the initial client message.
//
// Arguments:
//   - kappa: initialization secret shared with the server of 32 bytes.
//   - k: the 16-byte session key.
//   - ps: the client's real personal secret.
//
// The returned client message encrypts (com || k1), where com=PRF(k_com, ps) and
// k1 = k xor k0.
func ClientSetup(kappa, k, ps []byte) (*DeviceState, Message) {
        // ratchet states (line 1)
        rs_cm, rs_sm := KDF(kappa)

        // init counters (line 2)
        cnt_cm, cnt_sm := uint32(0), uint32(0)

        // commitment (line 3)
        k_com := RandomKey256()
        com := PRF(k_com, ps)

        // key and secret sharing (line 4)
        k0 := RandomKey128()
        k1 := xor(k, k0)

        // update counter (line 5)
        cnt_cm = cnt_cm + 1

        // kdf on rs_cm (line 6)
        k_cm, rs_cm := KDF(rs_cm)

        // EtM for (com, k1) (line 7)
        ct, at := EtM(k_cm, append(com, k1...), [2]uint32{cnt_cm, cnt_sm})

        // set device state (line 9)
        ds := &DeviceState{
                k_com:  k_com,
                rs_cm:  rs_cm,
                rs_sm:  rs_sm,
                cnt_cm: cnt_cm,
                cnt_sm: cnt_sm,
                k0:     k0,
        }

        // return (line 10)
        return ds, Message{
                Ciphertext: ct,
                AuthTag:    at,
                AD:         [2]uint32{cnt_cm, cnt_sm},
        }
}

// ClientDone (Figure 4) verifies the server message sm and, on success,
// commits the pending ratchet state and counter back into ds. It returns
// acc=false and leaves ds unchanged on failure.
```

```
func ClientDone(ds *DeviceState, sm Message) (bool, *DeviceState) {
        // extract fields from ds (line 1)
        rs_cm := ds.rs_cm
        rs_sm := ds.rs_sm
        cnt_cm := ds.cnt_cm
        cnt_sm := ds.cnt_sm

        // extract fields from sm (line 2)
        ct := sm.Ciphertext
        at := sm.AuthTag
        cnt_cm_prime := sm.AD[0]
        cnt_sm_prime := sm.AD[1]

        // FtK on rs_cm (line 3)
        k_sm, rs_sm, cnt_sm := FtK(rs_sm, cnt_sm, cnt_sm_prime)

        // VtD (line 5)
        acc, _ := VtD(k_sm, ct, at, [2]uint32{cnt_cm_prime, cnt_sm_prime})

        // if failure return (line 5)
        if !acc {
                return false, ds
        }

        // Update device state (lines 6 and 7)
        ds.rs_cm = rs_cm
        ds.rs_sm = rs_sm
        ds.cnt_cm = cnt_cm
        ds.cnt_sm = cnt_sm

        // return (line 8)
        return acc, ds
}

// Request (Figure 4) computes a new commitment to ps and encrypts it
// into a client message, advancing the client ratchet accordingly.
// It updates ds in place and returns the resulting message.
func Request(ds *DeviceState, ps []byte) (*DeviceState, Message) {
        // extract fields from ds (line 1)
        k_com := ds.k_com
        rs_cm := ds.rs_cm
        cnt_cm := ds.cnt_cm
        cnt_sm := ds.cnt_sm

        // compute commitment (line 2)
        com := PRF(k_com, ps)

        // increment cnt_cm (line 3)
        cnt_cm = cnt_cm + 1

        // KDF (line 4)
        k_cm, rs_cm := KDF(rs_cm)

        // EtM with com as pt (line 5)
        ct, at := EtM(k_cm, com, [2]uint32{cnt_cm, cnt_sm})

        // update ds (line 6)
        ds.rs_cm = rs_cm
        ds.cnt_cm = cnt_cm

        // return (line 7)
        return ds, Message{
                Ciphertext: ct,
                AuthTag:    at,
                AD:         [2]uint32{cnt_cm, ds.cnt_sm},
        }
}

// GetKey (Figure 4) processes a server message carrying k_1, verifies
// authenticity, and on success reconstructs k = k_0 xor k_1. It updates ds in
// place. On failure it returns acc=false and a nil key, representing \bot in
// the specifications.
func GetKey(ds *DeviceState, sm Message) (bool, *DeviceState, []byte) {
        // extract fields from ds (line 1)
        rs_cm := ds.rs_cm
        rs_sm := ds.rs_sm
        cnt_cm := ds.cnt_cm
        cnt_sm := ds.cnt_sm
        k0 := ds.k0

        // extract fields from sm (line 2)
```

```
        ct := sm.Ciphertext
        at := sm.AuthTag
        cnt_cm_prime := sm.AD[0]
        cnt_sm_prime := sm.AD[1]

        // FtK on rs_cm (line 3)
        k_sm, rs_sm, cnt_sm := FtK(rs_sm, cnt_sm, cnt_sm_prime)

        // VtD (line 5)
        acc, pt := VtD(k_sm, ct, at, [2]uint32{cnt_cm_prime, cnt_sm_prime})

        // if failure return (line 5)
        if !acc {
                return false, ds, nil
        }

        // recover key (line 6)
        k := xor(k0, pt)

        // Update device state (lines 7 and 8)
        ds.rs_cm = rs_cm
        ds.rs_sm = rs_sm
        ds.cnt_cm = cnt_cm
        ds.cnt_sm = cnt_sm

        // return on line 9
        return true, ds, k
}

// Clear (Figure 4) advances the server ratchet locally on the client so that
// old server messages are no longer decryptable and cannot be used to
// reconstruct the session key.
func Clear(ds *DeviceState) *DeviceState {
        // extract fields from ds (line 1)
        rs_sm := ds.rs_sm
        cnt_cm := ds.cnt_cm
        cnt_sm := ds.cnt_sm

        // update device state
        _, ds.rs_sm, ds.cnt_sm = FtK(rs_sm, cnt_sm, cnt_cm)

        return ds
}

// GetCountersClient retruns the counters from the client state.
func GetCountersClient(ds *DeviceState) (uint32, uint32) {
        return ds.cnt_cm, ds.cnt_sm
}
```

F.2 Server Code

```
package scheme

import (
        "bytes"
)

// ServerState holds the server's local state:
//
//   - k1: server share used to reconstruct k (k = k0 xor k1)
//   - com: commitment to the real ps
//   - rs_cm, rs_sm: ratchet states for client messages (rs_cm) and server
//     messages (rs_sm)
//   - cnt_cm, cnt_sm: counters for client messages and server messages
//
// Field names use underscores for parity with the spec.
type ServerState struct {
        k1  []byte
        com []byte

        rs_cm, rs_sm   []byte
        cnt_cm, cnt_sm uint32
}

// ServerSetup verifies the client's initial message cm, binds the commitment
// and k_1 into server state, and returns an authenticated ack.  On failure it
// returns acc=false, no server state, and a zero message.
func ServerSetup(kappa []byte, cm Message) (bool, *ServerState, Message) {
        // ratchet states (line 1)
        rs_cm, rs_sm := KDF(kappa)
```

```
        // init counters (line 2)
        cnt_cm, cnt_sm := uint32(0), uint32(0)

        // parse cm (line 3)
        ct := cm.Ciphertext
        at := cm.AuthTag
        cnt_cm_prime := cm.AD[0]
        cnt_sm_prime := cm.AD[1]

        // FtK on rs_cm (line 4)
        k_cm, rs_cm, cnt_cm := FtK(rs_cm, cnt_cm, cnt_cm_prime)

        // VtD on pt = (com, k1) (line 5)
        acc, pt := VtD(k_cm, ct, at, [2]uint32{cnt_cm_prime, cnt_sm_prime})

        // if fail return (line 6)
        if !acc {
                return false, nil, Message{}
        }

        // parse pt (line 7)
        com := pt[:32]
        k1 := pt[32:]

        // increase sm counter (line 8)
        cnt_sm++

        // KDF on rs_sm (line 9)
        k_sm, rs_sm := KDF(rs_sm)

        // encrypt ok (need 16 bytes message for AES-CBC) (line 10)
        ct, at = EtM(k_sm, []byte("server says: OK."), [2]uint32{cnt_cm, cnt_sm})

        // store server state (line 11)
        ss := &ServerState{
                rs_cm:  rs_cm,
                rs_sm:  rs_sm,
                cnt_cm: cnt_cm,
                cnt_sm: cnt_sm,
                com:    com,
                k1:     k1,
        }

        // return (line 12)
        sm := Message{
                Ciphertext: ct,
                AuthTag:    at,
                AD:         [2]uint32{cnt_cm, cnt_sm},
        }

        return true, ss, sm
}

// Response verifies a client request cm, checks the commitment against the one
// established at setup, and returns k_1 if valid. The detect flag is true when
// the request's commitment differs from the commitment to the real personal
// secret com, which the server stores in its the local state.
//
// On authentication failure it returns acc=false and a zero message.
func Response(ss *ServerState, cm Message) (bool, *ServerState, bool, Message) {
        // extract fields from ss (line 1)
        rs_cm := ss.rs_cm
        rs_sm := ss.rs_sm
        cnt_cm := ss.cnt_cm
        cnt_sm := ss.cnt_sm
        com := ss.com
        k1 := ss.k1

        // extract fields from sm (line 2)
        ct := cm.Ciphertext
        at := cm.AuthTag
        cnt_cm_prime := cm.AD[0]
        cnt_sm_prime := cm.AD[1]

        // FtK on rs_cm (line 3)
        k_cm, rs_cm, cnt_cm := FtK(rs_cm, cnt_cm, cnt_cm_prime)

        // VtD (line 4)
        acc, com_prime := VtD(k_cm, ct, at, [2]uint32{cnt_cm_prime, cnt_sm_prime})
```

```
        // if failure, return (line 5)
        if !acc {
                return false, ss, false, Message{}
        }

        // check commitment (line 6)
        detect := !bytes.Equal(com, com_prime)

        // FtK on rs_sm (line 7)
        _, rs_sm, cnt_sm = FtK(rs_sm, cnt_sm, cnt_sm_prime)

        // increment counter (line 8)
        cnt_sm++

        // apply KDF on rs_sm (line 9)
        k_sm, rs_sm := KDF(rs_sm)

        // EtM with k_1 as plaintext (line 10)
        ct, at = EtM(k_sm, k1, [2]uint32{cnt_cm, cnt_sm})

        // Update server state (lines 11 and 12)
        ss.rs_cm = rs_cm
        ss.rs_sm = rs_sm
        ss.cnt_cm = cnt_cm
        ss.cnt_sm = cnt_sm

        // Return (line 13)
        return acc, ss, detect, Message{
                Ciphertext: ct,
                AuthTag:    at,
                AD:         [2]uint32{cnt_cm, cnt_sm},
        }
}

// GetCountersServer retruns the counters from the server state.
func GetCountersServer(ss *ServerState) (uint32, uint32) {
        return ss.cnt_cm, ss.cnt_sm
}
```

F.3 Cryptographic Primitives

```
package scheme

import (
        "crypto/aes"
        "crypto/cipher"
        "crypto/hmac"
        "crypto/rand"
        "crypto/sha256"
        "errors"
        "io"

        "golang.org/x/crypto/hkdf"
)

// Message carries ciphertext, an authentication tag, and the associated-data
// counters cnt_cm and cnt_sm for client messages and server messages,
// respectively.
//
// AD[0] is cnt_cm and AD[1] is cnt_sm.
type Message struct {
        Ciphertext, AuthTag []byte
        AD                  [2]uint32
}

// xor returns the bytewise XOR of a and b up to len(a). It assumes len(a)==len(b).
func xor(a, b []byte) []byte {
        result := make([]byte, len(a))
        for i := range a {
                result[i] = a[i] ^ b[i]
        }
        return result
}

// PRF computes HMAC-SHA256(key, input) and returns the 32-byte digest.
func PRF(key, input []byte) []byte {
        mac := hmac.New(sha256.New, key)
        mac.Write(input)
        return mac.Sum(nil)
}
```

```
// KDF expands the given seed using HKDF-SHA256 with a fixed info string and
// returns two 32-byte keys (k1, k2).
func KDF(seed []byte) ([]byte, []byte) {
        h := sha256.New
        hkdfReader := hkdf.New(h, seed, nil, []byte("hkdf-based-kdf"))

        k1 := make([]byte, 32)
        k2 := make([]byte, 32)
        io.ReadFull(hkdfReader, k1)
        io.ReadFull(hkdfReader, k2)

        return k1, k2
}

// EtM (Figure 4) performs Encrypt-then-MAC: it derives (k_enc, k_mac) from key
// k, encrypts pt under k_enc, and authenticates the ciphertext and associated
// counters under k_mac. It panics only if the underlying encryption errors.
func EtM(k, pt []byte, ad [2]uint32) ([]byte, []byte) {
        k_enc, k_mac := KDF(k)
        ct, err := Encrypt(k_enc, pt)
        if err != nil {
                panic(err)
        }

        tag := MAC(k_mac, ct, ad)
        return ct, tag
}

// VtD (Figure 4) performs Verify-then-Decrypt: it derives (k_enc, k_mac) from
// key k, verifies the tag at over (ct, ad), and if valid decrypts ct under
// k_enc to return the plaintext. On failure it returns (false, nil).
func VtD(k, ct, at []byte, ad [2]uint32) (bool, []byte) {
        k_enc, k_mac := KDF(k)
        if !VerifyMAC(k_mac, ct, ad, at) {
                return false, nil
        }
        plaintext, err := Decrypt(k_enc, ct)
        if err != nil {
                return false, nil
        }
        return true, plaintext
}

// Encrypt encrypts a plaintext whose length is a multiple of 16 bytes using
// AES-CBC without padding. The randomly generated IV is prepended to the
// returned ciphertext.
func Encrypt(key, plaintext []byte) ([]byte, error) {
        if len(plaintext)%aes.BlockSize != 0 {
                return nil, errors.New("plaintext is not a multiple of block size")
        }
        block, err := aes.NewCipher(key[:16])
        if err != nil {
                return nil, err
        }
        iv := make([]byte, aes.BlockSize)
        if _, err := io.ReadFull(rand.Reader, iv); err != nil {
                return nil, err
        }

        ciphertext := make([]byte, len(plaintext))
        mode := cipher.NewCBCEncrypter(block, iv)
        mode.CryptBlocks(ciphertext, plaintext)

        // prepend IV
        return append(iv, ciphertext...), nil
}

// Decrypt reverses Encrypt for AES-CBC ciphertexts with a prepended IV and no
// padding. It returns an error if sizes are inconsistent.
func Decrypt(key, ciphertext []byte) ([]byte, error) {
        if len(ciphertext) < aes.BlockSize {
                return nil, errors.New("ciphertext too short")
        }
        block, err := aes.NewCipher(key[:16])
        if err != nil {
                return nil, err
        }

        iv := ciphertext[:aes.BlockSize]
        ciphertext = ciphertext[aes.BlockSize:]
```

```
        if len(ciphertext)%aes.BlockSize != 0 {
                return nil, errors.New("ciphertext is not a multiple of block size")
        }

        plaintext := make([]byte, len(ciphertext))
        mode := cipher.NewCBCDecrypter(block, iv)
        mode.CryptBlocks(plaintext, ciphertext)

        return plaintext, nil
}

// MAC returns HMAC-SHA256 over ct concatenated with the two counters.
func MAC(key, ct []byte, ad [2]uint32) []byte {
        data := append(ct, byte(ad[0]), byte(ad[1]))
        mac := hmac.New(sha256.New, key)
        mac.Write(data)
        return mac.Sum(nil)
}

// VerifyMAC compares the expected and provided tags.
func VerifyMAC(key, ct []byte, ad [2]uint32, tag []byte) bool {
        expected := MAC(key, ct, ad)
        return hmac.Equal(expected, tag)
}

// FtK (Figure 4) advances the ratchet position from cnt to cnt_prime.  It
// iteratively applies KDF on rs (if cnt_prime>cnt) and returns the derived key
// for ratchet position cnt_prime, the new seed rs, and the updated counter. If
// cnt_prime<=cnt it returns (nil, rs, cnt) to signal that no forward key is
// available.
func FtK(rs []byte, cnt, cnt_prime uint32) ([]byte, []byte, uint32) {
        k := rs
        if cnt_prime > cnt {
                for i := uint32(0); i < cnt_prime-cnt; i++ {
                        k, rs = KDF(rs)
                }

                return k, rs, cnt_prime
        }

        return nil, rs, cnt
}

// RandomKey128 returns a freshly generated 16-byte key.
func RandomKey128() []byte {
        return randomKey(16)
}

// RandomKey256 returns a freshly generated 32-byte key.
func RandomKey256() []byte {
        return randomKey(32)
}

// randomKey returns a freshly generated key of length bytes.
func randomKey(length int) []byte {
        key := make([]byte, length)
        _, err := rand.Read(key)
        if err != nil {
                panic("failed to generate random key: " + err.Error())
        }
        return key
}
```

G Changelog

G.1 10th April 2026

This version fixes the concrete bandwidth complexity of our construction when using 64-bits counters.